Twist and Shout: Faster memory checking arguments via one-hot addressing and increments

Srinath Setty
Microsoft Research

Justin Thaler
a16z crypto research and Georgetown University

Michael Zhu
a16z crypto

Abstract

Memory checking arguments enable an untrusted prover to prove that it is correctly processing reads and writes to memory. They are used widely in modern SNARKs, especially in zkVMs, where the prover proves the correct execution of a CPU including the correctness of memory operations.

We describe a new approach for memory checking, called the *method of one-hot addressing and increments*. We instantiate this method via two families of protocols, called Twist and Shout. Twist supports read/write memories, while Shout targets read-only memories (also known as lookup arguments). Both have logarithmic verifier costs. Unlike prior works, these protocols do not invoke "grand product" or "grand sum" arguments.

Twist and Shout reduce commitment costs in two Jolt settings: 32 RISC-V registers and structured lookup tables of size 2^{64}. With curve-based commitments, Twist at $d = 1$ has about $3.7\times$ fewer nonzero committed values and $1.7\times$ fewer field multiplications than Spice for 32 registers; Shout at $d = 2$ has at least $8\times$ fewer nonzero committed values and $3.4\times$ fewer field multiplications than Lasso for structured tables.

We integrate Twist and Shout into the Jolt zkVM. They speed up the Jolt prover $3\times$ while reducing proof size by about $5\times$.

Contents

1 Introduction **4**
- 1.1 High-Level Ideas and Techniques . . . 6

2 Problem formulation, prior work, and cost comparisons **8**
- 2.1 Background . . . 8
- 2.2 Formulation of the memory checking problem . . . 9
- 2.3 Prior work: Arguments via offline memory checking . . . 11
- 2.4 Overview of the Shout PIOP . . . 16
- 2.5 Overview of the Twist PIOP . . . 17

3 Prior offline memory-checking protocols **19**

4 Preliminaries **22**
- 4.1 The Jolt zkVM and the role of memory checking . . . 22
- 4.2 Multilinear extensions . . . 22
- 4.3 The sum-check protocol . . . 24
- 4.4 Zero-check PIOP . . . 24
- 4.5 One-hot encodings . . . 24

5 The Shout and Twist PIOPs **25**
- 5.1 The Shout PIOP . . . 25
- 5.2 Core Shout PIOP for d = 1 . . . 25
- 5.3 Checking correctness of one-hot encodings . . . 27
- 5.4 Shout for general d . . . 30
- 5.5 One-hot checking in small characteristic . . . 32
- 5.6 The Twist PIOP . . . 37

6 Additional technical preliminaries **40**
- 6.1 Prover costs in elliptic-curve-based SNARKs . . . 40
- 6.2 Linear-time sum-check proving . . . 44
- 6.3 SNARKs and commitment schemes . . . 45
- 6.4 Polynomial IOPs and polynomial commitments . . . 47
- 6.5 An optimization of Gruen for zero-check . . . 48

7 Fast Shout prover implementation (small memories) **49**
- 7.1 Core Shout prover for d = 1 . . . 49
- 7.2 Core Shout prover (general d, small memories) . . . 50
- 7.3 Booleanity-checking and one-hot-encoding-checking . . . 51
- 7.4 Cost summary for the combined Shout prover . . . 56
- 7.5 One-hot-checking prover in small characteristic . . . 56

8 Fast Shout prover for large, structured memories **57**
- 8.1 Sparse-dense sum-check protocol . . . 58
- 8.2 Extension to the Booleanity-checking sum-check when K to the 1/d is much larger than T . . . 60

8.3 Shout's read-checking sum-check . . . 65
8.4 The raf-evaluation sum-check and Hamming-weight-one check when K is much larger than T . . . 65
8.5 Cost summary for Shout when K is much larger than T . . . 66
8.6 A Shout variation with a linear prover dependence on d . . . 66
8.7 Fast prover implementation . . . 67

9 Fast Twist prover implementation 72
9.1 Val-evaluation sum-check . . . 72
9.2 Computing the Less-Than evaluation table . . . 73
9.3 Read-checking and write-checking sum-checks . . . 74
9.4 Cost summary . . . 80

10 Discussion and cost summary 81
10.1 Proof size and verifier costs . . . 81
10.2 Prover time . . . 82
10.3 Remarks on design choices . . . 84

11 Performance evaluation 84

12 Additional experimental results 87
12.1 Comparison of Lasso and Spice to Twist and Shout . . . 87
12.2 Jolt with Twist and Shout for RV64IMAC . . . 94

A Faster SNARKs for non-uniform computation 99
A.1 Overview . . . 99
A.2 Details of SpeedySpartan . . . 100
A.3 Details of Spartan++ . . . 107

1 Introduction

A zero-knowledge virtual machine (zkVM) is a cryptographic protocol that allows an untrusted prover to succinctly prove that it correctly ran some computer program Ψ on a witness.[1] In a zkVM, the computer program is specified to run on a specific CPU architecture, which is also called a virtual machine (VM) or an instruction set architecture (ISA). In a zkVM proof, the prover proves that it correctly ran Ψ cycle-by-cycle.

Generally speaking, zkVMs produce short proofs and have fast verification. Even if the zkVM first produces a proof π that is moderately large, π can be generically shrunk via *SNARK composition*. This means generating a smaller proof π' that establishes that the prover knows a valid π. Accordingly, the bottleneck in zkVM performance is prover runtime. zkVM provers today remain hundreds of thousands of times slower than native execution (i.e., the amount of work done to generate a proof π relative to that required to simply run Ψ with no proof of correctness).

Background on zkVMs and memory checking. zkVM provers prove that they did two things correctly, over and over. They first prove the correct execution of the "fetch-decode-execute" logic of the VM, once per cycle. Second, they prove that reads and writes to registers and RAM were executed faithfully.

A state-of-the-art zkVM, called Jolt [AST24], turns almost everything a VM does into reads and writes to memory. This includes the "fetch-decode-execute" logic of the VM. Roughly speaking, Jolt turns "fetch and decode" into a lookup into a read-only memory storing the program bytecode. Meanwhile, executing each primitive instruction is done via a lookup into a gigantic pre-determined table of size 2^{64}. The original Jolt implementation accomplishes this by using Lasso [STW24], which internally turns this one lookup into a giant table into a short sequence of lookups into smaller tables, and combines the results to get the output of the instruction.

In summary, a key bottleneck for zkVM provers, and for the Jolt prover in particular, is proving that it correctly processed all reads and writes to memory. Prior methods for this task are called *offline memory checking* techniques. These techniques date back to work of Lipton [Lip89, Lip90] and Blum et al. [BEG+91]. In the context of zkVMs, offline memory checking techniques allow a prover to commit to a constant number of values, and perform a constant number of field operations, per memory operation performed by the CPU. The baseline Jolt implementation uses Spice [SAGL18] as its memory-checking argument for registers and RAM, and Lasso [STW24] for read-only memories.

The memory checking problem and prior approaches. In a memory-checking argument, the prover commits to vectors specifying the addresses and values involved in each read and write operation. The goal is to prove that every read returns the value most recently written to the accessed address. Prior approaches reduce this task to a *grand product argument*, which is a significant cost driver for both prover time and proof size. Twist and Shout avoid grand product arguments entirely. See Section 4.1 for further background on Jolt's use of memory checking and Section 2.2 for a formal problem statement.

[1] Historically, many "zkVMs" provided only *succinctness* (short proofs with fast verification) without actually hiding the witness, despite the nomenclature. However, the Jolt zkVM, which is a focus of this work, now achieves true zero-knowledge using the BlindFold technique from HyperNova [KS24a] and Vega [KS25], which adds zero-knowledge to sum-check-based protocols with minimal overhead.

Our contributions. We describe a new approach to memory-checking that improves significantly on offline memory checking techniques, promising substantial end-to-end speedups for zkVM provers. For reasons that will become clear, we call our approach the *method of one-hot addressing and increments*.

We instantiate this method via two different families of protocols, called Twist and Shout. Twist supports read/write memories, while Shout targets read-only memories (also known as lookup arguments). Both families are parameterized by an integer $d \geq 1$, which we call the *dimension parameter*. The smaller the memory, the smaller d can be, and the cheaper each memory operation is for the prover as a result.

Two Jolt settings illustrate the gains: 32 RISC-V registers and structured lookup tables of size 2^{64}. With curve-based commitments, the leading terms in Figure 1 give about 3.7× fewer nonzero committed values and 1.7× fewer field multiplications for Twist with $d = 1$ on 32 registers than Spice, assuming $K = o(T)$. For structured tables, Figure 3 gives at least 8× fewer nonzero committed values and 3.4× fewer field multiplications for Shout with $d = 2$ than Lasso.

Twist and Shout also give the prover a cost profile that more closely matches real CPUs. Unlike offline memory-checking techniques, the method of one-hot addressing is significantly faster for small memories than big ones, and Twist is faster for sequences of memory accesses with high locality, as discussed below. This means that existing compiler toolchains that optimize for program execution time on real CPUs are more likely to also lead to fast zkVM provers once memory-checking is done via Twist and Shout.

Regarding implementation, we integrate Twist and Shout into the Jolt zkVM and the experimental results closely match our analytic estimates: Twist and Shout speed up the Jolt prover 3× while reducing proof size by about 5×. Section 11 has details. On a 32-core CPU, Jolt with Twist and Shout proves over one million cycles per second of a standard instruction set, RV64IMAC (i.e., RISC-V with the Multiplication extension, compressed instructions, atomics, and 64-bit registers). The prover's speed depends primarily on cycle count, with minimal variation observed due to workload-specific factors like the size of the program's RAM or which primitive instructions are executed. Jolt can prove over a million cycles in under two gigabytes of prover space. This suggests that Jolt proofs can be produced in resource-constrained environments such as browsers or mobile devices, and is more than sufficient to capture many important applications. For reference, proving knowledge of a valid ECDSA digital signature with Jolt takes under 250,000 cycles, while SHA3-hashing 40KB of data requires about one million cycles.

Subsequent work has also explained that Jolt with Twist and Shout integrated is highly amenable to a *streaming prover* [NTZ25]. This refers to an implementation of the Jolt prover that uses space sublinear in the number of CPU cycles T that are proved, with minimal slowdown relative to an implementation that uses space linear in T. This streaming approach has many benefits over the prevailing technique of controlling prover memory usage, sometimes called *continuations*, which involves breaking T cycles into small chunks, proving each chunk separately, and recursively aggregating the proofs [Zer23]. This streaming approach offers a path to proving *arbitrarily large statements* in under 2 GB of prover space, while avoiding the complications and overheads of the recursive continuations approach. An implementation of a streaming Jolt prover is underway.

Appendix A presents two further applications of Shout: SpeedySpartan for Plonkish constraints and

Spartan++ for customizable constraint systems (CCS), both of which require the online prover to commit only to its witness.

1.1 High-Level Ideas and Techniques

At the highest level, Twist and Shout leverage the power of the sum-check protocol [LFKN90] to minimize the amount of data the prover has to commit to, and to prove satisfaction of very large but very sparse constraint systems (i.e., a huge number of witness variables, but almost all of them are 0) while only "paying" for the sparsity of the witness (number of non-zeros) rather than the size of the witness.

A simple observation underlying Twist and Shout is that when memory addresses are specified in one-hot form, correctness of reads and writes becomes equivalent to very simple (i.e., rank-one) constraint systems. This observation is already explicit in prior works on lookup arguments [ZBK+22, ZGK+22, STW24, GM24].

These constraint systems can be directly checked for correctness via the standard sum-check-based zero-check polynomial interactive oracle proof (PIOP) for, say, rank-one constraint systems (R1CS) (see Section 4.4 for details). But there is a challenge: the constraint systems are very large, so large that we cannot tolerate a prover that runs in time linear in the number of witness variables, since for T operations into a memory of size K, there are (more than) $K \cdot T$ witness variables. Fortunately, the constraint systems are highly *sparse*: although they involve a huge number of variables, only about T of them are non-zero. This lets us implement the sum-check prover applied to these constraint systems in roughly $O(K+T)$ time rather than $O(K \cdot T)$ time. Roughly speaking, the sum-check prover does not "pay" for 0-variables, and neither does the prover when committing to those variables, at least when using a curve-based commitment scheme.

On top of this, for very large but "structured" read-only memories, where $K = T^C$ with $C \geq 1$, by invoking (and generalizing) the sparse-dense sum-check protocol from the Generalized-Lasso protocol [STW24], we can implement Shout's sum-check prover in time $O(C \cdot T) \ll O(K+T)$. Subsequent work [NTZ25, Appendix A] gives an alternative, called *prefix-suffix sum-check proving*, that can replace the sparse-dense algorithm when the lookup-table polynomial admits a suitable prefix-suffix decomposition.

When using dimension parameter $d > 1$, Twist and Shout further exploit that every "one-hot" vector (meaning a vector in $\{0,1\}^K$ with exactly one entry equal to 1) can be expressed as a tensor product of d smaller one-hot vectors, each of length $K^{1/d}$. We use this to control the negative effects of large memory sizes K, especially the size of the commitment key when using curve-based commitment schemes and the total number of committed bits when using hashing-based commitment schemes. This does come at the cost of higher-complexity constraints in the resulting constraint systems: they change from rank-1 to rank-d. We elaborate further on the dimension parameter d later in this section.

Twist needs to overcome an additional obstacle that threatens to bring in a $\Theta(K \cdot T)$ cost to the prover's runtime in the context of read/write memory. Specifically, a natural first attempt at designing Twist involves the prover committing to the value of *every register at every cycle*. This would be $K \cdot T$ committed non-zero values. Twist addresses this by instead having the prover commit to *increments*: the difference in value for each register cycle-by-cycle. Since at most one

register is written per cycle, this entails committing to only T rather than $K \cdot T$ non-zero values (moreover, all T of these non-zero values are "small" and hence fast to commit to). At the end of the sum-check protocol, when the verifier needs to compute a random evaluation of (the multilinear extension of) the register values themselves, we offload that evaluation to the prover with a second invocation of the sum-check protocol that relates the register values to the increments. At the end of *that* sum-check, the verifier needs to evaluate the multilinear extension of the increments at a random point, which it can do because the increments themselves were committed.

We describe an implementation of the Twist prover that is faster when memory operations are *local*. Here locality means that the same memory cells are accessed repeatedly within short spans of time. The prover can combine these accesses and do less work as a result. For $d = 1$, Section 9.3.3 bounds the total work in terms of the gaps between accesses, including the cost of first accesses and accesses after long gaps. To obtain this saving, the Twist prover binds variables in a different order than the Shout prover: the Twist prover binds "cycle-count-specification" (i.e., time) variables first, rather than "memory-address-specification" (i.e., memory) variables. This ensures that in the first $\log K$ rounds, temporally-close accesses to the same memory cell quickly "coalesce", which is what allows the prover to benefit from locality of memory access. In other words, by binding time variables first, the sparsity (i.e., number of non-zeros) in the vectors the prover is processing falls quickly round-over-round when memory accesses are local, and the prover's runtime in each round grows only with the number of non-zeros in these vectors.

The dimension parameter d: where it comes from and what it buys. When applied to memories of size K, the prover in the simplest variations of Twist and Shout commits to $K-1$ 0s and a single 1 per read or write operation. This makes these protocols especially efficient when combined with elliptic-curve-based commitment schemes, which are uniquely fast at committing to 0s: any committed 0s literally do not alter the commitment so they are free to commit, and committing to a 1 requires a single group operation requiring only 6–7 field operations. This performance profile is an excellent fit for the Jolt zkVM for RISC-V [And17], which currently achieves state-of-the-art prover performance and uses elliptic curve commitment schemes.

However, even when 0s are free to commit to, large memories are problematic for the simplest instantiations of our methods, for two reasons. The first is because the commitment key (which must be stored by the prover) gets very large. The second is that computing polynomial evaluation proofs can become a prover bottleneck.

To address both of these issues, we describe an entire family of protocols in which, for any desired integer $d \geq 1$, the prover only commits to roughly $d \cdot K^{1/d}$ 0s per read or write operation. Using a large value of d comes at a cost: the prover also commits to d 1s per read or write operation, and each committed 1 forces the prover to perform one group operation. Hence, large values of d mean vastly fewer 0s are committed, which keeps the commitment key size bounded and polynomial evaluation proofs fast to compute, but it also means slightly more 1s are committed, which modestly slows down the time to compute commitments. Larger values of d also increase the amount of work the Twist and Shout provers do "outside of commitments", and proof size grows linearly with d too. So one wants to choose d as small as possible, subject to the constraints that the commitment key is not too large and polynomial evaluation proofs are not a major contributor to prover time.

The values of d shown in our cost tables are representative choices; d need not be restricted to a

power of 2. The protocols allow any positive integer d, and when K is not exactly the dth power of an integer, one can pad the address space to have size $\left(\lceil K^{1/d} \rceil\right)^d$.

A final clarification about this cost accounting is that, in the address commitments used by Twist and Shout, the Hamming weight committed per read or write address is fixed: d committed 1s. The actual addresses read or written determine the *locations* of these 1s, not their number. Similarly, as discussed above, some prover implementations, especially the locality-aware Twist prover, can have data-dependent running time because they are faster on more local memory-access patterns. This does not contradict the standard simulation-based definition of zero knowledge, which concerns the proof transcript, but implementations that require prover time to be independent of the memory accesses being proved may need additional countermeasures.

Choosing a value $d \geq 2$ renders our methods suitable for memory sizes K in the millions or billions. It also allows our techniques to be productively instantiated with hashing-based multilinear commitment schemes over binary fields like Binius [DP23], FRI-Binius [DP24], and Blaze [BCF+24]. Even though committing to 0s is not "free" with these schemes, committing to values that are known a priori to be in $\{0, 1\}$ is cheap enough that our results still offer improvements over prior works.[2] For these commitment schemes, d should be set just large enough to ensure that commitment time is not a prover bottleneck.

Since the Twist and Shout provers commit to $dK^{1/d}$ bits per address, to ensure committing with a hashing-based scheme is sufficiently fast, $dK^{1/d}$ should be between a few dozen and a couple hundred bits, with larger memories leading to larger values of d (and more committed bits).

To summarize, in applications we expect d to be between 1 and 16 for both curve-based and binary-field hashing-based commitments. Larger memories necessitate larger values of d.

2 Problem formulation, prior work, and cost comparisons

2.1 Background

Polynomial IOPs and polynomial commitments. A polynomial interactive oracle proof (PIOP) lets the prover send polynomials to a verifier that queries their evaluations at a few points. A polynomial commitment scheme (PCS) makes this interaction succinct: the prover commits to each polynomial and later proves the requested evaluations. Applying the Fiat–Shamir transformation yields a succinct non-interactive argument of knowledge (SNARK). Section 6.4 gives further details.

Costs of elliptic-curve-based commitments. When a polynomial IOP is combined with an elliptic curve commitment scheme (e.g., HyperKZG [ZSC24]), the vast majority of the work done by the prover consists of elliptic curve group operations and finite field operations (over the field $\mathbb{F}$ used by the polynomial IOP, which is also the *scalar field* of an elliptic curve). For small memories, the method of one-hot addressing substantially lowers *both* the number of field operations over $\mathbb{F}$ and the number of group operations performed by the prover.

[2] For memories consisting of just a few cells, such as the 2 registers in the Cairo Virtual Machine [GPR21, Section 2.4] and even the 32 registers in RISC-V, our protocols improve on prior work even when hashing-based commitment schemes over non-binary fields are used.

There are three facts about elliptic curve commitments that the method of one-hot addressing takes heavy advantage of. The first is that committing to 0s is "free". That is, when committing to a vector $v \in \mathbb{F}^\ell$, any i for which $v_i = 0$ literally does not affect the commitment, and can just be "ignored" by the prover when computing the commitment to v. The number of committed 0s does influence some other prover costs arising in the protocol, especially the size of the commitment key, but it does not influence commitment time. Fortunately, the effect of the 0s on these other costs can be controlled by the choice of the dimension parameter d discussed in Section 1.1. Hence, in this informal overview it is both clearest and reasonably accurate to treat committed 0s as literally free for the prover. As discussed in the introduction, "free" here means that zero coefficients do not contribute group operations to the commitment computation; it does not mean that prover running time is totally independent of the actual addresses read or written in every implementation. The second is that *small* (but non-zero) values are cheap (but not free) to commit to. For example, any i such that $v_i = 1$ costs the prover just one group operation when computing the commitment.

For simplicity, in this work we consider any value in $\{0, 1, \ldots, 2^{64} - 1\}$ to be "small". We select the threshold of $2^{64} - 1$ as the limit of smallness for convenience of accounting: most zkVMs today use 32-bit or 64-bit data types, and so registers store arbitrary elements of $\{0, 1, \ldots, 2^{32} - 1\}$ or $\{0, 1, \ldots, 2^{64} - 1\}$. (In this work, we use the terms *register* and *memory cell* interchangeably.) Since the prover has no choice but to commit to the values read from or written to registers, it is convenient to call these values small. But this is also a reasonable notion of smallness: at practical instance sizes, committing to a 64-bit value with an elliptic curve commitment scheme takes a handful of group operations via Pippenger's bucketing algorithm (see, e.g., [EHB22, Section 4]). This is much cheaper than committing to an arbitrary field element, which can require a dozen group operations or more.

The third property of elliptic curve commitment schemes that Twist and Shout exploit is that, for some curve-based schemes, computing evaluation proofs is very fast and commitment keys are of modest size (square-root in the size of the committed polynomial). The most extreme example of this is Hyrax [WTS+18], in which a polynomial evaluation proof is simply a "partial evaluation" of the committed polynomial, which must be computed anyway en route to computing the evaluation itself. A downside of Hyrax is that the commitment is big (square-root in the size of the committed polynomial). Alternatives like Dory [Lee21] aim to achieve the good properties of Hyrax (square-root sized commitment keys, fast evaluation proofs, homomorphism) without the downsides. Our integration of Twist and Shout into the Jolt zkVM uses Dory as the polynomial commitment scheme, see Section 11 for details.

2.2 Formulation of the memory checking problem

Motivation. Most primitive RISC-V instructions read two registers (the "source registers" containing the inputs to the primitive instruction executed at that cycle) and write to one register (the "destination register") that stores the output of the primitive instruction. We denote the source registers ra_1 and ra_2 (ra is short for read address), and the destination register by wa (short for write address).

To force an untrusted prover to prove that it is reading and writing to registers correctly, RISC-V zkVMs demand that the prover commit to

- the register addresses ra_1, ra_2, and wa each cycle.

- the values that the prover claims are returned by the reads to ra_1 and ra_2,
- the value that is written to wa.

The goal of the memory checking problem is to devise a succinct argument that lets the prover prove that indeed the value committed for every read operation is the value most recently written to the relevant register.

Formulation for read/write memories. Say there are K memory cells, which we will refer to as *registers* for short (for example, there are $K = 32$ registers in RISC-V). In the case of read/write memory, let us assume that operations alternate between reads and writes. Let T denote the *number of cycles* being checked. More specifically, T denotes the number of read operations in the case of read-only memory. For read/write memory, each cycle consists of a read followed by a write, so there are T reads and T writes.

The memory-checking problem begins with the prover sending commitments to four vectors: raf, waf, rv and wv. We denote the j'th entry of these vectors with function notation, i.e., the j'th entry of raf is $\mathsf{raf}(j)$.

The j'th entries of raf and waf specify the address of the registers read and written in cycle j. In the context of zkVMs (and in typical offline-memory checking procedures like Spice), each register is indexed by one field element, and so raf and waf are size-T vectors of field elements, with $\mathsf{raf}(j), \mathsf{waf}(j) \in \mathbb{F}$ specifying the register addresses read and written at cycle j. Here, any injective mapping from registers to field elements suffices for the indexing. In large prime-order fields, the K field elements used to address the registers can simply be $\{0, 1, \ldots, K-1\}$; for simplicity, this is the mapping we use unless otherwise specified. Section 4.5 gives the address encoding for binary fields. We use the letter f at the end of raf and waf to reflect that these are addresses specified via one field element per address. The j'th entry of rv specifies the value returned by the j'th read operation, and wv specifies the value written by the write operation. rv and wv are vectors in $\mathbb{F}^T$ (i.e., registers store field elements).

The memory checking problem then demands that the prover establish that $\mathsf{rv}(j)$ equals $\mathsf{wv}(j')$ where $j' < j$ denotes the largest cycle number strictly less than j such that $\mathsf{waf}(j') = \mathsf{raf}(j)$. If no such j' exists, then it is required that $\mathsf{rv}(j) = 0$. Conceptually, this means that all registers are initialized to contain value 0, and every read to a register is required to return the value most recently written to that register.

It is also possible to demand that the contents of memory be initialized to a set of values that is not identically 0. In this case, either the vector of initial memory values (one value per cell) is committed by an honest party, or is simply public.

Read-only memories. Our formulation of the memory-checking problem for read-only memories is identical to read/write memories, except that the contents of the memory cells are public (i.e., known to both the verifier and the prover) and each cycle involves only a read operation, rather than a read followed by a write. We sometimes refer to the contents of the read-only memory as the *lookup table*.

One-hot encoding of addresses. Recall that in the above formulation of the memory-checking problem, each address is indexed by a single field element. This is how memory addresses are naturally specified in zkVM contexts. For example, in RISC-V, when performing a load or store from RAM, the relevant address is the value stored in the first source register, plus the "immediate" value for that instruction. Each zkVM register naturally stores a single field element, and the immediate value for any instruction is naturally represented by a single field element.

However, internal to the Twist and Shout proof systems, the register addresses must actually be specified via one-hot encoding. This means that within the Twist and Shout protocols, each address is a unit vector in $\{0,1\}^K \subseteq \mathbb{F}^K$. Accordingly, we introduce two vectors $\mathsf{ra}, \mathsf{wa} \in \mathbb{F}^{T \cdot K}$ that, for a sequence of T memory operations, specify all T memory addresses via one-hot encoding. We think of ra and wa as consisting of T rows each of length K. If the prover is honest, then the j'th row of ra, denoted $\mathsf{ra}(j)$, specifies via one-hot encoding the register read at cycle j, and similarly for wa; equivalently, $\mathsf{ra}(j)$ is the one-hot encoding of $\mathsf{raf}(j)$, and $\mathsf{wa}(j)$ is the one-hot encoding of $\mathsf{waf}(j)$. So if register ℓ is read at cycle j, then $\mathsf{ra}(j)$ is the ℓ'th unit vector $e_\ell \in \{0,1\}^K$. We denote the k'th entry of this vector by $\mathsf{ra}(k, j)$.

Because zkVMs naturally specify addresses via a single field element, when used within zkVMs, memory-checking arguments do need to give the verifier "access" to the vectors raf and waf, in which each address is specified via a single field element. In particular, the zkVM verifier needs to be able to evaluate the *multilinear extension polynomials* $\widetilde{\mathsf{raf}}$ and $\widetilde{\mathsf{waf}}$ at a random point. Twist and Shout support this. Even though $\widetilde{\mathsf{raf}}$ and $\widetilde{\mathsf{waf}}$ are *not* committed by the Twist and Shout provers (only $\widetilde{\mathsf{ra}}$ and $\widetilde{\mathsf{wa}}$ are), we still give a way for the verifier to evaluate $\widetilde{\mathsf{raf}}$ and $\widetilde{\mathsf{waf}}$ at a random point.

To use a term coined in work on Binius [DP23] and HyperPlonk [CBBZ23a], Twist and Shout can be integrated into zkVMs by treating $\widetilde{\mathsf{raf}}$ and $\widetilde{\mathsf{waf}}$ as *virtual polynomials*: they are not explicitly committed by the prover, but can still be evaluated by the verifier at any point, by asking the prover to provide the requested evaluation and then prove that the evaluation is correct. This is done by expressing the needed $\widetilde{\mathsf{raf}}$ and $\widetilde{\mathsf{waf}}$ evaluations in terms of $\widetilde{\mathsf{ra}}$ and $\widetilde{\mathsf{wa}}$, and then applying the sum-check protocol to compute this expression. This effectively reduces the verifier's task of evaluating $\widetilde{\mathsf{raf}}$ and $\widetilde{\mathsf{waf}}$ at a point, to the task of evaluating $\widetilde{\mathsf{ra}}$ and $\widetilde{\mathsf{wa}}$ at a different point. These evaluations can be obtained directly from the commitments to $\widetilde{\mathsf{ra}}$ and $\widetilde{\mathsf{wa}}$. See Section 5.3 for details.

2.3 Prior work: Arguments via offline memory checking

Memory-checking procedures such as Spice [SAGL18] work by reducing the task of proving that all reads and writes were processed correctly, to the task of proving two related vectors $a \in \mathbb{F}^\ell$ and $b \in \mathbb{F}^\ell$ are permutations of each other. The precise value of ℓ differs amongst different memory-checking procedures, but is typically $O(T + K)$. Confirming that a and b are permutations of each other is done via "Lipton's trick" [Lip89, Lip90]: the verifier picks a random $r \in \mathbb{F}$, and confirms that

$$\prod_{i=1}^{\ell}(a_i - r) = \prod_{i=1}^{\ell}(b_i - r). \tag{1}$$

Equation (1) clearly holds if a and b are permutations of each other, and if they are not, it holds with probability at most $\ell/|\mathbb{F}|$ over the random choice of r.

Offline memory-checking procedures confirm that Equation (1) holds using a *grand product argument*: a SNARK for computing the product of many committed values. Many memory-checking procedures in the literature invoke grand product arguments that interpret the committed vectors as univariate polynomials, and prove that a certain divisibility relationship holds between the committed polynomials. These arguments can lead to excellent verifier costs (often with proofs consisting of only a constant number of group elements), but generally lead to slow provers.

The grand product arguments used by Spice as instantiated in Jolt (as with the method of one-hot addressing) interpret committed vectors as *multilinear polynomials* and invoke the sum-check protocol of Lund, Fortnow, Karloff, and Nisan [LFKN90]. There are several such grand product arguments one can choose from, offering various tradeoffs between prover time and verifier costs [Tha13, Set20, SL20]. Depending on which grand product argument it invokes, Spice can have proofs of size between $O(\log^2 n)$ and $O(\log n)$ field elements, where n is the number of memory operations proven. The bigger the proofs, the faster the Spice prover.

There are only a handful of known approaches to offline memory-checking for read/write memory [SAGL18, ZGK$^+$18]. For *read-only* memories there is somewhat more diversity in solutions. Memory-checking arguments for read-only memories are also known as *lookup arguments*, and examples include Arya [BCG$^+$18], plookup [GW20b], cq [EFG22], Lasso [STW24], LogUp [Hab22, PH23], and others.

Baselines for read-only memory: Small or unstructured tables. Lasso's prover [STW24] commits to $3T + \min\{K, T\}$ small values in total, and performs about $12T + 12K$ field operations within the grand product argument. Inspired by the use of Thaler's grand product argument [Tha13] in Spark [Set20] and Lasso [STW24], LogUpGKR [PH23] combines the LogUp lookup argument [Hab22] with the GKR protocol. LogUpGKR's prover commits $2T + \min\{K, T\}$ small values, one T term fewer than Lasso in this cost model. But LogUpGKR's $21T + 21K$ field-operation count is about $1.75\times$ Lasso's $12T + 12K$ count. This stems from LogUpGKR's use of a "grand sum of rational values" in place of a grand product: summing two rationals, $a/b + c/d$, requires three products, namely $a \cdot d$, $b \cdot c$, and $b \cdot d$, to ultimately derive $a/b + c/d = (ad + bc)/bd$.

We state the costs of these baselines in Figures 1–3. For all baselines, we incorporate all of the most recent known optimizations to the prover in the GKR protocol and Thaler's grand product argument [DT24, Gru24]. Further, we assume that the multilinear extension of the table values is efficiently evaluable by the verifier (as is the case for all lookup tables in Jolt). Other relevant baselines are FLI [GM24], a folding scheme for lookups, and Proofs for Deep Thought [BC25], a folding scheme for read/write memory.

Baselines for read-only memory: Gigantic, structured tables. Lasso [STW24] gives two different approaches to performing lookups into gigantic, structured tables. By gigantic, we mean the table size K is on the order of T^C for some integer $C \geq 2$. One, called Generalized-Lasso, applies to any MLE-structured table but requires the prover to commit to c random field elements, which is generally a bottleneck for the prover. The other, called simply Lasso, applies to tables satisfying a natural decomposability property, which roughly means that one lookup into the giant table can be turned into $O(C)$ lookups into "subtables" of size at most $T^{1/C}$ (Lasso then applies a "base" lookup argument to prove validity of the $O(C)$ subtable lookups). Lasso does not require the

prover to commit to any random field elements. Other works have considered using different "base" lookup arguments within this approach [Dor24].

To prove that primitive instructions were correctly executed, Jolt applies Lasso to gigantic decomposable tables (namely, the entire evaluation table of the relevant primitive instruction). However, there are overheads that come from the interaction of how zkVMs work with Lasso's use of decompositions. Specifically, the addresses of the subtable lookups have two "parts", with one part coming from the first input to the primitive instruction, and the other coming from the second. This forces inputs to be decomposed into smaller chunks than would otherwise be necessary. For example, to compute the bitwise XOR of two 32-bit inputs x and y, Jolt-with-Lasso splits x and y each into four 8-bit chunks, and uses one subtable lookup (into a subtable of size 2^{16}) to compute the bitwise XOR of each chunk of x with the corresponding chunk of y. So even though the subtable addresses are 16 bits, x and y must each be decomposed into only 8-bit chunks. Furthermore, these committed 8-bit chunks must all be range-checked to confirm they are indeed in $\{0, 1, \ldots, 2^8 - 1\}$. As we will see, Shout avoids any explicit table decomposition, and hence avoids the above overheads when used in the context of a zkVM such as Jolt.

When Shout is applied to gigantic structured tables, the Shout prover does leverage decomposability-like structural properties to ensure it can quickly compute its messages within the sum-check protocol invoked by Shout. So an alternative perspective on the relationship between Shout and Lasso is that Shout "gets the table decompositions out of the protocol itself" and confines the decomposition "inside the head of the efficient prover". That is, whereas Lasso forces the prover to commit to decomposed inputs and forces the verifier to be aware of the decomposition, in Shout table decompositions (or related structural properties) are only used internally by the prover to compute sum-check messages quickly.

Closest related work: Generalized-Lasso. Shout can be viewed as a natural generalization or improvement of the Generalized-Lasso protocol from [STW24]. Shout with $d = 1$ is in fact equivalent to Generalized-Lasso, except that Shout uses any standard (i.e., dense) polynomial commitment scheme to commit to the one-hot encodings of addresses, rather than the sparse polynomial commitment scheme Spark [Set20] invoked by Generalized-Lasso. Shout thereby avoids Generalized-Lasso's need to commit to random field elements (which comes from Spark). However, setting $d = 1$ only works for very small lookup tables due to the constraints described in Section 1.1. At the other extreme, Shout with $d = \log K$ (the largest meaningful value of d) is roughly equivalent to combining Generalized-Lasso with a different sparse polynomial commitment scheme called "Spark-Naive" [Set20]. Setting d this large brings high costs.

From this standpoint, Shout can be viewed as an improvement in sparse polynomial commitment schemes rather than in lookup arguments. Shout effectively identifies a family of sparse polynomial commitment schemes, with all schemes in the family avoiding Spark's need to commit to random field elements. There is a different scheme for each value of $d \geq 1$, where committing to bigger, sparser polynomials calls for higher settings of d. Shout then simply combines Generalized-Lasso with this family of sparse polynomial commitment schemes.

Memory checker	Nonzero committed values	$\mathbb{F}$ multiplications	Proof size
Spice	$5R + 6W + 2K \approx 11T$	$80T + 80K \approx 80T$	$O(\log^2 T)$
Twist ($d = 1$)	$R + 2W = 3T$	$(5\log(K) + 22.5)T + O(K \log K)$	$O(\log T)$
Twist ($d = 2$)	$2R + 3W = 5T$	$(5\log(K) + 38.5)T + O(K)$	$O(\log T)$

Figure 1: Prover costs for Twist: read/write memory checking for $R = T$ reads and $W = T$ writes ($R + W = 2T$ memory operations in total). The approximations in the Spice row assume $K = o(T)$. The leading Twist coefficients dominate when $K \log K = o(T)$ for $d = 1$, or $K = o(T)$ for $d = 2$; the displayed bounds retain their additive terms. The Twist field-work bounds combine the alternative read- and write-checking costs with the $7.5T + K$ value-evaluation algorithm of Section 9.1, and include $6dT$ for Booleanity checking. For $d = 1$, the Twist prover performs many fewer field operations if memory accesses are local. Section 9.3.3 bounds this saving in terms of the gaps between accesses to the same cell. The dimension parameter d is discussed in Section 1.1; Section 11 describes the experimental choices of d.

Memory checker	Nonzero committed values	$\mathbb{F}$ multiplications	Proof size
Lasso	$3T + \min\{K, T\}$	$12T + 12K$	$O(\log^2 T)$
LogUpGKR	$2T + \min\{K, T\}$	$21T + 21K$	$O(\log^2 T)$
Shout ($d = 1$)	T	$4T + O(K \log K)$	$O(\log T)$
Shout ($d = 2$)	$2T$	$11T + O(K)$	$O(\log T)$

Figure 2: Prover costs for Shout (read-only memory checking for T reads, into a memory of size K). All committed values are small; reported Shout costs include field multiplications required to prove that committed addresses are valid one-hot encodings (including Booleanity-checking). The leading Shout coefficients dominate the displayed additive terms when $K \log K = o(T)$ for $d = 1$, or $K = o(T)$ for $d = 2$.

Memory checker	Parameters	Nonzero committed values	$\mathbb{F}$ multiplications	Proof size
Lasso	–	$\geq 16T$	$\geq 144T$	$O(\log^2 T)$
Shout	$d = 2, C = 4, c = 2$	$2T$	$42T$	$O(\log T)$
Shout	$d = 4, C = 4, c = 1$	$4T$	$59T$	$O(\log T)$
Shout	$d = 8, C = 8, c = 1$	$8T$	$147T$	$O(\log T)$

Figure 3: Prover costs for Shout when doing T lookups into an appropriately structured read-only memory of size K such that $K^{1/C} = o(T)$. All committed values are small; reported Shout costs include field multiplications required to prove that committed addresses are valid one-hot encodings (including Booleanity-checking). For comparison, we report (optimistic estimates of) the costs of Lasso as used in Jolt, whereby each lookup into the table of size $K \approx 2^{64}$ is decomposed into between 4 and 10 lookups into "subtables" of size 2^{16}, and where committing to the addresses for those subtables necessitates committing to 2 small values per subtable address and range-checking each such value.

Memory checker	Mem. size	Committed Bits	$\mathbb{F}$ multiplications	Proof size
Lasso	$K = 2^{64}$ (structured)	$\geq 928T$	$\geq 194T$	$O(\log^2 T)$
Shout ($d = 16$)	$K = 2^{64}$ (structured)	$256T$	$O(T)$	$O(\log T)$
Spice	$K = 32$	$421T$	$95T$	$O(\log^2 T)$
Twist ($d = 1$)	$K = 32$	$128T$	$O(T)$	$O(\log T)$

Figure 4: Amount of committed data measured in bits for Twist and Shout vs. prior work for two important applications: the 32 read/write registers of the RISC-V instruction set, and lookups into structured tables of size 2^{64} used for proving correct primitive instruction execution in Jolt [AST24]. Bits of committed data is the relevant measure of commitment costs when using binary-field hashing-based multilinear commitments like Binius [DP23, DP24] and Blaze [BCF+24]. T denotes the number of reads in the read-only case, and the number of alternating reads and writes in the read/write case ($2T$ total memory operations). We assume values read or written are 32 bits as in RISC-V, and that timestamps used in Spice are up to 32 bits (which is roughly sufficient for values of T up to 2^{30}). For Twist and Shout, we omit the field multiplications needed to prove Booleanity, as the commitment schemes themselves ensure this. For prior work (Lasso and Spice), we incorporate the slightly larger prover costs these protocols incur in the setting of small-characteristic fields [DP23, Section 4.4]. The $O(T)$ bounds include the additional one-hot check for small characteristic. Section 5.5 gives an explicit upper bound of $8.5T + O(K^{1/d} + \log T)$ multiplications per one-hot factor: at most $136T + o(T)$ for the 16 Shout factors and $17T + o(T)$ for Twist's read and write addresses. These are additional to the core sum-checks; commitment computation and openings are separate. Here K and the decomposition parameters are fixed at the values shown.

2.4 Overview of the Shout PIOP

Shout is targeted at read-only memories. Let $\mathsf{Val}(k)$ denote the fixed value of memory cell k. We first explain the case $d = 1$, in which the prover commits directly to the one-hot read-address polynomial $\widetilde{\mathsf{ra}}$, and then describe how the protocol uses smaller address encodings when $d > 1$. We use tildes for multilinear extensions and index addresses and cycles by Boolean strings of lengths $\log K$ and $\log T$, respectively.

The sum-check protocol in a nutshell. From the verifier's perspective, sum-check reduces a claim about a sum

$$\sum_{x \in \{0,1\}^\ell} g(x)$$

to an evaluation of $g(r)$ at a random field point $r \in \mathbb{F}^\ell$ chosen during the protocol. The verifier must then obtain this evaluation, either by computing it directly or by querying the polynomials defining g. When g has constant degree in each variable, communication is $O(\ell)$ field elements and soundness error is $O(\ell/|\mathbb{F}|)$. Section 4.3 gives the details.

The core protocol for $d = 1$. As observed in Baloo [ZGK+22] and Lasso [STW24], the lookups are valid if and only if, for every Boolean cycle j,

$$\mathsf{rv}(j) = \sum_{k \in \{0,1\}^{\log K}} \mathsf{ra}(k, j) \cdot \mathsf{Val}(k).$$

Indeed, $\mathsf{ra}(k, j)$ is 1 for the cell read at cycle j, and 0 for all other cells, so the sum returns precisely the value at the selected address. Taking multilinear extensions gives

$$\widetilde{\mathsf{rv}}(r_{\text{cycle}}) = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, r_{\text{cycle}}) \cdot \widetilde{\mathsf{Val}}(k).$$

The right-hand side is multilinear in r_{cycle} and agrees with the read values at Boolean cycles. It therefore defines the virtual read-value polynomial $\widetilde{\mathsf{rv}}$, without requiring the prover to commit separately to those values.

To check a claimed evaluation $\widetilde{\mathsf{rv}}(r_{\text{cycle}})$, the verifier applies sum-check to this sum over addresses. At the end, the verifier needs $\widetilde{\mathsf{ra}}(r_{\text{address}}, r_{\text{cycle}})$ and $\widetilde{\mathsf{Val}}(r_{\text{address}})$ for the random address point r_{address} chosen during sum-check. The first evaluation comes from the commitment to $\widetilde{\mathsf{ra}}$. For an *MLE-structured* lookup table, the verifier can compute the second in $O(\log K)$ time. Alternatively, the table polynomial can be committed in advance by an honest party, and its evaluation obtained from that commitment.

This is the entire core Shout PIOP for $d = 1$, specified in Figure 5. It is the core Generalized-Lasso protocol [STW24], using a standard multilinear polynomial commitment for $\widetilde{\mathsf{ra}}$ in place of Spark. The one-hot property of the committed addresses is checked separately, as described below.

Smaller address encodings for $d > 1$. Write an address as $k = (k_1, \ldots, k_d)$, with each $k_i \in \{0,1\}^{\log(K)/d}$. Instead of one length-$K$ one-hot vector per address, the prover commits to d

length-$K^{1/d}$ one-hot vectors. For each Boolean cycle j, $\mathsf{ra}_i(k_i, j)$ is 1 exactly when k_i is the ith part of the address read at that cycle. Thus, on Boolean inputs,

$$\mathsf{ra}(k, j) = \prod_{i=1}^{d} \mathsf{ra}_i(k_i, j).$$

The prover commits to the multilinear extensions $\widetilde{\mathsf{ra}}_1, \ldots, \widetilde{\mathsf{ra}}_d$. The full address polynomial $\widetilde{\mathsf{ra}}$ is now virtual. Its multilinear extension is obtained by interpolating over the cycle variable:

$$\widetilde{\mathsf{ra}}(k, z) = \sum_{j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(z, j) \cdot \prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(k_i, j).$$

Here $\widetilde{\mathsf{eq}}$ is the multilinear extension of equality on Boolean strings. The interpolation is needed because the product of the d factors is generally not multilinear in the cycle variables.

The read-checking sum-check is unchanged. When it requires $\widetilde{\mathsf{ra}}(r_{\text{address}}, r_{\text{cycle}})$, the verifier applies a second sum-check to

$$\widetilde{\mathsf{ra}}(r_{\text{address}}, r_{\text{cycle}}) = \sum_{j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r_{\text{cycle}}, j) \cdot \prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(r_{\text{address}}^{(i)}, j),$$

where $r_{\text{address}}^{(i)}$ is the ith block of address coordinates. This reduces the claim to one evaluation of each committed $\widetilde{\mathsf{ra}}_i$, together with an equality-polynomial evaluation the verifier computes itself. Figure 7, including Equation (16), specifies these two sum-checks.

Completing the protocol and implementing the prover. The remaining checks ensure that each committed address encoding selects exactly one cell and give the verifier access to the address as a single field element. Section 5.3 describes these checks; Section 5.5 gives the additional check needed in small characteristic. Efficient prover implementations exploit the sparsity of the address polynomials. For gigantic structured tables, they also exploit the table's structure through sparse-dense sum-check, avoiding the need to materialize the entire table (Sections 7 and 8).

2.5 Overview of the Twist PIOP

Twist must ensure that each read returns the value most recently written to the selected cell. We explain how it does this using three sum-checks, starting with one-hot address encodings that are assumed to be correct. For clarity, first take $d = 1$, so the prover commits directly to the read- and write-address polynomials $\widetilde{\mathsf{ra}}$ and $\widetilde{\mathsf{wa}}$. We use tildes for multilinear extensions, and index addresses and cycles by Boolean strings of lengths $\log K$ and $\log T$, respectively.

A direct approach would have the prover commit to all KT memory values: the contents of every cell at every cycle. Yet only one cell is written at each cycle. Twist exploits this by committing to just one increment per cycle and deriving the memory contents from those increments. The verifier then uses the read-checking and write-checking sum-checks to obtain the implied read and write values, and a third sum-check to evaluate the memory contents as needed.

Representing memory contents by increments. For each Boolean cycle $j \in \{0,1\}^{\log T}$, let $\mathsf{Inc}(j)$ be the field difference between the value written at that cycle and the previous value of the cell being written. The prover commits to the multilinear extension $\widetilde{\mathsf{Inc}}$ of these T increments. For a Boolean address $k \in \{0,1\}^{\log K}$, the write-address polynomial $\widetilde{\mathsf{wa}}(k,j)$ equals 1 if cell k is written at cycle j, and 0 otherwise. Thus $\mathsf{Inc}(j)\widetilde{\mathsf{wa}}(k,j)$ is the increment applied to cell k at that cycle.

Starting from zero-initialized memory, the value $\mathsf{Val}(k,j)$ stored at cell k at the start of cycle j is the sum of the increments applied to that cell at earlier cycles. Its multilinear extension is therefore

$$\widetilde{\mathsf{Val}}(k,j) = \sum_{j' \in \{0,1\}^{\log T}} \widetilde{\mathsf{Inc}}(j') \cdot \widetilde{\mathsf{wa}}(k,j') \cdot \widetilde{\mathsf{LT}}(j',j).$$

This is Equation (26) in Figure 9. On Boolean cycle indices, $\widetilde{\mathsf{LT}}(j',j)$ is 1 exactly when j' precedes j, and 0 otherwise, so the sum includes precisely the earlier increments to cell k. The displayed identity also holds at non-Boolean (k,j): its right-hand side is multilinear in (k,j) and agrees with Val on Boolean inputs. It defines $\widetilde{\mathsf{Val}}$ as a virtual polynomial, determined by the committed increments and address encodings without a separate commitment to the entire memory history.

Obtaining read and write values. The read-address polynomial $\widetilde{\mathsf{ra}}(k,j)$ similarly equals 1 if cell k is read at Boolean cycle j, and 0 otherwise. At that cycle, the read value is $\mathsf{Val}(k,j)$ at the cell selected by the read address. The write value is $\mathsf{Val}(k,j) + \mathsf{Inc}(j)$ at the cell selected by the write address. These values are determined by the increments and addresses, so neither the read values nor the write values require a separate commitment. Taking their multilinear extensions gives

$$\widetilde{\mathsf{rv}}(r') = \sum_{\substack{k \in \{0,1\}^{\log K} \\ j \in \{0,1\}^{\log T}}} \widetilde{\mathsf{eq}}(r',j) \cdot \widetilde{\mathsf{ra}}(k,j) \cdot \widetilde{\mathsf{Val}}(k,j),$$

$$\widetilde{\mathsf{wv}}(r'') = \sum_{\substack{k \in \{0,1\}^{\log K} \\ j \in \{0,1\}^{\log T}}} \widetilde{\mathsf{eq}}(r'',j) \cdot \widetilde{\mathsf{wa}}(k,j) \cdot \big(\widetilde{\mathsf{Val}}(k,j) + \widetilde{\mathsf{Inc}}(j)\big).$$

These are Equations (27) and (28) in Figure 9. Here $\widetilde{\mathsf{eq}}$ is the multilinear extension of equality on Boolean strings; its factors interpolate the Boolean-cycle values to the verifier's desired evaluation points $r', r'' \in \mathbb{F}^{\log T}$. The read-checking and write-checking sum-checks verify claimed evaluations by applying sum-check to these two sums. Running them with shared challenges makes both reduce to evaluations of the address polynomials and $\widetilde{\mathsf{Inc}}$, together with a single evaluation $\widetilde{\mathsf{Val}}(r_{\text{address}}, r_{\text{cycle}})$.

Evaluating the virtual memory polynomial. Since $\widetilde{\mathsf{Val}}$ is not committed directly, the verifier obtains this last evaluation with a third sum-check, the $\widetilde{\mathsf{Val}}$-evaluation sum-check. Substituting the evaluation point into the defining identity above gives

$$\widetilde{\mathsf{Val}}(r_{\text{address}}, r_{\text{cycle}}) = \sum_{j' \in \{0,1\}^{\log T}} \widetilde{\mathsf{Inc}}(j') \cdot \widetilde{\mathsf{wa}}(r_{\text{address}}, j') \cdot \widetilde{\mathsf{LT}}(j', r_{\text{cycle}}).$$

Applying sum-check to this sum reduces the claim to evaluations of $\widetilde{\mathsf{Inc}}$, $\widetilde{\mathsf{wa}}$, and $\widetilde{\mathsf{LT}}$: the first comes from its commitment, the second from the committed one-hot address encodings, and the verifier evaluates $\widetilde{\mathsf{LT}}$ itself. This completes the reduction from claimed read and write values to the committed increments and addresses.

One-hot address encodings. We use the same one-hot checks as Shout, and the same address-virtualization sum-checks when $d > 1$ (Section 2.4).

Section 5.6 gives the full Twist PIOP and its correctness statement. Section 9 describes efficient implementations of the Twist prover.

3 Prior offline memory-checking protocols

Read/write memories. For read/write memories, there are two high-level approaches. In the first approach, which we call the *reordering-based approach*, the prover commits to a *re-ordering* of the memory operations (both reads and writes) such that they are grouped by which memory cell is accessed, and ordered by timestep (i.e., cycle) within each group. Confirming that this "second" committed vector of memory operations is indeed a reordering of the first amounts to a permutation check. It must be proved that within each group the memory operations are indeed ordered by timestep. Given this ordering, it is then relatively straightforward to prove that each write to that cell returns the value most recently written to that cell. This approach was first implemented in [BSCG$^+$13, WSR$^+$15], but dates back to the much older work [Rob91, BCGT13]. For proving a permutation check, these old works employ a permutation network (a technique originally motivated by non-interactivity requirements of the PCP model). The approach was refined in vRAM [ZGK$^+$18] with the use of interaction to use fingerprinting techniques [Lip89, Lip90] (see Section 2.3) rather than a permutation network. See also a recent work by Ozdemir et al. [OLB24] that uses the reordering-based approach.

The second approach, which we refer to as "reordering-free approach", was first implemented and made practical by Spice [SAGL18], building on the older work of Blum et al. [BEG$^+$91]. Spartan [Set20] refined this approach to use fingerprinting-based multiset equality check, and employed it for proving sparse polynomial evaluations. This approach is further refined in Jolt [AST24], to use lookup arguments for establishing timestamp checks in the memory checking algorithm. Recently, Yang and Heath [YH23] describe an approach that is essentially the same as Spice's for proving memory operations.

In this approach, the prover does not reorder the memory operations, but commits to some extra data for each read and write to memory. Specifically, each cell is augmented to store not only a value but a timestamp, which is returned with each read and updated with each write. Also, each read to memory is followed by a "dummy write" operation that writes the returned value back to the same cell with the timestamp updated. Similarly, each write operation is preceded by a "dummy read" operation to that cell. The point of having the prover commit to this extra data (timestamps, dummy reads, and dummy writes) is that it ensures the set of "read tuples" (i.e., address, value, timestamp triples) returned by all read operations is a permutation of the set of all "write tuples" if and only if the value returned by every read operation is indeed the value most recently written to that cell.

At least as implemented in Jolt, the way that timestamps are computed on write operations in Spice necessitates range-checking two values. Specifically, Spice maintains a *global timestamp* gt. At the end of cycle i, gt is set to i. Here, we use the term cycle to refer to one read operation followed by one write operation, where one of the two operations is a "dummy" operation as described above.

In Spice, when a read operation returns a value along with timestamp t, the associated write

operation (if the prover is honest) uses timestamp $t' = \max\{t+1, \mathsf{gt}+1\}$. Intuitively, the point of using this timestamp update procedure is to prevent "time travel attacks", where a cheating prover returns on an early read a value that will only be written in the future to that cell, and returns on a later read a value that was written in the distant past to that cell. The timestamp update procedure in Spice forces the timestamps to increase monotonically as the cycles proceed, preventing this attack.

If the prover is honest, the timestamp t returned by a read operation at cycle i is always less than or equal to $\mathsf{gt} = i - 1$, and under this guarantee, the prescribed write timestamp t' simply equals $\mathsf{gt} + 1$. To ensure that t is indeed less than or equal to gt, Jolt confirms that both t itself and $\mathsf{gt} - t$ are in the range $\{0, \ldots, T-1\}$, where T is (an upper bound on) the total number of cycles that the zkVM is run for.

These "read-timestamp range checks" are implemented in Jolt by applying the Lasso lookup argument [STW24] to the table $\{0, \ldots, T-1\}$. In this application of Lasso, the prover commits to two small values per lookup, and hence four small values total. These four values also contribute one factor each (so four factors in total) to a grand product that must be proved within the invocation of Lasso.

Historical context on permutation arguments. The problem of proving that two committed lists are permutations of each other has a rich history predating its use in memory checking. Two main paradigms emerged from the shuffle proof literature for mix-nets. Furukawa and Sako [FS01] introduced the first, representing the shuffle via a permutation matrix and proving the committed matrix satisfies algebraic conditions that characterize permutation matrices. The sparsity of permutation matrix rows (each row is a one-hot vector with a single nonzero entry) implicitly yields an $O(1)$-per-row commitment cost in their protocol [Gro26], giving $O(n)$ total cost to commit to the full $n \times n$ matrix. Neff [Nef01] introduced the second paradigm, based on the polynomial identity $\prod_i (a_i - r) = \prod_i (b_i - r)$ for a random verifier challenge r (as in Equation (1)), which traces back to Lipton [Lip89, Lip90]. Groth and Lu [GL07] and Bayer and Groth [BG12] subsequently improved on the grand-product approach, eventually achieving sublinear proof size. All of the early shuffle argument protocols have linear-size proofs and are comparatively simple—they do not require the more involved machinery of Twist/Shout.

Twist and Shout revisit the one-hot/permutation-matrix approach but combine it with the sum-check protocol to prove memory-checking constraints directly, rather than first reducing to a permutation check. We further extend the sparsity observation to tensor-structured one-hot vectors (Section 5.4): an n-dimensional one-hot vector can be written as a tensor product of d vectors each of dimension $n^{1/d}$, reducing PCS commitment key size and the cost of computing evaluation proofs. This tensor decomposition is new to Twist/Shout and was not present in the earlier shuffle argument literature. Finally, we note that Shout can itself be used as a permutation argument: to prove a and b are permutations of each other, commit to a one-hot encoding of π and prove via Shout that $b_i = a_{\pi(i)}$ for all i.

Read-only memories and lookup arguments. Lookup arguments refer to protocols that allow a prover to prove that every value in one vector v is present in some position in another vector called a lookup table t. The verifier in this protocol holds commitments to both v and t (or, in many cases, t is sufficiently structured that no commitment to it is needed).

While lookup arguments as typically formulated do not care about the position of the entries in the lookup table, Lasso [STW24] introduced a generalization of lookup arguments, called *indexed lookups*, where there is an additional committed vector a, and the prover must prove that $\forall i \in [|v|]$, $v[i] = t[a[i]]$. For ease of reference, we refer to traditional lookup arguments as unindexed lookup arguments. Note that indexed lookup arguments trivially provide unindexed lookup arguments by having the prover provide a commitment to the vector a as its first message. There are also transformations from unindexed lookup arguments to indexed lookup arguments (see [STW24] for details). Indexed lookup arguments are equivalent to memory-checking arguments for read-only memory.

There are many lookup arguments in the literature [BCG$^+$18, GW20b, ZBK$^+$22, PK22] [ZGK$^+$22, EFG22, Hab22, STW24, PH23], and they all rely on fingerprinting techniques described in Section 2.3 to prove that the lookup relation holds. We now provide details on some of these works.

Among these, Lasso [STW24] directly builds on the reordering-free read-write memory approach, specifically it generalizes the approach in Spartan [Set20], which itself was specialized from the read/write memories in Spice [SAGL18]. Since the values are only read and never written, some simplifications are possible compared to the read/write setting solved in Spice: there is no need to perform range checks on timestamps, they can be simply incremented by 1.

The first explicit (unindexed) lookup argument was given in Arya [BCG$^+$18]. In Arya, the prover commits to the binary representation of address *multiplicities*, i.e., the number of times each address is read. The prover then proves that looked-up values are a permutation of the vector in which each table element is repeated a number of times equal to its (committed) multiplicity. The approach in Plookup [GW20b] can be viewed as similar to the reordering-based read-write memory in vRAM, with some simplifications arising from the unindexed lookup argument setting. In particular, in Plookup, the prover commits to a sorting of the table entries and looked-up values. The approach in LogUp [Hab22] is similar to plookup [GW20b] except that it changes the fingerprinting strategy: while plookup uses a grand product, LogUp uses a grand sum of inverses.

A key insight of Lasso [STW24] is that the use of sum-check-based layered grand product arguments [Tha13] avoids the need to commit to random field elements, resulting in large prover speedups. Inspired by this, LogUpGKR [PH23] employs a sum-check-based layered grand sum argument to improve the prover efficiency of LogUp. LogUpGKR can also be viewed as a refinement of the original lookup argument, Arya, in that the LogUpGKR prover commits only to the multiplicities of each address (unlike Arya, in LogUpGKR, these multiplicities are each specified via a single field element rather than via its binary representation).

Separately, a line of work starting with Caulk and culminating in cq [ZBK$^+$22, PK22, ZGK$^+$22, EFG22], seeks lookup arguments in which an expensive pre-processing phase is performed (requiring, e.g., a number of scalar multiplications that is quasilinear in table size) after which m lookups can be answered in time that depends only on m and not on table size. However, these solutions are concretely expensive (e.g., cq requires the prover to commit to seven random field elements per lookup).

4 Preliminaries

This section provides self-contained background on the Jolt zkVM, its use of memory checking, and the standard tools used throughout: multilinear extensions, the sum-check protocol, the zero-check PIOP, and one-hot encodings.

4.1 The Jolt zkVM and the role of memory checking

Jolt [AST24] is a zkVM for the RISC-V instruction set architecture. Jolt combines a polynomial IOP with a polynomial commitment scheme for multilinear polynomials, as described in Section 2.1. Jolt uses curve-based multilinear polynomial commitment schemes such as HyperKZG [ZSC24], Zeromorph [KT23], and Dory [Lee21].

Jolt turns nearly everything the VM does into reads and writes to memory. The RISC-V ISA has 32 general-purpose registers; in each CPU cycle, the VM reads at most two registers and writes to one. The baseline Jolt implementation uses an offline memory-checking argument called Spice [SAGL18] to force the prover to process these register reads and writes correctly. Beyond registers, Jolt handles instruction execution via lookups into a large pre-determined table of size 2^{64}, using a read-only memory-checking argument called Lasso [STW24]. In this baseline, memory-checking procedures account for roughly 45% of Jolt's prover time, with Spice (for registers and RAM) contributing about 20% and Lasso (for instruction lookups) about 25%.

Memory-checking arguments like Spice reduce the task of verifying correct reads and writes to proving that two related vectors are permutations of each other, via a technique dating to Lipton [Lip89, Lip90] and Blum et al. [BEG+91]. This permutation check is implemented using a *grand product argument*: with a random challenge r, the prover shows that $\prod_i (a_i - r) = \prod_i (b_i - r)$. Grand product arguments are a major cost driver: the fastest variant [Tha13] achieves proof size $O(\log^2 T)$, while alternatives with $O(\log T)$ proof size [SL20] require the prover to commit to partial products that are arbitrary field elements—far more expensive to commit to than the small values (addresses and register contents) that arise naturally from the memory-checking instance. Twist and Shout avoid grand product arguments entirely. See Section 2.2 for a formal statement of the memory-checking problem and Section 2.3 for further details on prior approaches.

4.2 Multilinear extensions

Our treatment of multilinear extension polynomials, polynomial commitment schemes, and SNARKs is standard and taken verbatim from Setty and Thaler [STW23].

An ℓ-variate polynomial $p\colon \mathbb{F}^\ell \to \mathbb{F}$ is said to be *multilinear* if p has degree at most one in each variable. Let $f\colon \{0,1\}^\ell \to \mathbb{F}$ be any function mapping the ℓ-dimensional Boolean hypercube to a field $\mathbb{F}$. A polynomial $g\colon \mathbb{F}^\ell \to \mathbb{F}$ is said to *extend* f if $g(x) = f(x)$ for all $x \in \{0,1\}^\ell$. It is well-known that for any $f\colon \{0,1\}^\ell \to \mathbb{F}$, there is a unique *multilinear* polynomial $\widetilde{f}\colon \mathbb{F}^\ell \to \mathbb{F}$ that extends f. The polynomial $\widetilde{f}$ is referred to as the *multilinear extension* (MLE) of f.

A basic fact about multilinear polynomials is the following.

Fact 4.1. Let $p\colon \mathbb{F}^n \to \mathbb{F}$ be any multilinear polynomial. Then for any $c \in \mathbb{F}$ and any $x' \in \{0,1\}^{n-1}$,

$$p(c, x') = (1 - c) \cdot p(0, x') + c \cdot p(1, x'). \tag{2}$$

The right hand side of Equation (2) can be computed with a single field multiplication, as it equals $p(0, x') + c \cdot (p(1, x') - p(0, x'))$.

A particular multilinear extension that arises frequently in the design of interactive proofs is $\widetilde{\mathsf{eq}}$, the MLE of the function $\mathsf{eq} : \{0,1\}^s \times \{0,1\}^s \to \mathbb{F}$ defined as follows:

$$\mathsf{eq}(x, e) = \begin{cases} 1 & \text{if } x = e \\ 0 & \text{otherwise.} \end{cases}$$

An explicit expression for $\widetilde{\mathsf{eq}}$ is:

$$\widetilde{\mathsf{eq}}(x, e) = \prod_{i=1}^{s} \left(x_i e_i + (1 - x_i)(1 - e_i)\right). \tag{3}$$

Equation (3) implies that $\widetilde{\mathsf{eq}}(r_1, r_2)$ can be evaluated at any point $(r_1, r_2) \in \mathbb{F}^s \times \mathbb{F}^s$ in $O(s)$ time.[3]

We also make use of the MLE of the *less-than* function. Given any vector $j = (j_1, \ldots, j_{\log T}) \in \{0,1\}^{\log T}$, define $\text{int}(j) = \sum_{i=1}^{\log T} 2^{i-1} \cdot j_i$. Here int is integer arithmetic, independent of the characteristic of $\mathbb{F}$. Then define

$$\mathsf{LT}(j', j) = \begin{cases} 1 & \text{if } \text{int}(j') < \text{int}(j) \\ 0 & \text{otherwise.} \end{cases}$$

It was shown in [STW24, Appendix G, see Equation 58 on Page 56] that $\widetilde{\mathsf{LT}}$ can be evaluated at any input $(r', r) \in \mathbb{F}^{\log T} \times \mathbb{F}^{\log T}$ with $O(\log T)$ field operations.

Multilinear extensions of vectors. Given a vector $u \in \mathbb{F}^m$, we will often refer to the *multilinear extension of* u and denote this multilinear polynomial by $\widetilde{u}$. $\widetilde{u}$ is obtained by viewing u as a function mapping $\{0,1\}^{\log m} \to \mathbb{F}$ in the natural way[4]: the function interprets its $(\log m)$-bit input $(i_0, \ldots, i_{\log m - 1})$ as the binary representation of an integer i between 0 and $m - 1$, and outputs u_i. $\widetilde{u}$ is defined to be the multilinear extension of this function.

An explicit expression for the MLE of a vector $u \in \mathbb{F}^m$ is: $\widetilde{u}(x) = \sum_{y \in \{0,1\}^{\log m}} u_y \cdot \widetilde{\mathsf{eq}}(y, x)$. For each $y \in \{0,1\}^{\log m}$, the multilinear polynomial $x \mapsto \widetilde{\mathsf{eq}}(y, x) = \prod_{i=1}^{\log m} (y_i x_i + (1 - y_i)(1 - x_i))$ is called the *yth multilinear Lagrange basis polynomial.*

Lemma 4.1 (Vu et al. [VSBW13])**.** *Given as input a point $r \in \mathbb{F}^{\log m}$, it is possible to evaluate all Lagrange basis polynomials at r using only m field multiplications. Given a vector $u \in \mathbb{F}^m$, it is possible to compute $\widetilde{u}(r)$ with $2m$ field multiplications.*

[3] Throughout this manuscript, we consider any field addition or multiplication to require constant time.

[4] All logarithms in this paper are to base 2.

4.3 The sum-check protocol

Let g be some ℓ-variate polynomial defined over a finite field $\mathbb{F}$. The purpose of the sum-check protocol is for the prover to provide the verifier with the following sum:

$$H \coloneqq \sum_{b \in \{0,1\}^\ell} g(b). \tag{4}$$

To compute H unaided, the verifier would have to evaluate g at all 2^ℓ points in $\{0,1\}^\ell$ and sum the results. The sum-check protocol allows the verifier to offload this hard work to the prover. It consists of ℓ rounds, one per variable of g. In round i, the prover sends a message consisting of d_i field elements, where d_i is the degree of g in its i'th variable, and the verifier responds with a single (randomly chosen) field element. If the prover is honest, this polynomial (in the single variable X_i) is

$$\sum_{(b_{i+1},\ldots,b_{\ell-1}) \in \{0,1\}^{\ell-i-1}} g(r_0, \ldots, r_{i-1}, X_i, b_{i+1}, \ldots, b_{\ell-1}). \tag{5}$$

Here, we are indexing both the rounds of the sum-check protocol and the variables of g starting from zero (i.e., indexing them by $\{0, 1, \ldots, \ell-1\}$), and $r_0, \ldots, r_{i-1}$ are random field elements chosen by the verifier across rounds $0, \ldots, i-1$ of the protocol.

The protocol has perfect completeness, and soundness error $\left(\sum_{i=1}^{\ell} d_i\right)/|\mathbb{F}|$. The verifier's runtime is $O\left(\sum_{i=1}^{\ell} d_i\right)$, plus the time required to evaluate g at a single point $r \in \mathbb{F}^\ell$. In the typical case that $d_i = O(1)$ for each round i, this means the total verifier time is $O(\ell)$, plus the time required to evaluate g at a single point $r \in \mathbb{F}^\ell$. This is exponentially faster than the 2^ℓ time that would generally be required for the verifier to compute H. See [AB09, Chapter 8] or [Tha22, §4.1] for details.

4.4 Zero-check PIOP

Let g be an ℓ-variate polynomial of, say, constant degree in each variable. The following standard interactive proof establishes that $g(x) = 0$ for all $x \in \{0,1\}^\ell$, while requiring the verifier to merely evaluate g at a single point in $\mathbb{F}^\ell$ (after processing a proof consisting of $O(\ell)$ field elements).

The verifier picks an input $r \in \mathbb{F}^\ell$ at random. The prover and verifier apply the sum-check protocol to confirm that

$$0 = \sum_{x \in \{0,1\}^\ell} \widetilde{\mathsf{eq}}(r, x) \cdot g(x). \tag{6}$$

This PIOP is perfectly complete. By a direct application of the Schwartz-Zippel lemma, the soundness error is $\ell/|\mathbb{F}|$ (plus the soundness error of the sum-check protocol itself).

4.5 One-hot encodings

Fix K and let z be an integer in $\{0, 1, \ldots K-1\}$. Addresses are indices; we describe their representation as field elements below. The (one-dimensional) *one-hot encoding* of z is the unit vector $e_z \in \mathbb{F}^K$ whose z'th entry is 1 and all other entries of which are 0.

Next, let us assume for simplicity that $K^{1/d}$ is an integer. For $d > 1$, the d-dimensional one-hot encoding of z refers to the collection of d unit vectors, say $v_1, \ldots, v_d$, each of length $K^{1/d}$, whose tensor product $v_1 \otimes v_2 \cdots \otimes v_d$ equals e_z. In other words, if we index the K entries of e_z by $k = (k_1, \ldots, k_d) \in \{0, \ldots, K^{1/d} - 1\}^d$, then $v_1, \ldots, v_d$ are the unique unit vectors such that the k'th entry of e_z equals

$$\prod_{i=1}^{d} v_i(k_i).$$

This implies that, if z itself corresponds to $(z_1, \ldots, z_d)$ via the natural bijection between $\{0, \ldots, K-1\}$ and $\{0, \ldots, K^{1/d} - 1\}^d$, then $v_i = e_{z_i} \in \{0,1\}^{K^{1/d}}$ for $i = 1, \ldots, d$.

Address representation over binary fields. Over $\mathbb{F} = \mathbb{F}_{2^b}$ with $b \geq \log K$, choose publicly known, $\mathbb{F}_2$-linearly independent elements $\beta_0, \ldots, \beta_{\log K - 1} \in \mathbb{F}$ and encode an address $k \in \{0,1\}^{\log K}$ as

$$\mathrm{enc}(k) = \sum_{\ell=0}^{\log K - 1} \beta_\ell k_\ell.$$

This encoding is injective on Boolean addresses and multilinear in their bits. It replaces the integer encoding $\sum_\ell 2^\ell k_\ell$, which collapses in characteristic two. In Equation (12) and Figures 6 and 8, use $\mathrm{enc}(k)$ as the address weight, so the virtual address polynomial returns this encoding of the selected address.

5 The Shout and Twist PIOPs

5.1 The Shout PIOP

Section 2.4 gives an overview of Shout. This section gives the formal PIOP for both $d = 1$ and general d, together with the one-hot checks. We focus here on specifying the PIOP and summarize its prover costs below.

Query conventions. A virtual polynomial is fixed by the polynomials defining it. Its evaluation claims are checked by sum-check, with terminal evaluations obtained from committed polynomials or further virtual queries. Every claim and round message precedes the verifier challenges used to check it; parallel instances sharing a challenge send all their round messages first. Comparing two fixed multilinear polynomials in ℓ variables at a fresh random point adds at most $\ell/|\mathbb{F}|$ to the soundness error.

5.2 Core Shout PIOP for $d = 1$

Figure 5 contains the core Shout PIOP when $d = 1$. It is identical to (the first sum-check invocation) in the Generalized-Lasso protocol [STW24].

Theorem 5.1. *Consider an instance of read-only memory-checking, and assume that for each $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{ra}}(\cdot, j)$ is the one-hot representation of the address read at cycle j. Then the PIOP in Figure 5 has perfect completeness and soundness error $(2\log(K) + \log(T))/|\mathbb{F}|$.*

1. $\mathcal{P}$ and $\mathcal{V}$ have agreed upon a size-K lookup table whose multilinear extension is given by $\widetilde{\mathsf{Val}}$ (which we assume to be evaluable in $O(\log K)$ time), and $\mathcal{P}$ has already committed to a multilinear polynomial $\widetilde{\mathsf{ra}}\colon \mathbb{F}^{\log K} \times \mathbb{F}^{\log T} \to \mathbb{F}$. $\mathcal{P}$ wishes to give the verifier query access to the virtual polynomial $\widetilde{\mathsf{rv}}\colon \mathbb{F}^{\log T} \to \mathbb{F}$, defined as the unique multilinear polynomial satisfying that: for all $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{rv}}(j) = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, j) \cdot \widetilde{\mathsf{Val}}(k)$.
2. $\mathcal{V} \to \mathcal{P}$: pick the desired evaluation point $r_{\text{cycle}} \in \mathbb{F}^{\log T}$ send it to $\mathcal{P}$.
3. $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to compute

$$\widetilde{\mathsf{rv}}(r_{\text{cycle}}) = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, r_{\text{cycle}}) \cdot \widetilde{\mathsf{Val}}(k).$$

4. Let r_{address} denote the randomness chosen over the course of the sum-check protocol. To perform $\mathcal{V}'s$ check in the final round of the sum-check protocol, $\mathcal{V}$ evaluates the committed polynomial $\widetilde{\mathsf{ra}}$ respectively at the random point $(r_{\text{address}}, r_{\text{cycle}})$, and $\mathcal{V}$ evaluates $\widetilde{\mathsf{Val}}(r_{\text{address}})$ directly in $O(\log K)$ time.

Figure 5: The core Shout PIOP when $d = 1$, assuming the lookup table is MLE-structured (i.e., $\mathcal{V}$ can evaluate $\widetilde{\mathsf{Val}}$ at a random input in $O(\log K)$ time).

Proof. Since $\widetilde{\mathsf{ra}}(\cdot, j)$ is the one-hot representation of the address read at cycle j, $\widetilde{\mathsf{rv}}(j)$ is indeed the table value stored at that address if and only if

$$\widetilde{\mathsf{rv}}(j) = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, j) \cdot \widetilde{\mathsf{Val}}(k). \tag{7}$$

Both the left hand side and right hand side of Equation (7) are multilinear polynomials in j. Hence they are equal as formal polynomials if and only if this equality holds for all $j \in \{0,1\}^{\log T}$. By the Schwartz-Zippel lemma, up to soundness error $\log(T)/|\mathbb{F}|$, in order to check that the left hand side and right hand side are the same multilinear polynomial it suffices for $\mathcal{V}$ to pick r_{cycle} at random from $\mathbb{F}^{\log T}$ and confirm that

$$\widetilde{\mathsf{rv}}(r_{\text{cycle}}) = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, r_{\text{cycle}}) \cdot \widetilde{\mathsf{Val}}(k). \tag{8}$$

Figure 5 applies the sum-check protocol to confirm this. The claim now follows from the completeness and soundness properties of the sum-check protocol (Section 4.3). □

According to our formulation of the memory-checking problem (Section 2.2), one should separately check that for each $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{ra}}(\cdot, j)$ is indeed the one-hot representation of some address $\mathsf{raf}(j) \in \{0, 1, \ldots, K-1\}$ that is read at cycle j (and query access to the multilinear polynomial $\widetilde{\mathsf{raf}}$ should be granted to the verifier). A PIOP for establishing this is given in Section 5.3 below.

The prover in the core Shout PIOP when $d = 1$ can be implemented efficiently; the resulting costs are summarized below.

5.3 Checking correctness of one-hot encodings

Suppose the prover has committed to (the multilinear extension of) a vector $\mathsf{ra} \in \left(\mathbb{F}^K\right)^T$ and is also prepared to grant the verifier query access to a $\log(T)$-variate multilinear virtual polynomial $\widetilde{\mathsf{raf}}$. The prover claims that for each $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{raf}}(j) \in \{0, 1, \ldots, K-1\}$ and the vector $\widetilde{\mathsf{ra}}(\cdot, j)$ as $\cdot$ ranges over $\{0,1\}^{\log K}$, equals the one-hot encoding of $\widetilde{\mathsf{raf}}(j)$. We describe a PIOP to check these claims. In other words, the PIOP allows the verifier to:

- Confirm that $\widetilde{\mathsf{ra}}(k, j) \in \{0, 1\}$ for all $k \in \{0,1\}^{\log K}$.
- Confirm that $\widetilde{\mathsf{ra}}(k, j)$ equals 1 for exactly one register $k \in \{0,1\}^{\log K}$.
- Obtain query access to $\widetilde{\mathsf{raf}}(j)$, where the $k \in \{0,1\}^{\log K}$ from the previous bullet is the binary representation of $\widetilde{\mathsf{raf}}(j)$.

Checking the first bullet. To check the first bullet point, simply apply the zero-check PIOP to confirm that for all $(k, j) \in \{0,1\}^{\log K} \times \{0,1\}^{\log T}$,

$$\widetilde{\mathsf{ra}}(k, j)^2 - \widetilde{\mathsf{ra}}(k, j) = 0. \tag{9}$$

The prover in this zero-check PIOP performs only $O(K) + 2T$ field multiplications.

Checking the second bullet. As long as the field characteristic is at least K, the second bullet point is equivalent to the following constraint holding for all $j \in \{0,1\}^{\log T}$:

$$1 = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, j). \tag{10}$$

Since the left hand side and right hand side of Equation (10) are both multilinear polynomials in j (in fact, the left hand side has degree 0), satisfaction of this constraint system is equivalent to $\sum_k \widetilde{\mathsf{ra}}(k, j)$ being the constant polynomial 1. Hence, the entire constraint system can be checked (with soundness error $\log(T)/|\mathbb{F}|$) by having the verifier pick a random $r' \in \mathbb{F}^{\log T}$ and confirming that

$$1 = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, r'). \tag{11}$$

Since $\widetilde{\mathsf{ra}}$ is multilinear, it is easy to see that, so long as the field characteristic is greater than 2, this sum is in fact equal to

$$K \cdot \widetilde{\mathsf{ra}}(2^{-1}, \ldots, 2^{-1}, r'),$$

(see for example [AW09, End of Section 1.4] for reference). And so Equation (11) can be confirmed to hold by the verifier with a single evaluation query to $\widetilde{\mathsf{ra}}$, at point $(2^{-1} \ldots, 2^{-1}, r')$.

Alternatively, one can simply apply the sum-check protocol directly to compute Equation (11). This has the additional benefit that it lowers the number of evaluation queries to $\widetilde{\mathsf{ra}}$ performed by the verifier (i.e., by running this sum-check in parallel with the one used to check the third bullet below, we can ensure that both sum-checks wind up evaluating $\widetilde{\mathsf{ra}}$ at the same point).

Remark 5.2. If the field characteristic is less than K, then Equation (10) is not equivalent to confirming that $\widetilde{\mathsf{ra}}(k,j)$ equals 1 for exactly one register $k \in \{0,1\}^{\log K}$. Other approaches are possible in this setting. For example, in the case of binary fields, one can instead check that all pairwise products $\mathsf{ra}(k,j) \cdot \mathsf{ra}(k',j)$ for $k \neq k'$ within each row j are 0. Combined with Booleanity and the row-sum condition in Equation (10), this guarantees that exactly one entry in each row equals 1. Section 5.5 gives the complete check, with $O(\log T + \log(K)/d)$ communication and $O(K^{1/d} + T)$ prover work per one-hot factor. The address encoding over binary fields is specified in Section 4.5.

Checking the third bullet. Given the first two bullet points hold, the third bullet point is equivalent to the following constraint holding for all $j \in \{0,1\}^{\log T}$:

$$\widetilde{a}(j) = \sum_{k \in \{0,1\}^{\log K}} \left(\sum_{i=0}^{\log(K)-1} 2^i k_i \right) \cdot \widetilde{\mathsf{ra}}(k,j). \tag{12}$$

Since the left hand side and right hand side of Equation (12) are both multilinear polynomials in j, satisfaction of this constraint system implies that $\widetilde{a}$ equals the right hand side of Equation (12) as formal polynomials (and vice versa). Hence, the entire constraint system can be checked (with soundness error $\log(T)/|\mathbb{F}|$) by having the verifier pick a random $r' \in \mathbb{F}^{\log T}$ and confirming that

$$\widetilde{\mathsf{raf}}(r') = \sum_{k \in \{0,1\}^{\log K}} \left(\sum_{i=0}^{\log(K)-1} 2^i k_i \right) \cdot \widetilde{\mathsf{ra}}(k,r').$$

If $\widetilde{\mathsf{raf}}$ is committed, its claimed evaluation must be authenticated against that commitment. If $\widetilde{\mathsf{raf}}$ is virtual, the following sum-check supplies its evaluation instead. To compute

$$\sum_{k \in \{0,1\}^{\log K}} \left(\sum_{i=0}^{\log(K)-1} 2^i k_i \right) \cdot \widetilde{\mathsf{ra}}(k,r'),$$

the prover and verifier can apply the sum-check protocol, at the end of which the verifier needs to evaluate $\widetilde{\mathsf{ra}}(r,r')$ for a random point $(r,r') \in \mathbb{F}^{\log K} \times \mathbb{F}^{\log T}$. This evaluation can be obtained by the verifier from the commitment to $\widetilde{\mathsf{ra}}$.

Figure 6 provides the entire PIOP. The following theorem is immediate from completeness and soundness of the sum-check protocol and the discussion above.

Theorem 5.3. *Over a field of characteristic at least K, Figure 6 satisfies perfect completeness and has soundness error $O((\log(K) + \log(T))/|\mathbb{F}|)$. That is, if $\mathsf{ra}(j)$ is not the one-hot encoding of some value $\mathsf{raf}(j) \in \{0,1,\ldots,K-1\}$ for all $j \in \{0,1\}^{\log T}$, or if the claim that $\widetilde{\mathsf{raf}}(r') = y$ is false, the verifier will accept with probability at most $O((\log(K) + \log(T))/|\mathbb{F}|)$.*

Proof. First, suppose that for some $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{ra}}(\cdot, j)$ is not the correct one-hot-representation of some value in $\{0,1,\ldots,K-1\}$. Then either $\widetilde{\mathsf{ra}}(k,j) \notin \{0,1\}$ for some $k \in \{0,1\}^{\log K}$, or $\sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k,j) \neq 1$. We claim this implies that the verifier in Figure 6 rejects with sufficiently high probability.

Observe that the following polynomial $g(r, r')$ is multilinear:

$$g(r, r') = \sum_{k \in \{0,1\}^{\log K},\ j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r, k)\widetilde{\mathsf{eq}}(r', j)\left(\widetilde{\mathsf{ra}}(k, j)^2 - \widetilde{\mathsf{ra}}(k, j)\right).$$

Furthermore, if there is any $(k, j) \in \{0,1\}^{\log K} \times \{0,1\}^{\log T}$ such that $\widetilde{\mathsf{ra}}(k, j) \notin \{0, 1\}$, g is not the identically-zero polynomial. Hence, by the Schwartz-Zippel lemma, with probability at least $1 - (\log(K) + \log(T))/|\mathbb{F}|$ over the random choice of r, r', Equation (14) fails to hold. In this event, the soundness guarantee of the sum-check protocol ensures that the verifier rejects within the Booleanity-checking sum-check with probability at least $1 - 3(\log(K) + \log(T))/|\mathbb{F}|$.

Let $h(j) = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, j)$. h is multilinear in j, and if there is any $j \in \{0,1\}^{\log T}$ for which $h(j) \neq 1$, then h is not the identically-1 polynomial. In this case, by the Schwartz-Zippel lemma, with probability at least $1 - \log(T)/|\mathbb{F}|$ over the random choice of r_{cycle}, $1 = h(r_{\text{cycle}})$ will fail to hold. In this event, the soundness guarantee of the sum-check protocol ensures that the verifier will reject in the Hamming weight 1 check with probability at least $1 - \log(K)/|\mathbb{F}|$. This completes the proof of the claim.

Finally, suppose that $\widetilde{\mathsf{raf}}(r_{\text{cycle}}) \neq y$. We claim that $\widetilde{\mathsf{raf}}(r_{\text{cycle}})$ satisfies the following equality:

$$\widetilde{\mathsf{raf}}(r_{\text{cycle}}) = \sum_{k \in \{0,1\}^{\log K}} \left(\sum_{i=0}^{\log(K)-1} 2^i \cdot k_i \right) \cdot \widetilde{\mathsf{ra}}(k, r_{\text{cycle}}), \tag{13}$$

Indeed, the right hand side of this equation is multilinear in r_{cycle} and agrees with raf whenever $r_{\text{cycle}} \in \{0,1\}^{\log T}$. Since $\{0,1\}^{\log T}$ is an interpolating set for multilinear polynomials, the left hand side and right hand side must be equal as formal polynomials in r_{cycle}.

Hence, if $\widetilde{\mathsf{raf}}(r_{\text{cycle}}) \neq y$, soundness of the sum-check protocol guarantees that the verifier will reject in the $\widetilde{\mathsf{raf}}$-evaluation sum-check invocation with probability at least $1 - 2\log(K)/|\mathbb{F}|$. This completes the proof of the soundness property stated in the theorem.

Perfect completeness is immediate from perfect completeness of the sum-check protocol, combined with the following facts:

- If every $\widetilde{\mathsf{ra}}(\cdot, j)$ is a valid one-hot encoding for all $j \in \{0,1\}^{\log T}$, then $\widetilde{\mathsf{ra}}(k, j) \in \{0, 1\}$ for all $(k, j) \in \{0,1\}^{\log(K)} \times \{0,1\}^{\log T}$, and hence $\widetilde{\mathsf{ra}}(k, j)^2 - \widetilde{\mathsf{ra}}(k, j) = 0$.
- If every $\widetilde{\mathsf{ra}}(\cdot, j)$ is a valid one-hot encoding for all $j \in \{0,1\}^{\log T}$, then $\sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, j) = 1$ for all $j \in \{0,1\}^{\log T}$, and since the left hand side is a multilinear polynomial in j and $\{0,1\}^{\log T}$ is an interpolating set for such polynomials, this implies $\sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, j) = 1$ holds for all $j \in \mathbb{F}^{\log T}$.
- Equation (13) holds.

This completes the proof of the theorem. □

1. As input, $\mathcal{P}$ has already committed to a multilinear polynomial $\widetilde{\mathsf{ra}}\colon \mathbb{F}^{\log K} \times \mathbb{F}^{\log T} \to \mathbb{F}$. $\mathcal{P}$ claims that for each $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{ra}}(\cdot, j)$ is the one-hot representation of some value $\widetilde{\mathsf{raf}}(j) \in \{0, 1, \ldots, K-1\}$, and (optionally) that $\widetilde{\mathsf{raf}}(r_{\text{cycle}}) = y$.
2. $\mathcal{V}$ picks $r \in \mathbb{F}^{\log K}$ and $r' \in \mathbb{F}^{\log T}$ at random and sends (r, r') to $\mathcal{P}$.
3. **(Booleanity check)**: $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to confirm that

$$0 = \sum_{k \in \{0,1\}^{\log K},\ j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r, k)\widetilde{\mathsf{eq}}(r', j)\left(\widetilde{\mathsf{ra}}(k, j)^2 - \widetilde{\mathsf{ra}}(k, j)\right). \tag{14}$$

4. Let $(r'_{\text{address}}, r'_{\text{cycle}}) \in \mathbb{F}^{\log K} \times \mathbb{F}^{\log T}$ be the randomness chosen over the course of the sum-check protocol invoked in Line 3.
5. **(Hamming weight 1 check)**: $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to confirm that

$$1 = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, r'_{\text{cycle}}).$$

6. **($\widetilde{\mathsf{raf}}$-evaluation sum-check)**: In parallel with the sum-check invocation in Line 5, $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to confirm that

$$y = \sum_{k \in \{0,1\}^{\log K}} \left(\sum_{i=0}^{\log(K)-1} 2^i \cdot k_i \right) \cdot \widetilde{\mathsf{ra}}(k, r_{\text{cycle}}).$$

   Let r''_{address} be the randomness chosen by the verifier over the course of this sum-check instance.
7. To finish the three sum-checks, the verifier queries the following evaluations of the committed polynomial. The Booleanity check uses $\widetilde{\mathsf{ra}}(r'_{\text{address}}, r'_{\text{cycle}})$; the Hamming-weight check uses $\widetilde{\mathsf{ra}}(r''_{\text{address}}, r'_{\text{cycle}})$; and the address-evaluation check uses $\widetilde{\mathsf{ra}}(r''_{\text{address}}, r_{\text{cycle}})$. The verifier computes all equality factors and address weights locally. There are three evaluation queries in general, or two if $r_{\text{cycle}} = r'_{\text{cycle}}$. The prover can use standard batching techniques to give a single opening proof for these evaluations.

Figure 6: A PIOP for checking that for each $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{ra}}(\cdot, j)$ is the (one-dimensional) one-hot representation of some value $\widetilde{\mathsf{raf}}(j) \in \{0, 1, \ldots, K-1\}$. This PIOP also grants the verifier query access to the virtual polynomial $\widetilde{\mathsf{raf}}$ (i.e., when only $\widetilde{\mathsf{ra}}$, not $\widetilde{\mathsf{raf}}$, is sent/committed by the prover). The Hamming-weight 1 check assumes the field characteristic is at least K; see Remark 5.2 for details of what can be done in fields of smaller characteristic.

5.4 Shout for general d

The difference between Shout for $d = 1$ and Shout for $d > 1$ is simply that when $d > 1$, ra is virtualized in terms of $\widetilde{\mathsf{ra}}_1, \ldots, \widetilde{\mathsf{ra}}_d$. That is, at the end of the core Shout PIOP (Figure 5), rather than the verifier obtaining $\widetilde{\mathsf{ra}}(r_{\text{address}}, r_{\text{cycle}})$ directly from a commitment to $\widetilde{\mathsf{ra}}$, the verifier instead invokes the $\widetilde{\mathsf{ra}}$-virtualization sum-check. Figure 8 specifies the PIOP for proving that each of $\widetilde{\mathsf{ra}}_1, \ldots, \widetilde{\mathsf{ra}}_d$ is one-hot.

1. $\mathcal{P}$ and $\mathcal{V}$ have agreed upon a size-K lookup table whose multilinear extension is given by $\widetilde{\mathsf{Val}}$ (which we assume to be evaluable in $O(\log K)$ time). $\mathcal{P}$ has already committed to multilinear polynomials $\widetilde{\mathsf{ra}}_1, \ldots, \widetilde{\mathsf{ra}}_d \colon \mathbb{F}^{\log(K)/d} \times \mathbb{F}^{\log T} \to \mathbb{F}$. $\mathcal{P}$ wishes to give the verifier query access to the virtual polynomial $\widetilde{\mathsf{rv}} \colon \mathbb{F}^{\log T} \to \mathbb{F}$, defined as the unique multilinear polynomial satisfying that: for all $j \in \{0,1\}^{\log T}$,

$$\widetilde{\mathsf{rv}}(j) = \sum_{k=(k_1,\ldots,k_d) \in \left(\{0,1\}^{\log(K)/d}\right)^d} \widetilde{\mathsf{ra}}(k,j) \cdot \widetilde{\mathsf{Val}}(k),$$

   where

$$\widetilde{\mathsf{ra}}(k,j) = \sum_{j' \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(j,j') \left(\prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(k_i,j') \right).$$

2. $\mathcal{V} \to \mathcal{P}$: pick the desired evaluation point $r_{\text{cycle}} \in \mathbb{F}^{\log T}$ at random and send it to $\mathcal{P}$.
3. **(Read-checking for Shout)**: $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to confirm that

$$\widetilde{\mathsf{rv}}(r_{\text{cycle}}) = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, r_{\text{cycle}}) \cdot \widetilde{\mathsf{Val}}(k). \tag{15}$$

4. **($\widetilde{\mathsf{ra}}$-virtualization sum-check)**: Let $r_{\text{address}} = (r^{(1)}_{\text{address}}, \ldots, r^{(d)}_{\text{address}}) \in \left(\mathbb{F}^{\log(K)/d}\right)^d$ denote the randomness chosen over the $\log(K)$ rounds of the read-checking sum-check protocol. To perform its final check in the read-checking sum-check, $\mathcal{V}$ evaluates $\widetilde{\mathsf{Val}}$ at r_{address} in $O(\log K)$ time, and obtains $\widetilde{\mathsf{ra}}(r_{\text{address}}, r_{\text{cycle}})$ by applying the sum-check protocol to the right hand side of the below equality:

$$\widetilde{\mathsf{ra}}(r_{\text{address}}, r_{\text{cycle}}) = \sum_{j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r_{\text{cycle}}, j) \cdot \prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(r^{(i)}_{\text{address}}, j). \tag{16}$$

5. Let r'_{cycle} denote the randomness chosen during the $\log T$ rounds of the $\widetilde{\mathsf{ra}}$-virtualization sum-check. To perform its final check in the $\widetilde{\mathsf{ra}}$-virtualization sum-check, $\mathcal{V}$ queries the committed polynomials $\widetilde{\mathsf{ra}}_1, \ldots, \widetilde{\mathsf{ra}}_d$ respectively at $(r^{(i)}_{\text{address}}, r'_{\text{cycle}})$, and evaluates $\widetilde{\mathsf{eq}}$ at the relevant point $(r_{\text{cycle}}, r'_{\text{cycle}})$ in $O(\log T)$ time.

Figure 7: The core Shout PIOP with integer parameter $d > 1$, assuming the lookup table is MLE-structured (i.e., $\mathcal{V}$ can evaluate $\widetilde{\mathsf{Val}}$ at a random input in $O(\log K)$ time).

Theorem 5.4. *Figures 7 and 8 satisfy perfect completeness. Assume that, for every $j \in \{0,1\}^{\log T}$, the address $(\widetilde{\mathsf{ra}}_1(\cdot, j), \ldots, \widetilde{\mathsf{ra}}_d(\cdot, j))$ is the d-dimensional one-hot encoding of some value $\widetilde{\mathsf{raf}}(j) \in \{0, 1, \ldots, K-1\}$. Then Figures 7 and 8 have soundness error at most*

$$O(d\,(\log T + \log K))/|\mathbb{F}|.$$

That is, the verifier in Figure 7 will reject with overwhelming probability if there is any $j \in \{0,1\}^{\log T}$ such that $\widetilde{\mathsf{rv}}(j) \neq \widetilde{\mathsf{Val}}(k)$, where $k \in \{0,1\}^{\log K}$ is the binary representation of the address whose d-dimensional one-hot encoding is given by $(\widetilde{\mathsf{ra}}_1(j), \ldots, \widetilde{\mathsf{ra}}_d(j))$.

Proof. The completeness and soundness of Figures 7 and 8 is nearly identical to Theorems 5.1 and 5.3. The only difference is that in Figures 7 and 8, $\widetilde{\mathsf{ra}}$ is virtualized in terms of $\widetilde{\mathsf{ra}}_1, \ldots, \widetilde{\mathsf{ra}}_d$. Hence, completeness of Figures 7 and 8 follows from the fact that any one-hot vector $\mathsf{ra}(\cdot, j) \in \{0,1\}^K$ has a corresponding d-dimensional one-hot encoding $(\mathsf{ra}_1(\cdot, j), \ldots, \mathsf{ra}_d(\cdot, j))$. In other words, there exist one-hot vectors $\mathsf{ra}_i(\cdot, j) \in \{0,1\}^{K^{1/d}}$ such that

$$\mathsf{ra}(k, j) = \prod_{i=1}^{d} \mathsf{ra}_i(k_i, j)$$

for all $k = (k_1, \ldots, k_d) \in \left(\{0,1\}^{\log(K)/d}\right)^d$. In this case

$$\widetilde{\mathsf{ra}}(k, j) = \sum_{j' \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(j, j') \prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(k_i, j') \tag{17}$$

is an equality of formal polynomials, because the right hand side is multilinear in k and j and is easily seen to agree with $\widetilde{\mathsf{ra}}(k, j)$ whenever $(k, j) \in \{0,1\}^{\log K} \times \{0,1\}^{\log T}$. This ensures completeness of the $\widetilde{\mathsf{ra}}$-virtualization sum-check invoked in Figures 7 and 8.

The claimed soundness bounds then follow from Theorems 5.1 and 5.3 combined with the fact that the $\widetilde{\mathsf{ra}}$-virtualization sum-check introduces soundness error $O((d+1)\log T)/|\mathbb{F}|$, as the summand on the right hand side of Equation (17) has degree at most $d+1$ in each variable of j'.

□

5.5 One-hot checking in small characteristic

We now give a one-hot check that works over any field, including binary fields. The issue, recalled from Remark 5.2, is that a row of 0s and 1s can sum to 1 without containing exactly one 1. For example, over a binary field, a row containing three 1s also sums to 1. We show how to rule out such rows. Consider a table with T rows, each containing n entries, where T and n are powers of two. Our goal is to check that every row contains exactly one 1 and is otherwise zero. Let $m = \log n$ and $t = \log T$, and let $\widetilde{X} \colon \mathbb{F}^m \times \mathbb{F}^t \to \mathbb{F}$ be the table's multilinear extension. The Boolean variables $k \in \{0,1\}^m$ index entries within a row, and $j \in \{0,1\}^t$ indexes rows. The prover commits to $\widetilde{X}$ before the verifier chooses any of the random points used below.

1. As input, $\mathcal{P}$ has already committed to multilinear polynomials $\widetilde{\mathsf{ra}}_1, \ldots, \widetilde{\mathsf{ra}}_d \colon \mathbb{F}^{\log(K)/d} \times \mathbb{F}^{\log T} \to \mathbb{F}$. $\mathcal{P}$ claims that for each $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{ra}}_1(\cdot, j), \ldots, \widetilde{\mathsf{ra}}_d(j)$ is the correct d-dimensional one-hot representation of some value $\widetilde{\mathsf{raf}}(j) \in \{0, 1, \ldots, K-1\}$, and also (optionally) that $\widetilde{\mathsf{raf}}(r'_{\text{cycle}}) = y$.
2. $\mathcal{V}$ picks $r \in \mathbb{F}^{\log(K)/d}$ and $r' \in \mathbb{F}^{\log T}$ at random and sends (r, r') to $\mathcal{P}$. Assuming that r'_{cycle} was chosen by the verifier at random after $\widetilde{\mathsf{ra}}_1, \ldots, \widetilde{\mathsf{ra}}_d$ and $\widetilde{\mathsf{rv}}$ were committed, then $\mathcal{V}$ can set $r' = r'_{\text{cycle}}$.
3. **(Booleanity check)**: $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol d times in parallel to confirm that for $i = 1, \ldots, d$,
$$0 = \sum_{k \in \{0,1\}^{\log(K)/d},\ j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r, k)\widetilde{\mathsf{eq}}(r', j)\left(\widetilde{\mathsf{ra}}_i(k, j)^2 - \widetilde{\mathsf{ra}}_i(k, j)\right).$$
4. **(Hamming weight 1 check)**: Let $r''_{\text{cycle}} \in \mathbb{F}^{\log T}$ be any value chosen at random by $\mathcal{V}$ after $\widetilde{\mathsf{ra}}_i$ was committed. For each $i = 1, \ldots, d$, $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to confirm that
$$1 = \sum_{k_i \in \{0,1\}^{\log(K)/d}} \widetilde{\mathsf{ra}}_i(k_i, r''_{\text{cycle}}).$$
5. **($\widetilde{\mathsf{raf}}$-evaluation sum-check)**: In parallel with the Booleanity check (and the read-checking sum-check from the core Shout PIOP, see Line 3 of Figure 7), $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to confirm that
$$y = \sum_{k=(k_1,\ldots,k_d) \in \left(\{0,1\}^{\log(K)/d}\right)^d,\ j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r'_{\text{cycle}}, j)\left(\sum_{i=1}^{d} \sum_{\ell=0}^{\log(K)/d-1} 2^{(i-1)\cdot\log(K)/d+\ell} \cdot k_{i,\ell}\right) \widetilde{\mathsf{ra}}(k, j).$$
At the end of this sum-check protocol, $\mathcal{V}$ obtains the necessary evaluation of $\widetilde{\mathsf{ra}}(k, j)$ by invoking the $\widetilde{\mathsf{ra}}$-virtualization sum-check of Figure 7.

Figure 8: A PIOP for checking that for each $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{ra}}_1(\cdot, j), \ldots, \widetilde{\mathsf{ra}}_d(j)$ is the correct d-dimensional one-hot representation of some value $\widetilde{\mathsf{raf}}(j) \in \{0, 1, \ldots, K-1\}$, and also granting the verifier query access to the virtual polynomial $\widetilde{\mathsf{raf}}$ (i.e., enabling only $\widetilde{\mathsf{ra}}$, not $\widetilde{\mathsf{raf}}$, to be sent/committed by the prover). Some hashing-based commitment schemes directly ensure Booleanity of committed values, in which case the Booleanity check (Line 3) can be omitted. In small characteristic, additionally apply Section 5.5. Over binary fields, use the address encoding from Section 4.5 in place of the integer weights.

What needs to be checked? The existing Booleanity check enforces $\widetilde{X}(k,j) \in \{0,1\}$ for all Boolean (k,j). In small characteristic, the condition

$$\sum_{k \in \{0,1\}^m} \widetilde{X}(k,j) = 1 \qquad \text{for every } j \in \{0,1\}^t \tag{18}$$

does not by itself imply Hamming weight one: in characteristic p, it only fixes the weight modulo p. To address this, we enforce

$$\widetilde{X}(k,j)\widetilde{X}(k',j) = 0 \qquad \text{for every Boolean } j \text{ and distinct Boolean } k, k'. \tag{19}$$

Equation (19) ensures that a row contains at most one 1. Equation (18) rules out an all-zero row. Together, they ensure that each row contains exactly one 1. Equation (18) is checked as in Figure 8, using sum-check rather than evaluation at 2^{-1}.

Checking all pairs at once. We use random weights to combine the products in Equation (19) into one polynomial for each row. Sample independent $r, s \leftarrow \mathbb{F}^m$. As is standard in sum-check-based PIOPs, we use the multilinear Lagrange basis to expand these $2m = 2\log n$ random field elements into coefficients $\widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(s,k')$ for all address pairs $(k,k') \in \{0,1\}^m \times \{0,1\}^m$. Define

$$\begin{aligned} \widetilde{A}(j) &= \widetilde{X}(r,j), & \widetilde{B}(j) &= \widetilde{X}(s,j), \\ \widetilde{C}(j) &= \sum_{k \in \{0,1\}^m} \widetilde{X}(k,j)\widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(s,k), & \Delta(j;r,s) &= \widetilde{A}(j)\widetilde{B}(j) - \widetilde{C}(j). \end{aligned} \tag{20}$$

Fix $j \in \{0,1\}^t$. Assume that the row entries satisfy $\widetilde{X}(k,j) \in \{0,1\}$ for every $k \in \{0,1\}^m$; this is the condition enforced by the existing Booleanity check. The row-sum check in Equation (18) already rules out an all-zero row. Our remaining goal is to rule out two or more 1s. We will show that $\Delta(j;r,s)$ is the zero polynomial in r,s exactly when the row contains at most one 1. Thus, checking that $\Delta(j;r,s)$ is the zero polynomial in r,s for this fixed row j, together with the row-sum check, establishes that the row contains exactly one 1.

Because $\widetilde{X}$ is multilinear in its address variables, its Lagrange basis expansions are

$$\begin{aligned} \widetilde{A}(j) = \widetilde{X}(r,j) &= \sum_{k \in \{0,1\}^m} \widetilde{X}(k,j)\widetilde{\mathsf{eq}}(r,k), \\ \widetilde{B}(j) = \widetilde{X}(s,j) &= \sum_{k' \in \{0,1\}^m} \widetilde{X}(k',j)\widetilde{\mathsf{eq}}(s,k'). \end{aligned}$$

Multiplying these two sums gives

$$\begin{aligned} \widetilde{A}(j)\widetilde{B}(j) &= \left(\sum_{k \in \{0,1\}^m} \widetilde{X}(k,j)\widetilde{\mathsf{eq}}(r,k)\right)\left(\sum_{k' \in \{0,1\}^m} \widetilde{X}(k',j)\widetilde{\mathsf{eq}}(s,k')\right) \\ &= \sum_{k,k' \in \{0,1\}^m} \widetilde{X}(k,j)\widetilde{X}(k',j)\widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(s,k'). \end{aligned}$$

This sum includes both pairs with $k = k'$ and pairs with $k \neq k'$. The terms with $k = k'$ sum to

$$\sum_{k \in \{0,1\}^m} \widetilde{X}(k,j)^2\widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(s,k) = \sum_{k \in \{0,1\}^m} \widetilde{X}(k,j)\widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(s,k) = \widetilde{C}(j),$$

where the first equality uses Booleanity, as enforced by the Booleanity check in Line 3 of Figure 8. Subtracting $\widetilde{C}(j)$ therefore removes exactly these terms and leaves only pairs of distinct addresses:

$$\Delta(j; r, s) = \sum_{\substack{k,k' \in \{0,1\}^m \\ k \neq k'}} \widetilde{X}(k, j)\widetilde{X}(k', j)\widetilde{\mathsf{eq}}(r, k)\widetilde{\mathsf{eq}}(s, k'). \tag{21}$$

If the row contains at most one 1, then for every distinct pair k, k', at least one of $\widetilde{X}(k, j)$ and $\widetilde{X}(k', j)$ is zero. Every term in the sum is therefore zero, and $\Delta(j; r, s) = 0$ for all r, s.

Conversely, suppose the row contains 1s at two distinct addresses $k_0, k_1 \in \{0, 1\}^m$. Set $r = k_0$ and $s = k_1$. At these Boolean points, the Lagrange weights select just the ordered pair (k_0, k_1); all other terms vanish. Thus $\Delta(j; k_0, k_1) = \widetilde{X}(k_0, j)\widetilde{X}(k_1, j) = 1$. This exhibits an evaluation at which Δ is nonzero, proving that it is not the zero polynomial in r, s.

For each fixed Boolean row $j \in \{0, 1\}^t$, we have therefore shown that the row contains at most one 1 exactly when $\Delta(j; r, s)$ is the zero polynomial *in* r, s. We test this condition for the row by evaluating $\Delta(j; r, s)$ at random independent $r, s \in \mathbb{F}^m$. A row with at most one 1 always passes. For a row with two or more 1s, the polynomial in r, s is nonzero and has total degree at most $2m$, so the test passes with probability at most $2m/|\mathbb{F}|$, by Schwartz–Zippel.

Checking all rows at once. The preceding argument gives a test for a single Boolean row j: sample r, s and check that $\Delta(j; r, s) = 0$. A row with at most one 1 always passes, while a row with two or more 1s fails with high probability. Our original goal is to establish that *every row contains exactly one* 1. The existing Booleanity and row-sum checks ensure that every row consists of 0s and 1s and contains at least one 1. Thus, it remains to check that every row passes the test above.

We sample one pair r, s and check

$$\Delta(j; r, s) = 0 \qquad \text{for every } j \in \{0, 1\}^t.$$

If all rows contain exactly one 1, these equalities all hold. If any row contains two or more 1s, fix one such row j_0 before sampling r, s. The preceding argument shows that its equality fails with probability at least $1 - 2m/|\mathbb{F}|$. Thus, checking these equalities for all Boolean rows achieves our goal, up to this error probability.

It remains to check all these equalities efficiently. With r, s now fixed, define the multilinear extension of the row values $\Delta(j; r, s)$:

$$\widetilde{Q}(z) = \sum_{j \in \{0,1\}^t} \widetilde{\mathsf{eq}}(z, j)\Delta(j; r, s).$$

Since $\widetilde{Q}(j) = \Delta(j; r, s)$ at every Boolean j, the multilinear polynomial $\widetilde{Q}$ is identically zero exactly when every row passes. We therefore sample an independent $z \in \mathbb{F}^t$ and use sum-check to verify $\widetilde{Q}(z) = 0$ via the defining sum. If any row fails, $\widetilde{Q}$ is nonzero, so $\widetilde{Q}(z) = 0$ with probability at most $t/|\mathbb{F}|$. This is the zero-check PIOP of Section 4.4.

The multilinear extension is essential here: $\Delta = \widetilde{A}\widetilde{B} - \widetilde{C}$ is generally not multilinear in j. Even when all its Boolean row values are zero, it can be nonzero at non-Boolean points. Evaluating $\Delta(z; r, s)$ directly could therefore reject an honest prover; the sum above evaluates $\widetilde{Q}(z)$ instead.

The protocol. In addition to r, s, the verifier samples an independent $z \leftarrow \mathbb{F}^t$. The first sum-check verifies the weighted sum just described. It ends with a claimed evaluation of $\widetilde{C}$, which we check using a second sum-check.

1. Run sum-check over $j \in \{0,1\}^t$ for the claim

$$0 = \sum_{j \in \{0,1\}^t} \widetilde{\mathsf{eq}}(z, j)\big(\widetilde{A}(j)\widetilde{B}(j) - \widetilde{C}(j)\big). \tag{22}$$

   Let $u \in \mathbb{F}^t$ be the final sum-check point. The verifier queries the committed polynomial $\widetilde{X}$ to obtain $a = \widetilde{X}(r, u)$ and $b = \widetilde{X}(s, u)$. Let c denote the claimed value of $\widetilde{C}(u)$. The final check uses $\widetilde{\mathsf{eq}}(z, u)(ab - c)$.

2. Check the claimed value c by a sum-check over $k \in \{0,1\}^m$ for

$$c = \sum_{k \in \{0,1\}^m} \widetilde{X}(k, u)\widetilde{\mathsf{eq}}(r, k)\widetilde{\mathsf{eq}}(s, k). \tag{23}$$

   At its final point $v \in \mathbb{F}^m$, the verifier queries $\widetilde{X}(v, u)$ and computes $\widetilde{\mathsf{eq}}(r, v)$ and $\widetilde{\mathsf{eq}}(s, v)$ itself.

All three polynomials $\widetilde{A}, \widetilde{B}, \widetilde{C}$ are defined by $\widetilde{X}$, so the prover need only commit to $\widetilde{X}$. As usual in sum-check, the claim c is fixed before the verifier samples the random challenges used to check it. The verifier accepts only if both sum-checks and the three queries to $\widetilde{X}$ verify.

Proposition 5.5. *For a table with T rows of length n, the sum-checks in Equations* (22) *and* (23)*, together with the Booleanity and row-sum checks, verify that every row contains exactly one* 1*. The combined protocol has perfect completeness and soundness error $O((\log T + \log n)/|\mathbb{F}|)$.*

The two additional sum-checks require $O(\log T + \log n)$ communication and verifier field work. The prover uses $O(T + n)$ field operations and space, with at most

$$8.5T + O(n + \log T) \tag{24}$$

field multiplications, including preparation of the evaluation arrays.

Proof. For completeness, Equation (19) gives $\Delta(j; r, s) = 0$ for every Boolean j, so Equation (22) is true for all r, s, z. The claim in Equation (23) is true by definition of $\widetilde{C}$. Perfect completeness follows from that of sum-check.

For soundness, suppose some row j_0 contains at least two 1s. By Equation (21) and Schwartz–Zippel, $\Delta(j_0; r, s)$ is nonzero except with probability at most $2m/|\mathbb{F}|$ over r, s. Conditional on this, $\widetilde{Q}$ is nonzero, since $\widetilde{Q}(j_0) = \Delta(j_0; r, s) \neq 0$. Thus $\widetilde{Q}(z) = 0$ with probability at most $t/|\mathbb{F}|$. When $\widetilde{Q}(z) \neq 0$, the first sum-check proves a false sum, with soundness error at most $3t/|\mathbb{F}|$ if its final evaluations are correct. The verifier obtains the two evaluations of $\widetilde{X}$ from its commitment, and an incorrect claim c passes the second sum-check with probability at most $3m/|\mathbb{F}|$. A union bound gives soundness error at most $(5m + 4t)/|\mathbb{F}|$ for the two additional sum-checks. The bounds for the existing Booleanity and row-sum checks are added separately, as is the error of the polynomial commitment scheme when compiling the PIOP.

The communication and verifier-work bounds follow from the $m + t$ sum-check rounds, each of constant degree. The prover-cost bounds are proved in Section 7.5. □

5.6 The Twist PIOP

Section 2.5 explains how Twist represents memory contents by increments and uses the read-checking, write-checking, and $\widetilde{\mathsf{Val}}$-evaluation sum-checks to obtain read and write values. The two parts of Figure 9 specify the core PIOP in full. Section 9 gives efficient prover implementations.

The PIOP in this figure is sound assuming that all one-hot encodings are provided correctly, i.e., that for all $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{ra}}(j)$ and $\widetilde{\mathsf{wa}}(j)$ are the correct d-dimensional one-hot encodings of some values $\widetilde{\mathsf{raf}}(j)$ and $\widetilde{\mathsf{waf}}(j)$ in $\{0,1,\ldots,K-1\}$. To confirm this, one can invoke the same one-hot-encoding check for the write-address polynomials, replacing $\widetilde{\mathsf{ra}}_i$ and $\widetilde{\mathsf{raf}}$ with $\widetilde{\mathsf{wa}}_i$ and $\widetilde{\mathsf{waf}}$ as appropriate.

Theorem 5.6. *Figure 9 has perfect completeness and soundness error at most*

$$O((d \log T + \log K)/|\mathbb{F}|).$$

That is, the verifier in Figure 9 will reject with overwhelming probability if the claimed *value of* $\widetilde{\mathsf{rv}}(r')$ *does not equal the actual value* $\widetilde{\mathsf{rv}}(r')$ *for the unique multilinear polynomial* $\widetilde{\mathsf{rv}}$ *satisfying that following definition: For all* $j \in \{0,1\}^{\log T}$*,*

$$\widetilde{\mathsf{rv}}(j) = \widetilde{\mathsf{Val}}(k, j), \tag{25}$$

where $k \in \{0,1\}^{\log K}$ *is the binary representation of the address whose* d*-dimensional one-hot encoding is given by* $(\widetilde{\mathsf{ra}}_1(j), \ldots, \widetilde{\mathsf{ra}}_d(j))$*, and* $\widetilde{\mathsf{Val}}(k, j)$ *is the value most recently written to address* k *prior to the* j*'th write operation.*

Proof. For completeness, observe that given any sequence of write address and write values, there is a unique set of increments that imply those write values, in the sense that $\widetilde{\mathsf{wv}}(j)$ defined via Equation (28) indeed outputs the appropriate write value for all $j \in \{0,1\}^{\log T}$. Moreover, under this set of increments, $\widetilde{\mathsf{Val}}(k, j)$ (defined via Equation (26)) indeed equals the value stored at cell k at the start of cycle j, for all $k \in \{0,1\}^{\log K}$ and j in $\{0,1\}^{\log T}$. Furthermore, $\widetilde{\mathsf{rv}}(j)$ (defined via Equation (27)) indeed equals the value returned by the j'th read operation. Accordingly, completeness of Figure 9 follows from completeness of the sum-check protocol, as invoked via the read-checking, write-checking, and $\widetilde{\mathsf{Val}}$-evaluation sum-checks.

In a similar manner, the claimed soundness bound follows directly from the soundness of the sum-check protocol and trivial degree bounds on the relevant polynomials. That is, the verifier will reject except with the asserted probability, unless the prover makes honest claims regarding the values $\widetilde{\mathsf{rv}}(r')$ and $\widetilde{\mathsf{wv}}(r'')$, where $\widetilde{\mathsf{rv}}$ and $\widetilde{\mathsf{wv}}$ are the multilinear polynomials derived from the committed $\widetilde{\mathsf{Inc}}$ per Equations (28) and (27).

□

1. As input, $\mathcal{P}$ has already committed to the multilinear polynomials $\widetilde{\mathsf{Inc}}\colon \mathbb{F}^{\log T} \to \mathbb{F}$ and $\widetilde{\mathsf{ra}}_1, \widetilde{\mathsf{wa}}_1, \ldots, \widetilde{\mathsf{ra}}_d, \widetilde{\mathsf{wa}}_d\colon \mathbb{F}^{\log(K)/d} \times \mathbb{F}^{\log T} \to \mathbb{F}$. If the prover is honest, then $\widetilde{\mathsf{Inc}}$ specifies the unique non-zero increment at cycle j if one exists, and zero otherwise (i.e., $\mathsf{Inc}(j)$ is the field difference between the value written at cycle j and the value stored at the accessed location just prior to that write operation). If $d > 1$, wa and ra are virtual polynomials obtained by taking the product of the corresponding d one-hot factors and summing over the cycle variable. $\widetilde{\mathsf{Val}}\colon \mathbb{F}^{\log K} \times \mathbb{F}^{\log T} \to \mathbb{F}$ is a virtual polynomial that is defined as

$$\widetilde{\mathsf{Val}}(k, j) = \sum_{j' \in \{0,1\}^{\log T}} \widetilde{\mathsf{Inc}}(j') \cdot \widetilde{\mathsf{wa}}(k, j') \cdot \widetilde{\mathsf{LT}}(j', j). \tag{26}$$

   Intuitively, $\widetilde{\mathsf{Val}}(k, j)$ returns the value stored at cell k at the *start* of cycle j (before the j'th write operation occurs). The prover wishes to give the verifier query access to the virtual polynomials $\widetilde{\mathsf{rv}}$ and $\widetilde{\mathsf{wv}}$, defined via Equations (27) and (28) below.
2. $\mathcal{V} \to \mathcal{P}$: pick a desired evaluation point $r' \in \mathbb{F}^{\log T}$ for $\widetilde{\mathsf{rv}}$ and $r'' \in \mathbb{F}^{\log T}$ for $\widetilde{\mathsf{wv}}$ and send it to $\mathcal{P}$. Also pick $r \in \mathbb{F}^{\log K}$ at random and send it to $\mathcal{P}$. Then $\mathcal{V}$ and $\mathcal{P}$ run the following two instances of the sum-check protocol in parallel:
3. **Read-checking sum-check.** $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to compute

$$\widetilde{\mathsf{rv}}(r') = \sum_{k \in \{0,1\}^{\log K},\ j' \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r', j') \cdot \widetilde{\mathsf{ra}}(k, j') \cdot \widetilde{\mathsf{Val}}(k, j') \tag{27}$$

4. **Write-checking sum-check**. $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to compute:

$$\widetilde{\mathsf{wv}}(r'') = \sum_{k \in \{0,1\}^{\log K},\ j' \in \{0,1\}^{\log(T)}} \widetilde{\mathsf{eq}}(r'', j') \cdot \widetilde{\mathsf{wa}}(k, j') \cdot \left(\widetilde{\mathsf{Val}}(k, j') + \widetilde{\mathsf{Inc}}(j')\right). \tag{28}$$

Figure 9: The core Twist PIOP for parameter $d \geq 1$, assuming correctness of the one-hot decompositions.

5. At the end of the above instances of the sum-check protocol, for random values $r_{\text{address}}, r_{\text{cycle}}$, the verifier has to evaluate $\widetilde{\mathsf{wa}}(r_{\text{address}}, r_{\text{cycle}})$, $\widetilde{\mathsf{ra}}(r_{\text{address}}, r_{\text{cycle}})$, $\widetilde{\mathsf{Val}}(r_{\text{address}}, r_{\text{cycle}})$, and $\widetilde{\mathsf{Inc}}(r_{\text{cycle}})$. The $\widetilde{\mathsf{wa}}$, $\widetilde{\mathsf{ra}}$ evaluations are obtained directly from the commitments to these polynomials if $d = 1$, or via an invocation of the corresponding virtualization sum-check that relates $\widetilde{\mathsf{ra}}$ or $\widetilde{\mathsf{wa}}$ to its d one-hot factors. The $\widetilde{\mathsf{Inc}}$ evaluation is obtained from the commitment to $\widetilde{\mathsf{Inc}}$. The $\widetilde{\mathsf{Val}}$ evaluation is obtained from the $\widetilde{\mathsf{Val}}$-evaluation sum-check below.
6. **$\widetilde{\mathsf{Val}}$-evaluation sum-check.** $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to compute

$$\widetilde{\mathsf{Val}}(r_{\text{address}}, r_{\text{cycle}}) = \sum_{j' \in \{0,1\}^{\log T}} \widetilde{\mathsf{Inc}}(j') \cdot \widetilde{\mathsf{wa}}(r_{\text{address}}, j') \cdot \widetilde{\mathsf{LT}}(j', r_{\text{cycle}}).$$

7. To perform $\mathcal{V}'s$ check in the final round of this sum-check protocol, the verifier has to evaluate $\widetilde{\mathsf{Inc}}$ at a random point as well as $\widetilde{\mathsf{LT}}$ and $\widetilde{\mathsf{wa}}$. The verifier computes the $\widetilde{\mathsf{LT}}$ evaluation on its own in $O(\log T)$ time. The evaluation of $\widetilde{\mathsf{Inc}}$ is obtained via the commitment to $\widetilde{\mathsf{Inc}}$. The $\widetilde{\mathsf{wa}}$ evaluation is obtained directly from the commitment to $\widetilde{\mathsf{wa}}$ if $d = 1$ or via another invocation of the $\widetilde{\mathsf{wa}}$-virtualization sum-check otherwise (a more efficient approach is to apply standard batching techniques to reduce multiple evaluation claims about a single committed polynomial to one claim).

Fig. 9 continued.

6 Additional technical preliminaries

6.1 Prover costs in elliptic-curve-based SNARKs

Cost of field operations. On modern CPUs, multiplying two elements of a 256-bit prime field using Montgomery multiplication costs between 50 and 100 CPU cycles [Xop]. Field additions for such fields are an order of magnitude cheaper.

Elliptic curve commitments. In order to understand the advantages of the method of increments, it is necessary to explain the prover cost profile in SNARKs that use elliptic curve commitments. Three points are paramount: (1) committing to 0s is "free" (2) commitment to "small" values is faster than committing to "big values", and (3) under appropriate conditions, we can "ignore" the cost of computing evaluation proofs. Details follow.

Let $\ell = \log(n)$ and p be an ℓ-variate multilinear polynomial over a field $\mathbb{F}$. Let $\mathbb{G}$ be a cryptographic group with scalar field equal to $\mathbb{F}$.

With elliptic-curve-based polynomial commitment schemes, the commitment to an ℓ-variate multilinear polynomial p is simply a multi-exponentiation of size $n = 2^\ell$, namely:

$$\prod_{x_i \in \{0,1\}^\ell} g_i^{y_i}, \tag{29}$$

where $y_i = p(x_i)$ is the evaluation p at input $x_i \in \{0,1\}^\ell$. (In this paper, we treat $\mathbb{G}$ as a multiplicative group, so the product in Equation (29) refers to the group operation.) Here, each $g_i \in \mathbb{G}$ is an element of the *commitment key*, a publicly known vector of group elements of length $n = 2^\ell$ that is needed to produce commitments.

Equation (29) is also referred to as a *Pedersen vector commitment*, with the vector at hand being the exponents in Equation (29), i.e., the vector y of evaluations of p as its input ranges over $\{0,1\}^\ell$. Accordingly, we will often refer to the procedure of committing to p as "committing to $n = 2^\ell$ values", those values being the entries of y.

Commitment costs. The "smaller" the y_i values, the faster it is to compute the commitment in Equation (29). Specifically, using Pippenger's bucketing algorithm, for any desired $B \ll n$, roughly one group operation is incurred for each $y_i \in \{0, 1, \ldots, B\}$, two group operations are incurred for each $y_i \in \{B+1, \ldots, B^2\}$, and in general c group operations are required for each $y_i \in \{B^{c-1}+1, \ldots, B^c\}$. In other words, committing to a $c\log(n)$-bit value y_i requires about c group operations.

Accordingly, in order to understand the cost of committing to a vector, it is not enough to analyze the *length* of the committed vector, one must also analyze how big or small the vector's values are.

Notice in particular that if $y_i = p(x_i) = 0$, then $g_i^{y_i} = 1$, and hence the multi-exponentiation in Equation (29) is unaffected by the i'th term. This means that committing to 0s is free for the prover, in the sense that any 0-values that are committed do not alter the commitment whatsoever.

We refer to the number of non-zero entries of y as the *sparsity* of p and of y. The above means that the time to commit to p and y depend only on the sparsity, and not the ambient dimension $n = 2^\ell$.

The ambient dimension of y does factor into other prover costs, discussed shortly, but not into the time required to compute the commitment.

Another important special case is an exponent $y_i = 1$. The total contribution to the commitment of all such exponents can be computed with *exactly* one group operation per entry i with $y_i = 1$, as this contribution is:

$$\prod_{i\colon y_i=1} g_i.$$

Pippenger's bucketing algorithm achieves an amortized cost of *very close to* one group operation per committed value in $\{2, \ldots, B\}$, but not exactly one. The smaller B is relative to the length n of the committed vector y (or more precisely, the number of entries of y outside of $\{0, 1\}$), the closer the amortized cost of commitment is to one group operation per value y_i in $\{2, \ldots, B\}$.

Not only is it fast to commit to small positive values via Equation (29), it is also fast to commit to "small negative values", i.e., values in $\{-B^d, \ldots, -1\}$. To do this, one can in preprocessing invert each element of the original commitment key, so that the commitment key becomes $(g_1, \ldots, g_n, h_1, \ldots, h_n)$, where $h_i = g_i^{-1}$. Then committing to a negative value y_i can be done by committing to the positive value $-y_i$ using group element h_i in place of g_i. The Twist prover will need this in order to commit to "increments" quickly, since increments can be negative.

The cost of computing evaluation proofs. While *commitment time* depends only on the sparsity of p when using elliptic-curve-based polynomial commitment schemes, committed 0s are not literally free. They do affect the time required to compute evaluation proofs.

Specifically, if the committed vector y has length n, then evaluation proofs for multilinear polynomial commitment schemes like HyperKZG and Zeromorph require the prover to compute commitments to (i.e., multi-exponentiations over) a vector v of length n with *random* entries (in fact, several such vectors). If y is sparse and only contains small values, committing to v as required in the evaluation proof is potentially orders of magnitude more expensive than merely committing to y. This is because v is *not* sparse, nor are its entries small.

Furthermore, if using HyperKZG or Zeromorph, committed 0s also affect the size of the "powers-of-tau" structured reference string (SRS) [KZG10] used by these commitment schemes. Specifically, to commit to an ℓ-variate multilinear polynomial with these schemes, one needs an SRS of size 2^ℓ regardless of the sparsity of the committed polynomial.

These issues are less severe for other curve-based commitment schemes like Hyrax [WTS+18], Dory [Lee21], or Sona [STW24], which are all transparent and have commitment keys square root in the size of the committed polynomial. Moreover, Hyrax allows the prover to avoid cryptographic operations entirely when computing evaluation proofs, while Dory and Sona require the prover to perform a number cryptographic operations that is only square root in the size of the committed polynomial (see Section 11 for more information on the concrete efficiency of Dory's evaluation proofs). In addition, both of the above issues (large evaluation proof computation time, and large SRS) can be mitigated further by the standard amortization technique described next.

6.1.1 Amortizing the cost of computing evaluation proofs via homomorphism

Elliptic curve commitment schemes automatically come with extremely powerful amortization properties for evaluation proofs that render this concern moot, at least so long as n is not *too* much bigger than m. For example, let $\ell = \ell_1 + \ell_2$. Rather than committing to p directly, instead commit to 2^{ℓ_1} different ℓ_2-variate multilinear polynomials, where for each $z \in \{0,1\}^{\ell_1}$, the z'th committed polynomial is $g_z(x) = p(z, x)$.

Suppose the verifier requests the evaluation $p(r, r')$ for $r \in \mathbb{F}^{\ell_1}$ and $r' \in \mathbb{F}^{\ell_2}$. The verifier on its own can, via homomorphism, compute a commitment to the ℓ_2-variate polynomial $g(x) = p(r, x)$, and the prover merely needs to provide an evaluation proof for $g(r_2)$. Computing this evaluation proof is roughly 2^{ℓ_1} times cheaper than computing an evaluation proof if p had been committed directly.

The downside of this technique is that the commitment size grows by a factor of 2^{ℓ_1}, and the verifier has to perform 2^{ℓ_1} group exponentiations[5] to homomorphically compute the commitment to $g(x) = p(r, x)$. In many contexts, this is an acceptable increase in verifier costs even for 2^{ℓ_1} in the dozens or hundreds. For example, HyperKZG and Zeromorph evaluation proofs require the verifier to do a couple of dozen group exponentiations regardless, so having the verifier do an extra couple of dozen exponentiations at most doubles verifier costs. And SNARK verifiers do more than merely process commitments, so doubling the verifier's costs to process commitments may mean less than a doubling of total verifier costs.

Comparison to sparse polynomial commitment schemes. Readers may wonder about the relationship between the discussion above and the notion of *sparse polynomial commitment schemes* such as Spark [Set20]. These are polynomial commitment schemes that enable the prover to efficiently commit to an ℓ-variate multilinear polynomial p in time proportional to the number T of inputs $x \in \{0,1\}^{\ell}$ such that $p(x) \neq 0$. We refer to T as the *sparsity* of p. Haven't we just asserted that a simple Pedersen vector commitment achieves exactly this property, and also that the cost of evaluation proofs can be ignored due to amortization techniques? Doesn't that mean that any elliptic-curve based polynomial commitment scheme immediately gives a sparse polynomial commitment scheme?

The answer is that sparse polynomial commitment schemes like Spark are generally targeted at settings where $n = 2^{\ell}$ is so much larger than the sparsity T that the cost of computing evaluation proofs *cannot* be ignored, even given the excellent amortization properties described above.

For example, the situation where $n = T^2$ naturally arises in SNARKs for R1CS or circuit satisfiability, and this is the setting that motivated the development of Spark. In the zkVM context, n turns out to equal $K \cdot T$ where K is the size of the memory being checked. There are two potential differences between our setting and Spark's:

- We are interested in settings where T is in the millions or billions or larger (in a zkVM context, T corresponds roughly to the number of cycles in the VM's execution), but where K may (or may not) be much smaller. For example, in the setting of RISC-V registers, K is just 32. This is a setting where the cost of evaluation proofs can definitely be ignored due to amortization, but this may not be the case if $n \geq T^2$ as considered in Spark.

[5] More precisely, a multi-exponentiation of length 2^{ℓ_1}.

- Spark is a polynomial commitment scheme for *arbitrary* sparse polynomials. Whereas, Twist and Shout do not need to commit to arbitrary sparse polynomials, but rather to polynomials whose evaluations can be viewed as a $T \times K$ matrix with *exactly one non-zero entry per row.*

6.1.2 An analytic cost model: relating group operations to field operations

When committing to a multilinear polynomial p defined over field $\mathbb{F}$ using an elliptic curve group $\mathbb{G}$, $\mathbb{F}$ will always be the *scalar field* of $\mathbb{G}$, which is why we can interpret evaluations of p as *exponents* of group elements in Equation (29). Elements of $\mathbb{G}$ correspond to pairs of elements (x, y) in a *different* field $\mathbb{B}$ called the *base field* of $\mathbb{G}$. And group operations are naturally defined in terms of base field operations. Specifically, multiplying two group elements together typically requires about 6 base-field multiplications [GW20a].

Despite the fact that $\mathbb{B}$ and $\mathbb{F}$ are different fields, often operations in $\mathbb{B}$ and $\mathbb{F}$ have the same cost. For example, if the elliptic curve is BN254, both field implementations involve 256-bit Montgomery arithmetic [Mon85]. This allows us to directly relate the cost of group operations to $\mathbb{F}$-operations: a single operation in $\mathbb{G}$ has roughly the same cost as 6 multiplications in $\mathbb{F}$. We ignore the cost of field additions, as in 256-bit fields they are an order of magnitude cheaper than field multiplications.

Some pairing-friendly curves, like BLS12-381, have a larger base field than scalar field (e.g., the base field is 381 bits for BLS12-381 while the scalar field is only 256 bits). For these fields and curves, the method of increments is still attractive relative to memory-checking techniques. This is because the method of increments has substantially lower commitment costs compared to memory-checking techniques (by commitment costs, we mean the number of group operations required to commit to data). And the bigger the base field of the elliptic curve group, the more expensive those group operations are.

Summary of our analytic cost model. On CPUs, we consider a 256-bit field multiplication to be 40-80 times more expensive than a native multiplication of two 64-bit data types [MvOV01, Chapter 14]. We consider a group operation (often called a group addition, using additive-group terminology) to be roughly 6 times as expensive as a field operation [GW20a]. We treat small committed values as costing exactly one group operation (see Section 2.1 and Section 6.1 for discussion). We treat random values as costing about 11 group operations to commit to (as this is an optimistic bound on the runtime of Pippenger's algorithm at realistic input sizes [EHB22]). We ignore field additions, as well as multiplications by small constants such as 2, since $2x$ can be computed with a single addition: $2x = x + x$.

A scalar multiplication (aka group exponentiation) by an arbitrary field element is equivalent in cost to roughly 400 group operations via the standard double-and-add algorithm, and a pairing evaluation is equivalent in cost to roughly 10 scalar multiplications.

Caveats. Of course, any such cost model will only approximately match actual system performance. Different cryptographic groups have different base field sizes, and real hardware is complicated (e.g., some operations are memory-bound rather than compute bound). However, the above approximate costs are carefully informed by examination of how field operations and group operations are actually computed, as well as by microbenchmarks.

Any inaccuracies in our cost model generally understate our improvements over prior works. For example, we treat all small committed values as costing exactly one group operation, when in fact committing to a 1-value truly requires one group operation, and committing to larger-but-still-small values is somewhat more expensive. Since our protocols commit to more 1-values and fewer larger-but-still-small values than prior work, this cost model understates our speedups.

Indeed, our experiments (Section 11) confirm that the concrete performance of Twist and Shout closely match or exceed the predictions provided by our analytic cost model and exact operation counts.

6.2 Linear-time sum-check proving

Suppose the sum-check protocol is applied to compute $\sum_{x \in \{0,1\}^n} \widetilde{\mathsf{eq}}(r', x) \cdot g(x)$ for some fixed $r' \in \mathbb{F}^n$, where

$$g(x) = \prod_{i=1}^{\ell} p_i(x), \tag{30}$$

and where each p_i is multilinear. Let $N = 2^n$. The standard linear-time sum-check prover algorithm [CTY11, Tha13] has the prover begin by storing arrays $A_1, \ldots, A_\ell$ each of size N with entries indexed by $\{0,1\}^n$, where $A_i(x)$ stores $p_i(x)$. The prover also stores an array B containing all evaluations $\widetilde{\mathsf{eq}}(r, x)$ as x ranges over $\{0,1\}^n$. It is known how to build this array with N field multiplications in total.

In round m of the sum-check protocol, for each $c' \in \{0, 1, 2, \ldots, \ell + 1\}$, the prover must compute

$$s_m(c') = \sum_{y \in \{0,1\}^{n-m}} \mathsf{eq}(r', r_1, \ldots, r_{m-1}, c', y) g(r_1, \ldots, r_{m-1}, c, y), \tag{31}$$

where s_m is the degree-$(\ell + 1)$ univariate polynomial sent by the sum-check prover in round ℓ.

Accordingly, the sum-check prover at the start of each round m makes sure that A_i stores $p_i(r_1, \ldots, r_{m-1}, y)$ as y ranges over $\{0,1\}^{n-(m-1)}$. By Fact 4.1, the following procedure suffices to ensure this. At the end of each round m, when the verifier selects that round's random challenge $r_m \in \mathbb{F}$, the sum-check prover updates each entry $A_i(y)$ for $y \in \{0,1\}^{n-m}$ to:

$$\begin{aligned} A_i(y) \leftarrow p_i(r_1, \ldots, r_m, y) &= (1 - r_m) \cdot p_i(r_1, \ldots, r_{m-1}, 0, y) + r_m \cdot p_i(r_1, \ldots, r_{m-1}, 1, y) \\ &= A_i(0, y) + r_m \cdot (A_i(y, 1) - A_i(y, 0)) \,. \end{aligned} \tag{32}$$

We refer to this procedure as the prover *binding* the m'th variable to r_m. A similar update is performed on the array B of evaluations $\widetilde{\mathsf{eq}}(r', r_1, \ldots, r_m, y)$.

Given the A_i arrays at each round m, the prover can evaluate $g(r_1, \ldots, r_{m-1}, c', y)$ for the $\ell + 2$ relevant points c' and all $y \in \{0,1\}^{n-m}$ with $\ell - 1$ field multiplications per point. Multiplying $g(r_1, \ldots, r_{m-1}, c', y)$ by the value $\widetilde{\mathsf{eq}}(r', r_1, \ldots, r_{m-1}, c', y)$ (which is easily derivable from the array B without additional general field multiplications) costs an additional field multiplication per point, and this counts all field operations needed to compute the quantities appearing in Equation (31).

Hence, the prover's total work across all rounds of the sum-check protocol is $\ell \cdot N$ field multiplications across all binding operations for the arrays $A_1, \ldots, A_\ell$, plus $2N$ operations to compute and then

bind the B array, plus $\ell \cdot (\ell + 2) \cdot N$ field multiplications across all rounds to compute the prover's messages given the arrays $A_1, \ldots, A_\ell$. That is

$$(\ell^2 + 3\ell + 2)N$$

field multiplications in total.

Recent optimizations have brought this cost down. Dao and Thaler [DT24] show how to effectively eliminate the $2N$ operations required to build and bind the B array of $\widetilde{\mathsf{eq}}$ evaluations. And it's well-known that the prover need not consider the evaluation point $c' = 1$ in each round because when the prover is honest, $s_m(1) = s_{m-1}(r_{m-1}) - s_m(0)$. On top of this, Gruen [Gru24] effectively showed that the evaluation point $\ell+1$ can be omitted as well (see Section 6.5 for details). Combining these optimizations, the number of field multiplications performed by the prover falls to roughly

$$(\ell^2 + \ell) \cdot N.$$

6.2.1 A standard optimization for verifier costs: Batching parallel sum-check instances

A standard technique to control proof size when running $t > 1$ parallel sum-check instances is to instead apply a single instance of the sum-check protocol to a random linear combination of the t claims.

This adds at most $t/|\mathbb{F}|$ to the soundness error, and avoids a t-fold increase in proof size. In the context of Twist and Shout, it can also have prover time benefits by allowing for some reuse of work between sum-check instances run in parallel (see for example Section 7.3).

6.3 SNARKs and commitment schemes

SNARKs We adapt the definition provided in [KST22].

Definition 6.1. Consider a relation $\mathcal{R}$ over public parameters, structure, instance, and witness tuples. A non-interactive argument of knowledge for $\mathcal{R}$ consists of PPT algorithms $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ and deterministic $\mathcal{K}$, denoting the generator, the prover, the verifier and the encoder respectively with the following interface.

- $\mathcal{G}(1^\lambda) \to \mathsf{pp}$: On input security parameter λ, samples public parameters pp.
- $\mathcal{K}(\mathsf{pp}, \mathsf{s}) \to (pk, \mathsf{vk})$: On input structure s, representing common structure among instances, outputs the prover key pk and verifier key vk.
- $\mathcal{P}(pk, u, w) \to \pi$: On input instance u and witness w, outputs a proof π proving that $(\mathsf{pp}, \mathsf{s}, u, w) \in \mathcal{R}$.
- $\mathcal{V}(\mathsf{vk}, u, \pi) \to \{0, 1\}$: On input the verifier key vk, instance u, and a proof π, outputs 1 if the instance is accepting and 0 otherwise.

A non-interactive argument of knowledge satisfies *completeness* if for any PPT adversary $\mathcal{A}$

$$\Pr\left[\begin{array}{c|l} \mathcal{V}(\mathsf{vk}, u, \pi) = 1 & \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda), \\ (\mathsf{s}, (u, w)) \leftarrow \mathcal{A}(\mathsf{pp}), \\ (\mathsf{pp}, \mathsf{s}, u, w) \in \mathcal{R}, \\ (pk, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}), \\ \pi \leftarrow \mathcal{P}(pk, u, w) \end{array} \end{array}\right] = 1.$$

A non-interactive argument of knowledge satisfies *knowledge soundness* if for all PPT adversaries $\mathcal{A}$ there exists a PPT extractor $\mathcal{E}$ such that for all randomness ρ

$$\Pr\left[\begin{array}{c|l} \begin{array}{l} \mathcal{V}(\mathsf{vk}, u, \pi) = 1, \\ (\mathsf{pp}, \mathsf{s}, u, w) \notin \mathcal{R} \end{array} & \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda), \\ (\mathsf{s}, u, \pi) \leftarrow \mathcal{A}(\mathsf{pp}; \rho), \\ (pk, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}), \\ w \leftarrow \mathcal{E}(\mathsf{pp}, \rho) \end{array} \end{array}\right] = \mathrm{negl}(\lambda).$$

A non-interactive argument of knowledge is succinct if the verifier's time to check the proof π and the size of the proof π are at most polylogarithmic in the size of the statement proven.

Polynomial commitment scheme We adapt the definition from [BFS20]. Write $\mathbb{F}[\ell]$ for the space of ℓ-variate multilinear polynomials over $\mathbb{F}$. A polynomial commitment scheme for multilinear polynomials is a tuple of four protocols $\mathsf{PC} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Eval})$:

- $\mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda, \ell)$: takes as input ℓ (the number of variables in a multilinear polynomial); produces public parameters pp.
- $\mathcal{C} \leftarrow \mathsf{Commit}(\mathsf{pp}, g)$: takes as input a ℓ-variate multilinear polynomial over a finite field $g \in \mathbb{F}[\ell]$; produces a commitment $\mathcal{C}$.
- $b \leftarrow \mathsf{Open}(\mathsf{pp}, \mathcal{C}, g)$: verifies the opening of commitment $\mathcal{C}$ to the ℓ-variate multilinear polynomial $g \in \mathbb{F}[\ell]$; outputs $b \in \{0, 1\}$.
- $b \leftarrow \mathsf{Eval}(\mathsf{pp}, \mathcal{C}, r, v, \ell, g)$ is a protocol between a PPT prover $\mathcal{P}$ and verifier $\mathcal{V}$. Both $\mathcal{V}$ and $\mathcal{P}$ hold a commitment $\mathcal{C}$, the number of variables ℓ, a scalar $v \in \mathbb{F}$, and $r \in \mathbb{F}^\ell$. $\mathcal{P}$ additionally knows an ℓ-variate multilinear polynomial $g \in \mathbb{F}[\ell]$. $\mathcal{P}$ attempts to convince $\mathcal{V}$ that $g(r) = v$. At the end of the protocol, $\mathcal{V}$ outputs $b \in \{0, 1\}$.

Definition 6.2. A tuple of four protocols $(\mathsf{Gen}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Eval})$ is an extractable polynomial commitment scheme for multilinear polynomials over a finite field $\mathbb{F}$ if the following conditions hold.

- **Completeness.** For any ℓ-variate multilinear polynomial $g \in \mathbb{F}[\ell]$ and every $r \in \mathbb{F}^\ell$, let $v = g(r)$. Then
$$\Pr\left\{\begin{array}{c} \mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda, \ell);\ \mathcal{C} \leftarrow \mathsf{Commit}(\mathsf{pp}, g): \\ \mathsf{Eval}(\mathsf{pp}, \mathcal{C}, r, v, \ell, g) = 1 \wedge v = g(r) \end{array}\right\} \geq 1 - \mathrm{negl}(\lambda)$$
- **Binding.** For any PPT adversary $\mathcal{A}$, size parameter $\ell \geq 1$,
$$\Pr\left\{\begin{array}{c} \mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda, \ell);\ (\mathcal{C}, g_0, g_1) = \mathcal{A}(\mathsf{pp}); \\ b_0 \leftarrow \mathsf{Open}(\mathsf{pp}, \mathcal{C}, g_0);\ b_1 \leftarrow \mathsf{Open}(\mathsf{pp}, \mathcal{C}, g_1): \\ b_0 = b_1 \neq 0 \wedge g_0 \neq g_1 \end{array}\right\} \leq \mathrm{negl}(\lambda)$$

- **Knowledge soundness.** Eval is a succinct argument of knowledge for the following NP relation given $\mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda, \ell)$.

$$\mathcal{R}_{\mathsf{Eval}}(\mathsf{pp}) = \{\langle (\mathcal{C}, r, v), (g) \rangle : g \in \mathbb{F}[\ell] \wedge g(r) = v \wedge \mathsf{Open}(\mathsf{pp}, \mathcal{C}, g) = 1\}$$

6.4 Polynomial IOPs and polynomial commitments

Modern SNARKs are constructed by combining a type of interactive protocol called a *polynomial IOP* [BFS20] with a cryptographic primitive called a *polynomial commitment scheme* [KZG10]. The combination yields a succinct *interactive* argument, which can then be rendered non-interactive via the Fiat-Shamir transformation [FS86], yielding a SNARK.

Roughly, a polynomial IOP is an interactive protocol where, in one or more rounds, the prover may "send" to the verifier a very large polynomial g. Because g is so large, one does not wish for the verifier to read a complete description of g. Instead, in any efficient polynomial IOP, the verifier only "queries" g at one point (or a handful of points). This means that the only information the verifier needs about g to check that the prover is behaving honestly is one (or a few) evaluations of g.

In turn, a polynomial commitment scheme enables an untrusted prover to succinctly *commit* to a polynomial g, and later provide to the verifier any evaluation $g(r)$ for a point r chosen by the verifier, along with a proof that the returned value is indeed consistent with the committed polynomial. Essentially, a polynomial commitment scheme is exactly the cryptographic primitive that one needs to obtain a succinct argument from a polynomial IOP. Rather than having the prover send a large polynomial g to the verifier as in the polynomial IOP, the argument system prover instead cryptographically commits to g and later reveals any evaluations of g required by the verifier to perform its checks.

Costs of some specific polynomial commitment schemes. The following elliptic-curve-based polynomial commitments are of particular interest to us. Two are HyperKZG and Zeromorph [ZSC24, KT23]. These are homomorphic commitment schemes for multilinear polynomials. The commitment consists of one group element in a pairing friendly group $\mathbb{G}_1$, and committing to an ℓ-variate multilinear polynomial p consists of applying a multi-scalar multiplication (MSM) to the vector of evaluations of p across all inputs in $\{0,1\}^\ell$. The commitment key is the powers-of-tau SRS (also used in KZG commitments [KZG10]) of size $N = 2^\ell$. Evaluation proofs have logarithmic size and verifying them requires performing two or three pairings and an MSM of logarithmic size. Computing the evaluation proof requires, for each $i = 1, \ldots, \ell$, a constant number of MSMs of length 2^i (the scalars in this MSM are random field elements, if the evaluation point is random).

Dory [Lee21] is a transparent elliptic-curve-based polynomial commitment scheme that also uses pairings. An attractive property of Dory is that its commitment key is sublinear size, consisting of only $\sqrt{N}$ elements of $\mathbb{G}_1$ and $\sqrt{N}$ elements of $\mathbb{G}_2$. Committing to an ℓ-variate multilinear polynomial entails performing roughly $\sqrt{N}$ MSMs each of length $\sqrt{N}$ (as with HyperKZG and Zeromorph, the scalars the MSMs are applied to comprise evaluations of p across all inputs in $\{0,1\}^\ell$). In addition, committing requires computing a *multi-pairing* of size $\sqrt{N}$.[6] Dory has a transparent linear-time pre-processing phase, which produces a verification key of size just $O(\log N)$. Evaluation proofs

[6] A multi-pairing of size S is an expression of the form $\prod_{i=1}^{S} e(a_i, b_i)$ where each $a_i \in \mathbb{G}_1$ and $b_i \in \mathbb{G}_2$.

consist of $6 \log N$ target group elements and verifying them involves logarithmically many scalar multiplications in the target group.

6.5 An optimization of Gruen for zero-check

Suppose the polynomial $g(x)$ in Equation (6) has degree d in each variable of x. Then $\widetilde{\mathsf{eq}}(r, x) \cdot g(x)$ has degree $d+1$ in each variable of x. As a consequence, when applying the sum-check protocol to $\widetilde{\mathsf{eq}}(r, x) \cdot g(x)$, the prover's message $s_i(X)$ in each round i is a univariate polynomial of degree $d+1$. This has implications for prover time: the prover must evaluate s_i at $d+2$ points, say $\{0, 1, \ldots, d+1\}$ (in fact, the evaluation $s_i(1)$ can be omitted and/or had "for free" as it is quickly derivable from $s_i(0)$ and s_{i-1}. See Section 6.2 for details).

Gruen [Gru24, Section 3] shows that the sum-check protocol can be modified so that in each round i the prover only has to compute a degree-d polynomial s'_i, not the degree $d+1$ polynomial s_i (and s_i can then be derived from s'_i, in time that depends only on d). This saves the prover the work of evaluating the relevant polynomial at one point (say, point $d+1$). (Gruen describes his optimization as having the prover send s'_i instead of s_i, but this requires modifying the standard sum-check verifier so as to process s'_i rather than s_i. We prefer to leave the standard sum-check verification procedure unchanged).

The idea is roughly to define s'_i to "leave out" the contribution of $\widetilde{\mathsf{eq}}(r, x)$ to s_i when defining s'_i. This reduces the degree of s_i by one. More precisely, since $\widetilde{\mathsf{eq}}(r, x)$ factors into a product of terms where each term depends on a single variable, s'_i leaves out the contribution of variable i to $\widetilde{\mathsf{eq}}(r, x)$. This contribution is independent of the other variables still being summed over in round i, so the prover can "add this contribution back in" (i.e., compute s_i from s'_i) in time independent of the number of terms being summed.

Specifically, if $(\mathsf{r}_1, \ldots, \mathsf{r}_{i-1})$ denotes the randomness chosen by the sum-check verifier in rounds $1, \ldots, i-1$, then recall that

$$\begin{aligned} s_i(c) &= \sum_{x' \in \{0,1\}^{\ell - i}} \widetilde{\mathsf{eq}}(r, \mathsf{r}_1, \ldots, \mathsf{r}_{i-1}, c, x') \cdot g(\mathsf{r}_1, \ldots, \mathsf{r}_{i-1}, c, x') \\ &= A \cdot B(c) \cdot C(c), \end{aligned}$$

where

$$\begin{aligned} A &= \prod_{j=1}^{i-1} \left(r_j \mathsf{r}_j + (1 - r_j)(1 - \mathsf{r}_j) \right), \\ B(c) &= r_i c + (1 - r_i)(1 - c), \\ C(c) &= \sum_{x' \in \{0,1\}^{\ell - i}} \widetilde{\mathsf{eq}}(r_{i+1}, \ldots, r_\ell, x') \, g(\mathsf{r}_1, \ldots, \mathsf{r}_{i-1}, c, x'). \end{aligned}$$

The modified polynomial $s'_i(X)$ for round i is

$$s'_i(c) = A \cdot C(c).$$

Crucially, for each $c \in \{0, \ldots, d+1\}$, $s_i(c) = s'_i(c) \cdot B(c)$. Effectively, this modification has removed from the prover's message $s_i(c)$ the degree-1 factor $B(c)$ that the i'th variable of x contributed to the term $\widetilde{\mathsf{eq}}(r, x)$.

The prover can compute $s'_i(c)$ for $d+1$ points via the standard linear-time sum-check proving algorithm (Section 6.2). These $d+1$ evaluations fully specify s'_i since s'_i has degree d rather than $d+1$ (note that, per a standard observation, the evaluation $s'_i(0)$ can be computed in constant time from $s'_i(1)$ and $s_i(0)$).

So once the prover has computed s'_i, in time that depends only on d it can compute $s_i(c)$ at the $d+2$ points $c \in \{0, \ldots, d+1\}$, thereby fully specifying s_i.

7 Fast Shout prover implementation (small memories)

This section explains how to implement the prover in the various invocations of the sum-check protocol in Shout. Our treatment assumes familiarity with the standard linear-time sum-check prover implementation [CTY11, Tha13] covered in the preliminaries (Section 6.2). In this section, we are generally focused upon the case that $K = o(T)$, and hence we do not seek to avoid additive terms in the prover time of $O(K)$. This is what we mean by "small" memories in the section title.

Because Shout with $d = 1$ is a special case, we present the fast prover for $d = 1$ in Section 7.1, before turning to the general case (Figures 7 and 8) in Sections 7.2 and 7.3.

7.1 Core Shout prover for $d = 1$

An algorithm for small or unstructured memories. Recall that the core Shout PIOP prover with $d = 1$ (Figure 5) applies the sum-check protocol to prove that

$$\widetilde{\mathsf{rv}}(r_{\text{cycle}}) = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, r_{\text{cycle}}) \cdot \widetilde{\mathsf{Val}}(k). \tag{33}$$

With T field multiplications, the prover can compute a vector E storing $\widetilde{\mathsf{eq}}(j, r_{\text{cycle}})$ as j ranges over $\{0,1\}^{\log T}$ [VSBW13] (see Lemma 4.1). Given E, computing a vector F storing all $\widetilde{\mathsf{ra}}(k, r_{\text{cycle}})$ evaluations for all $k \in \{0,1\}^{\log K}$ can be done with only additions and lookups into E. Specifically, $\widetilde{\mathsf{ra}}(k, r_{\text{cycle}}) = \sum_{j:\, \widetilde{\mathsf{ra}}(k,j)=1} \widetilde{\mathsf{eq}}(j, r_{\text{cycle}})$. Given this array, the standard linear-time sum-check proving algorithm (Section 6.2) requires $3K$ field multiplications.[7] Hence, the total number of field multiplications done by the core Shout PIOP prover is: $T + 3K$.

Theorem 7.1. *The prover in the core Shout PIOP with $d = 1$ (Figure 5) can be implemented in $3K + T$ field multiplications.*

An algorithm for large, structured memories. If $K \gg T$, we would not be happy with the additive $3K$ term in the prover runtime above. However, we generally do not expect Shout to be applied with d set to 1 in this case, as it would either lead to a very large commitment key (in the case of an elliptic curve commitment) or high time to compute commitments (in the case of a

[7] Section 6.2 states a time bound of $2K$ field multiplications, but this does not account for the cost of computing the claimed answer, in this case $\widetilde{\mathsf{rv}}(r_{\text{cycle}})$. Given the array F, $\widetilde{\mathsf{rv}}(r_{\text{cycle}})$ can be computed with K additional field multiplications.

hashing-based commitment scheme, where 0s are not free to commit to). Still, for completeness, we outline a prover implementation that avoids this linear-in-K term. This implementation is an immediate consequence of the sparse-dense sum-check protocol from [STW24, Appendix G].

Indeed, if $K \gg T$ the vector F above is *sparse*: only (at most) T of its entries are non-zero. Suppose that $K \leq T^c$ for some constant $c > 0$. The sparse-dense sum-check protocol of [STW24, Appendix G] identifies conditions on $\widetilde{\mathsf{Val}}$ that guarantee that the prover (in the sum-check protocol invoked to establish Equation (33) holds) runs in time $O(c \cdot T)$. [STW24, Appendix G] also showed that important lookup tables of size 2^{64} arising in Jolt satisfy the necessary structural conditions for this runtime bound to hold. See Section 8.1 for further details.

7.2 Core Shout prover (general d, small memories)

Recall that the core Shout PIOP for any $d \geq 1$ (Figure 7) is comprised of two invocations of the sum-check protocol. The read-checking sum-check (Equation 15) is identical to the sum-check for Shout with $d = 1$ (Equation 33), so the same prover algorithm can be applied. We instead turn our attention to the $\widetilde{\mathsf{ra}}$-virtualization sum-check.

7.2.1 $\widetilde{\mathsf{ra}}$-virtualization

Let $r_{\text{address}} = (r^{(1)}_{\text{address}}, \ldots, r^{(d)}_{\text{address}}) \in \left(\mathbb{F}^{\log(K)/d}\right)^d$ denote the randomness chosen over the $\log(K)$ rounds of the read-checking sum-check protocol. Recall that the $\widetilde{\mathsf{ra}}$-virtualization sum-check proves the following:

$$\widetilde{\mathsf{ra}}(r_{\text{address}}, r_{\text{cycle}}) = \sum_{j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r_{\text{cycle}}, j) \cdot \prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(r^{(i)}_{\text{address}}, j) \tag{34}$$

Before the sum-check protocol even begins, the prover can compute the following array E^* with one entry per $j \in \{0,1\}^{\log T}$.

$$E^* \text{ stores } \widetilde{\mathsf{eq}}(r_{\text{cycle}}, j) \text{ for all } j \in \{0,1\}^{\log T}. \tag{35}$$

With a total of $dK^{1/d}$ multiplications, the prover can compute d arrays $E_1, \ldots, E_d$, where the j'th entry of E_i stores $\widetilde{\mathsf{ra}}_i(r^{(i)}_{\text{address}}, j)$ (as j ranges over $\{0,1\}^{\log T}$). Indeed, $\widetilde{\mathsf{ra}}_i(r^{(i)}_{\text{address}}, j)$ simply equals $\widetilde{\mathsf{eq}}(r^{(i)}_{\text{address}}, k^{(i)})$ where $k = (k^{(1)}, \ldots, k^{(d)}) \in \{0,1\}^{\log K}$ is the register read at cycle j. And it's known how to compute a size-$K^{1/d}$ array storing all evaluations $\widetilde{\mathsf{eq}}(r^{(i)}_{\text{address}}, k^{(i)})$ with $K^{1/d}$ field multiplications (see Lemma 4.1). Each entry of E_i can be computed with one lookup into its respective size-$K^{1/d}$ array.

With these arrays E_i in hand, as well as the array E^* computed before round one (see Expression (35)), the standard linear-time sum-check proving algorithm (Section 6.2) implements the sum-check prover applied to compute Expression (34) with about $(d^2 + d + 1)T$ field multiplications: d^2T for evaluating $s_j(0), s_j(2), s_j(3), \ldots, s_j(d)$ in each round $j = 1, \ldots, \log(T)$, dT for binding the arrays E_i round-over-round, and T for binding the array E^* round-over-round. Now-standard optimizations [DT24, Gru24] reduce this to d^2T, as the optimization of [Gru24, Section 3] covered in Section 6.5 eliminates the need to evaluate $s_j(d)$ and [DT24] eliminates the cost of binding the array E^*.

Below, we discuss a further optimization that effectively eliminates the cost of binding E_i when $K^{1/d} = o(T)$, bringing the total number of prover multiplications down to only about $(d^2 - d)T$ when $K = o(T)$.

An optimization leveraging small memory size. When $K^{1/d} = o(T)$, the following additional optimization applies. Since $\widetilde{\mathsf{ra}}_i(r^{(i)}_{\text{address}}, j)$ takes on at most $K^{1/d}$ distinct values across all $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{ra}}_i(r^{(i)}_{\text{address}}, r_{\log(K)+1}, \ldots, r_m, j)$ takes on at most $K^{2^m/d}$ distinct values as j ranges over $\{0,1\}^{\log(T)-m}$. In fact, each of these $K^{2^m/d}$ values is a size-$2^m/d$ *sum* of values from a set $\mathcal{S}$ of size $K^{1/d} \cdot 2^m$ quantities that can all be computed in $O(K^{1/d}2^m)$ time. Namely, $\mathcal{S}$ consists of each of the $K^{1/d}$ distinct values in $\widetilde{\mathsf{ra}}_i(r^{(i)}_{\text{address}}, j)$, times a value of the form $\widetilde{\mathsf{eq}}(r_1, \ldots, r_m, j')$ as j' ranges over $\{0,1\}^m$. This means that as long as $K^{1/d} \cdot 2^m \ll T$, we can speed up the task of binding $\widetilde{\mathsf{ra}}_i$ for each of the first m rounds. Rather than costing $T/2^j$ field multiplications at the end of round j, it can instead cost $O(K^{1/d} \cdot 2^m)$ field multiplications.

To summarize the costs of the core Shout prover for general d:

- $5K$ field multiplications to implement the read-checking sum-check.
- T multiplications to compute the array E^*.
- dT multiplications to bind the arrays of $\widetilde{\mathsf{ra}}_1, \ldots, \widetilde{\mathsf{ra}}_d$ evaluations over the $\log T$ rounds of the $\widetilde{\mathsf{ra}}$-virtualization sum-check. This cost falls to $o(T)$ if $K^{1/d} = o(T)$.
- d^2T multiplications given the above arrays to evaluate the prover's messages over the $\log T$ rounds of the $\widetilde{\mathsf{ra}}$-virtualization sum-check.

We have established the following theorem.

Theorem 7.2. *For $d > 1$, the core Shout PIOP prover (Figure 7) can be implemented with $(d^2 + d + 1)T + 5K + o(T)$ field multiplications. If $K^{1/d} = o(T)$, this falls to $(d^2 + 1)T + 5K + o(T)$.*

7.3 Booleanity-checking and one-hot-encoding-checking

Here is how to efficiently implement the prover in the PIOP of Figure 8 when an additive $O(K^{1/d}\log(K))$ term in the prover cost is acceptable. When $K^{1/d}\log(K)$ is bigger than T, one should instead use our generalization of the sparse-dense sum-check protocol given later, in Section 8 (that protocol has a much better dependence on K, but a worse dependence on T, by a significant constant factor).

Let us start with the Booleanity-checking sum-check invocation (Line 3 of Figure 8), which for each $i = 1, \ldots, d$ applies the sum-check protocol to confirm that

$$0 = \sum_{k \in \{0,1\}^{\log(K)/d},\ j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r, k)\widetilde{\mathsf{eq}}(r', j)\left(\widetilde{\mathsf{ra}}_i(k, j)^2 - \widetilde{\mathsf{ra}}_i(k, j)\right).$$

We bind the $\log(K)/d$ variables of k first, followed by the $\log T$ variables of j.

First $\log(K)/d$ rounds. Before the first round, the prover builds arrays B and D of size $K^{1/d}$ and T respectively, storing the following values:

$$B \text{ stores } \mathsf{eq}(r, k) \text{ for all } k \in \{0,1\}^{\log(K)/d} \tag{36}$$

and

$$D \text{ stores } \mathsf{eq}(r', j) \text{ for all } j \in \{0,1\}^{\log T}. \tag{37}$$

By Lemma 4.1, computing B requires $K^{1/d}$ field multiplications, while computing D requires T field multiplications.

At the end of round m, let $\mathsf{r} = (\mathsf{r}_1, \ldots, \mathsf{r}_m)$ denote the random values chosen by the sum-check prover in the first m rounds. Via standard binding techniques (Section 6.2), the prover can ensure that at the end of round m, the array B has shrunk to length $K^{1/d}/2^m$, and for every $k' \in \{0,1\}^{\log(K)/d-m}$, $B[k']$ stores $\widetilde{\mathsf{eq}}(r, \mathsf{r}_1, \ldots, \mathsf{r}_m, k')$.

Recall that at the end of any round $m \leq \log(K)/d$ when r_m is selected, for each $y_{\mathsf{cycle}} \in \{0,1\}^{\log T}$, there is exactly one $y'_{\mathsf{mem}} \in \{0,1\}^{\log(K)/d-m}$ for which $\widetilde{\mathsf{ra}}_i(\mathsf{r}_1, \ldots, \mathsf{r}_m, y'_{\mathsf{mem}}, y_{\mathsf{cycle}})$ is non-zero, and if register $k \in \{0,1\}^{\log K}$ was read in cycle y_{cycle}, then this non-zero value equals

$$\widetilde{\mathsf{eq}}((k_1, \ldots, k_m), (\mathsf{r}_1, \ldots, \mathsf{r}_m)).$$

The prover can maintain an array F that at the end of round m has size 2^m and stores all 2^m such values, i.e.,

$$F \text{ stores } \widetilde{\mathsf{eq}}((k_1, \ldots, k_m), (\mathsf{r}_1, \ldots, \mathsf{r}_m)) \text{ for all } (k_1, \ldots, k_m) \in \{0,1\}^m \tag{38}$$

Via standard techniques (see Lemma 4.1), maintaining F costs $K^{1/d}$ field multiplications across the first $\log(K)/d$ rounds.

Next, consider a specific i from the set $\{1, \ldots, d\}$. The prover can, with only additions, before the protocol begins, compute a size-$K^{1/d}$ array G_i that for $k \in \{0,1\}^{\log(K)/d}$ stores

$$G_i[k] = \sum_{j \in \{0,1\}^{\log T} :\, \widetilde{\mathsf{ra}}_i(k,j)=1} D[j].$$

In the middle of round m, let us use $B[c', k']$ as shorthand for

$$(1 - c') \cdot B[0, k'] + c' \cdot B[1, k'].$$

and $F[k_1, \ldots, k_{m-1}, c']$ as shorthand for

$$F[k_1, \ldots, k_{m-1}] \cdot \widetilde{\mathsf{eq}}(k_m, c'),$$

and where in the middle of round $m = 1$, the factor $F[k_1, \ldots, k_{m-1}]$ is simply 1.

The prover's message s_m in round $m \leq \log(K)/d$ of sum-check for the i'th Booleanity check satisfies that $s_m(c)$ equals:

$$\sum_{k=(k_1,\ldots,k_m,k') \in \{0,1\}^m \times \{0,1\}^{\log(K)/d-m}} G_i[k] \cdot B[c, k'] \cdot \left(F[k_1, \ldots, k_{m-1}, c]^2 - F[k_1, \ldots, k_{m-1}, c]\right). \tag{39}$$

In round m, the prover has to evaluate $s_m(c)$ for all relevant values of c. Using Gruen's technique (Section 6.5), there are 2 relevant values of c, say $c \in \{0, 2\}$. Given the arrays G_i, B, and F,

both $s_m(0)$ and $s_m(2)$ can be computed via Expression (39) with the following total number of multiplications:

$$2^m + 2K^{1/d}.$$

Here, the 2^m multiplications are needed to compute $F[k_1, \ldots, k_{m-1}, c]^2$ from $F[k_1, \ldots, k_{m-1}, c]$ for all 2^m relevant values of $(k_1, \ldots, k_{m-1}, c)$.

The above description refers to implementing the Booleanity-checking sum-check for a single i. That sum-check in fact must be applied d times (once for each $i = 1, \ldots, d$). However, a full d-fold cost increase does not quite occur, since several arrays can be reused between for all $i = 1, \ldots, d$. This holds in particular for arrays B, D, and F.

In summary, across the first $\log(K)/d$ rounds of the sum-check protocol, the prover's costs are as follows.

- $K^{1/d}$ field multiplications to build the array B.
- T multiplications to build D.
- $K^{1/d}$ multiplications to bind the array B across all rounds.
- $K^{1/d}$ multiplications to build up the array F across all $\log(K)/d$ rounds.
- $K^{1/d}$ multiplications to $F[k_1, \ldots, k_{m-1}, c]^2$ for all relevant values of $(k_1, \ldots, k_{m-1}, c)$ across all rounds m.
- For each $i = 1, \ldots, d$, $2K^{1/d} \log(K^{1/d})$ field multiplications to evaluate $s_1(c), \ldots, s_m(c)$ across all $\log(K)/d$ rounds for $c \in \{0, 2\}$ given the above arrays. Across all $i = 1, \ldots, d$, this is $2K^{1/d} \cdot \log(K)$ multiplications total.

Summing the above yields $T + (2\log(K) + 4) \cdot K^{1/d}$ field multiplications in total for the first $\log(K)/d$ rounds for all d Booleanity-checking sum-check invocations across all $i = 1, \ldots, d$.

Last $\log T$ rounds. The last $\log T$ rounds of sum-check are used to compute

$$\begin{aligned} &\widetilde{\mathsf{eq}}(r, \mathsf{r}) \cdot \sum_{j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r'_{\text{cycle}}, j) \left(\widetilde{\mathsf{ra}}_i(\mathsf{r}, j)^2 - \widetilde{\mathsf{ra}}_i(\mathsf{r}, j) \right) \\ &\quad = \widetilde{\mathsf{eq}}(r, \mathsf{r}) \cdot \sum_{j \in \{0,1\}^{\log T}} D[j] \cdot \left(\widetilde{\mathsf{ra}}_i(\mathsf{r}, j)^2 - \widetilde{\mathsf{ra}}_i(\mathsf{r}, j) \right) \end{aligned} \tag{40}$$

Given the contents of the size-$K^{1/d}$ array F (Expression (38)) at the end of round $\log(K)/d$, the prover can, with no further multiplications, construct for each $i = 1, \ldots, d$, an array H_i of length T such that

$$H_i[j] = \widetilde{\mathsf{ra}}_i(\mathsf{r}, j).$$

Indeed, if k is the register read at cycle j then

$$\widetilde{\mathsf{ra}}_i(\mathsf{r}, j) = \widetilde{\mathsf{eq}}(k, \mathsf{r}),$$

and at the end of round $\log(K)/d$ this value is simply equal to $F[k]$. Then Expression (40) equals:

$$\widetilde{\mathsf{eq}}(r, \mathsf{r}) \cdot \sum_{j \in \{0,1\}^{\log T}} D[j] \cdot (H_i[j] \cdot H_i[j] - H_i[j]) \,.$$

Given the arrays H_i and D, the standard linear-time sum-check proving algorithm (Section 6.2) implements the final $\log(T)$ rounds of the sum-check protocol (i.e., applies the sum-check protocol to compute Expression (40)) with the following costs, for each $i = 1, \ldots, d$.

- T field multiplications to bind the array D across all $\log T$ rounds.
- dT field multiplications to bind the arrays $H_1, \ldots H_d$ across all $\log T$ rounds.
- $2 \cdot 2 \cdot d \cdot T$ field multiplications suffice to complete the prover's work given the above arrays.

Hence, without further optimization, the total cost for the prover of the last $\log T$ rounds is $(5d+1)T$ field multiplications. Added to the cost of the first $\log K$ rounds, this is $(5d+2)T + O(K^{1/d} \log(K))$ field multiplications. However, further significant further optimizations are possible.

Further optimizations.

- So long as $\widetilde{\mathsf{ra}}_1, \ldots, \widetilde{\mathsf{ra}}_d$ are committed before the core Shout PIOP (Figure 7) commences, one can set r' in Figure 8 to r_{cycle}, i.e., share verifier-chosen randomness between the core Shout PIOP and the one-hot-encoding-checking PIOP. This makes the array D (Expression (37)) built by the Booleanity-checking prover the same as the array E^* (Expression (35)) built by the core Shout PIOP prover. This eliminates T field multiplications from the cost of the Booleanity-checking prover (and thereby renders the number of prover field multiplications for the first $\log(K/d)$ rounds independent of T).
- The techniques of Dao and Thaler [DT24] directly apply to reduce the cost of binding the array $D = E^*$ to a low-order number of field multiplications (i.e., $O(\sqrt{T})$ of them). Ignoring low-order terms, this eliminates another T field multiplications from the prover's cost.
- Exactly as in the $\widetilde{\mathsf{ra}}$-virtualization sum-check of the core Shout PIOP (Section 7.2.1), in round $\log(K)/d + m$ of sum-check, the number of distinct values in each array $H_1, \ldots, H_d$ is at most K^{2^m}, and each of these values is a sum of at most 2^m elements of a set of size $2^m \cdot K$ (and all quantities in this set can be computed in time $O(K2^m)$). This can be exploited to save most of the T multiplications devoted to binding each of the arrays $H_1, \ldots, H_d$. It also saves the work of squaring $H_1[c, x'], \ldots, H_d[c, x']$ for each $x' \in \{0,1\}^{\log(T)-t}$ in each sum-check round, for each evaluation point c. This is a total of $2dT$ multiplications saved in total.

This brings to the total number of prover multiplications for Booleanity-checking down from $(5d+2)T + O(K^{1/d} \log K)$ to essentially $3dT + O(K^{1/d} \log K)$.

$\widetilde{\mathsf{raf}}$-evaluation sum-check when $d = 1$ (Line 6 of Figure 6). This is a $\log(K)$-round sum-check protocol. The prover sends a degree-2 univariate polynomial in each round. It's easy to make the prover run in time $O(K)$ once given an E array of length K, that stores at cell $k' \in \{0,1\}^{\log(K)}$ the

value $\widetilde{\mathsf{ra}}(k', r'_{\text{cycle}})$. Observe that

$$\widetilde{\mathsf{ra}}(k', r'_{\text{cycle}}) = \sum_{j \in \{0,1\}^{\log T} : \widetilde{\mathsf{ra}}(k', j) = 1} \widetilde{\mathsf{eq}}(j, r'_{\text{cycle}}). \tag{41}$$

Hence, if given an array D' of size T storing all values of them form $\widetilde{\mathsf{eq}}(j, r'_{\text{cycle}})$ as j ranges over $\{0,1\}^{\log T}$, the array E can be computed with no additional multiplications. D' can be computed with T field multiplications.

However, if $K = o(T)$, this procedure can be optimized further to avoid even the T multiplications needed to build D', using techniques similar to those in [DT24] that exploit multiplicative structure in the definition of $\widetilde{\mathsf{eq}}$ (Equation (3)). This is done by avoiding building the "full" size-T array D', whose sole purpose is to help initialize the array E. Instead, the prover builds a smaller array D'' of size $T/2^\ell$ whose definition is identical to D' but "leaves out" the last ℓ coordinates of j and r'_{cycle}. Specifically, for each $j' \in \{0,1\}^{\log(T)-\ell}$, letting $r = r'_{\text{cycle}}$,

$$D''[j'] \leftarrow \prod_{m=1}^{\log(T)-\ell} \widetilde{\mathsf{eq}}(r_m, j'_m).$$

The prover also builds an array D''' of size 2^ℓ the "incorporates" the last ℓ coordinates that were left out of the definition of D''. That is, for all $j'' \in \{0,1\}^i$,

$$D'''[j''] \leftarrow \prod_{m=1}^{i} \widetilde{\mathsf{eq}}(r_{\log(T)-m+\ell}, j''_\ell).$$

Then, for each register address $k' \in \{0,1\}^{K/d}$, the prover computes 2^ℓ quantities, one per $j'' \in \{0,1\}^\ell$, namely:

$$v_{k',j''} \coloneqq \sum_{j=(j',j'') \in \{0,1\}^{\log(T)-\ell} \times \{0,1\}^\ell : : \widetilde{\mathsf{ra}}(k', j) = 1} D''[j'].$$

Then recalling from Equation (41) that

$$D'[k'] = \sum_{j \in \{0,1\}^{\log T} : \widetilde{\mathsf{ra}}(k', j) = 1} \widetilde{\mathsf{eq}}(r'_{\text{cycle}}, j),$$

it follows that

$$D'[k'] = \sum_{j'' \in \{0,1\}^\ell} D'''[j''] \cdot v_{k',j''}.$$

The cost of this protocol for building E_i is $T/2^\ell + 2^{\ell+1}$ field multiplications. Minimizing this quantity, the cost to compute the array E is $O(\sqrt{TK}) = o(T)$ multiplications.

Hamming-weight-one check for general d (Line 4 of Figure 8). This sum-check (for general d) is similar to the $\widetilde{\mathsf{raf}}$-evaluation sum-check for $d = 1$, but even simpler, as the prover only sends a degree-1 polynomial in each round. Given the arrays E_i computed for the $\widetilde{\mathsf{raf}}$-evaluation sum-check, the Hamming-weight-one sum-check prover can be implemented with $O(dK^{1/d})$ field multiplications if $dK^{1/d} = o(T)$.

$\widetilde{\mathsf{raf}}$-evaluation sum-check for $d > 1$ (Line 5 of Figure 8). Recall that here the sum-check protocol is applied to compute:

$$\sum_{k=(k_1,\ldots,k_d)\in\left(\{0,1\}^{\log(K)/d}\right)^d,\ j\in\{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r'_{\text{cycle}}, j)\left(\sum_{i=1}^{d}\sum_{\ell=0}^{\log(K)/d-1} 2^{(i-1)\cdot\log(K)/d+\ell}\cdot k_{i,\ell}\right)\cdot\prod_{i=1}^{d}\widetilde{\mathsf{ra}}_i(k_i,j).$$

This is similar to the read-checking sum-check in Shout, which was applied to compute:

$$\sum_{k=(k_1,\ldots,k_d)\in\left(\{0,1\}^{\log(K)/d}\right)^d,\ j\in\{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r_{\text{cycle}}, j)\left(\prod_{i=1}^{d}\widetilde{\mathsf{ra}}_i(k_i,j)\right)\cdot\widetilde{\mathsf{Val}}(k).$$

The only difference is that the $\widetilde{\mathsf{raf}}$-evaluation sum-check replaces $\widetilde{\mathsf{Val}}(k)$ with

$$\sum_{i=1}^{d}\sum_{\ell=0}^{\log(K)/d-1} 2^{(i-1)\cdot\log(K)/d+\ell}\cdot k_{i,\ell},$$

which is the MLE of the function that maps k to $\text{int}(k)$.

In applications, one can set the random value r_{cycle} chosen by the verifier before the read-checking PIOP to equal r'_{cycle}. In this case, the prover in the $\widetilde{\mathsf{raf}}$-evaluation sum-check can be implemented with only K field multiplications of additional work, by batching Shout's core read-checking sum-check instance with the $\widetilde{\mathsf{raf}}$-evaluation sum-check per Section 6.2.1. Effectively, at the start of both sum-checks, the prover simply replaces $\widetilde{\mathsf{Val}}(k)$ with $\widetilde{\mathsf{Val}}(k) + z\cdot\widetilde{\text{int}}(k)$, where z is randomness used in the standard batching technique of Section 6.2.1.

7.4 Cost summary for the combined Shout prover

Suppose $K = o(T)$. In this case, recall Section 7.2 gave an upper bound for the core Shout PIOP prover (Figure 5) of roughly the following number of field multiplications:

$$(d^2+1)T.$$

And Section 7.3 gave an upper bound for the one-hot-encoding-checking PIOP prover (Figure 8) of $O(K^{1/d}\log K) + 3dT$ when $K^{1/d}\log K = o(T)$. Summing the two, when $K = o(T)$ and $K^{1/d}\log K = o(T)$, the total prover work is at most

$$(d^2+3d+1)\cdot T.$$

7.5 One-hot-checking prover in small characteristic

We establish the prover-cost bounds of Proposition 5.5 for the sum-checks in Equations (22) and (23). We use the notation of Section 5.5: the table has T rows of length n, and $t = \log T$.

Proof of the prover-cost bounds in Proposition 5.5. For the runtime, let a_j be the position of the 1 in row j. Precompute $R[k] = \widetilde{\mathsf{eq}}(r, k)$ and $S[k] = \widetilde{\mathsf{eq}}(s, k)$ and their pointwise product in $O(n)$

time. The arrays of Boolean evaluations of $\widetilde{A}, \widetilde{B}, \widetilde{C}$ are then $R[a_j], S[a_j], R[a_j]S[a_j]$, respectively, and can be built in $O(T)$ time. Building the array of $\widetilde{\mathsf{eq}}(z, \cdot)$ evaluations and running the first, constant-degree sum-check take $O(T)$ time using the standard linear-time prover algorithm, which binds variables as described in Section 6.2. After u is known, build

$$U[k] = \widetilde{X}(k, u) = \sum_{j:a_j=k} \widetilde{\mathsf{eq}}(u, j)$$

in $O(T + n)$ time. The second sum-check binds variables in the three length-n arrays U, R, S and takes $O(n)$ time. The direct evaluations of $\widetilde{X}(r, u)$ and $\widetilde{X}(s, u)$ are already available from the first sum-check's arrays after all variables have been bound.

Multiplication count for Equations (22) and (23). Build R, S and their pointwise products with $O(n)$ multiplications. For $z_{>i} = (z_{i+1}, \ldots, z_t)$, build the arrays of $\widetilde{\mathsf{eq}}(z_{>i}, \cdot)$ evaluations for all rounds i with at most $T/2$ multiplications.

In round i of the first sum-check, let $N = T/2^i$. Apply Gruen's optimization (Section 6.5) to factor out the contribution to $\widetilde{\mathsf{eq}}$ from the current variable. Also factor out the known scalar contributed by the variables already bound, and write the remaining quadratic as $Q(c) = q_0 + q_1 c + q_2 c^2$. For each of the N pairs, its quadratic coefficient is the product of the differences of the two $\widetilde{A}$ entries and the two $\widetilde{B}$ entries, times the corresponding $\widetilde{\mathsf{eq}}(z_{>i}, \cdot)$ evaluation. Thus computing q_2 costs $2N$ multiplications. Computing either $Q(0)$ or $Q(1)$ also costs $2N$: one multiplication for $\widetilde{A}\widetilde{B}$, subtract $\widetilde{C}$, then multiply by the same $\widetilde{\mathsf{eq}}(z_{>i}, \cdot)$ evaluation.

We also use the usual sum-check saving of one evaluation per round (Section 6.2). For the factored polynomial Q, the current sum claim determines the weighted sum $(1 - z_i)Q(0) + z_i Q(1)$ after division by the known scalar above. Thus, only one of $Q(0)$ and $Q(1)$ needs to be computed directly. If $z_i \neq 0$, compute $Q(0)$; if $z_i = 0$, compute $Q(1)$ instead. If the known scalar is zero, the entire round message is zero and no division is needed. The evaluations at 0 and 1, together with q_2, determine Q, so message construction costs at most $4N$ multiplications, plus $O(1)$ scalar operations.

Binding the current variable in the arrays for $\widetilde{A}, \widetilde{B}, \widetilde{C}$ costs $3N$ multiplications. The first sum-check therefore costs at most $7T + O(\log T)$ multiplications, in addition to constructing the arrays of $\widetilde{\mathsf{eq}}$ evaluations. Finally, constructing the weights $\widetilde{\mathsf{eq}}(u, j)$ and accumulating $U[k]$ costs at most $T + O(n)$ multiplications; the second sum-check costs $O(n)$. This gives Equation (24), with $O(\log T)$ field inversions. This bound uses standard binding and does not rely on the optimizations for small memories. □

8 Fast Shout prover for large, structured memories

The sparse-dense sum-check protocol from (Generalized-)Lasso [STW24, Appendix G] is the key tool allowing the Shout prover to efficiently prove T lookups into (structured) memories of size vastly larger than T. For tables admitting a suitable prefix-suffix decomposition, subsequent work [NTZ25, Appendix A] gives prefix-suffix sum-check proving as an alternative prover algorithm for the same sum-check protocol.

8.1 Sparse-dense sum-check protocol

8.1.1 Informal Overview

Consider two very large vectors $a, b \in \mathbb{F}^N$ and suppose we want to compute their inner product

$$\langle a, b \rangle = \sum_{i=1}^{N} a_i \cdot b_i. \tag{42}$$

Say we want to outsource this work to an untrusted prover using the sum-check protocol. A standard application would apply the sum-check protocol to the polynomial $\widetilde{a} \cdot \widetilde{b}$ where $\widetilde{a}$ and $\widetilde{b}$ are the multilinear extensions of a and b respectively. Note that $\widetilde{a} \cdot \widetilde{b}$ is a polynomial in $\log N$ variables with degree two in each variable. The standard linear-time prover implementation of the sum-check protocol in this context (Section 6.2) would require about $4N$ field operations. However, the prover can be implemented far faster if a and b are structured in certain ways. [STW24, Appendix G] refers to such a fast prover implementation as the *sparse-dense sum-check protocol*.[8]

Suppose we use two (not necessarily multilinear) extensions $\hat{a}$ and $\hat{b}$ of a and b. At the end of round 1, when the first variable gets bound to $r_1 \in \mathbb{F}$, both $\hat{a}$ and $\hat{b}$ become $(\log(N) - 1)$-variate polynomials $\hat{a}(r_1, x_2, \ldots, x_{\log N})$ and $\hat{b}(r_1, x_2, \ldots, x_{\log N})$. The sparse-dense sum-check protocol asks: how do these "new" polynomials relate to the "old" polynomials $\hat{a}(x_1, \ldots, x_{\log N})$ and $\hat{b}(x_1, \ldots, x_{\log N})$? In particular, is the relationship between $\hat{a}(x_1, \ldots, x_{\log N})$ and $\hat{a}(r_1, x_2, \ldots, x_{\log N})$ largely independent of $x_2, \ldots, x_{\log N}$, and similarly for $\hat{b}(x_1, \ldots, x_{\log N})$ and $\hat{b}(r_1, x_2, \ldots, x_{\log N})$? If so, then major optimizations are possible.

Let r_i denote the random field element chosen by the sum-check verifier in round i. Suppose that we can chunk the entries of a and b into two groups, $H(0)$ and $H(1)$, such that when variable one gets bound to r_1, each evaluation $\hat{a}(i)$ for $i \in H(0)$ gets multiplied by the same value (say m_{0,r_1}) and similarly each term a_i of a in $H(1)$ gets multiplied by m_{1,r_1}. Suppose further that each term b_i of b in $H(0)$ (respectively, $H(1)$) gets multiplied by the same value, say v_{0,r_1} and v_{1,r_1}.

Illustrative example. A key example to have in mind is

$$\hat{b}(x) = \widetilde{\mathsf{eq}}(r', x),$$

for some random vector $r' \in \mathbb{F}^{\log N}$. Then for any $x = (k, y)$ with $k \in \{0, 1\}$ and $y \in \{0, 1\}^{\log(N)-1}$,

$$\hat{b}(r_1, y)/\hat{b}(k, y) = \widetilde{\mathsf{eq}}(r'_1, r_1)/\widetilde{\mathsf{eq}}(r'_1, k).$$

In other words, changing the first coordinate fed to $\hat{b}$ from k to r_1 results in a multiplicative update that depends only on k, r_1, and r'_1.

Hence, letting H_0 and H_1 denote points in $\{0, 1\}^{\log(N)}$ of the form $(0, y)$ and $(1, y)$ respectively, $\hat{b}$ satisfies the informal condition above if we define

$$v_{0,r_1} = \widetilde{\mathsf{eq}}(r'_1, r_1)/(1 - r'_1) \tag{43}$$

[8] To clarify, the term "sparse-dense sum-check protocol" used in [STW24, Appendix G] refers to a fast *prover implementation*. The protocol itself (i.e., the checks the verifier does on the prover's messages) is no different than in the standard sum-check protocol.

and

$$v_{1,r_1} = \widetilde{\mathsf{eq}}(r'_1, r_1)/r'_1. \tag{44}$$

Meanwhile, if $\hat{a}$ is multilinear (i.e., $\hat{a} = \widetilde{a}$) then by Fact 4.1,

$$\hat{a}(r_1, y) = (1 - r_1)\hat{a}(0, y) + r_1\hat{a}(1, y). \tag{45}$$

So in this example, we have the following expression for the prescribed prover's message in round two of sum-check, where r_1 is the random challenge in round 1:

$$\begin{aligned}
s_1(r_1) &= \sum_{y \in \{0,1\}^{\log(N)-1}} \widetilde{a}(r_1, y) \cdot \hat{b}(r_1, y) \\
&= \left(\sum_{(0,y)\colon y \in \{0,1\}^{\log(N)-1}} (1 - r_1)\hat{a}(0, y) \cdot \hat{b}(r_1, y) \right) + \left(\sum_{(1,y)\colon y \in \{0,1\}^{\log(N)-1}} r_1\hat{a}(1, y) \cdot \hat{b}(r_1, y) \right) \\
&= \left(\sum_{i \in H(0)} a_i \cdot b_i \cdot (1 - r_1) \cdot v_{0,r_1} \right) + \left(\sum_{i \in H(1)} a_i \cdot b_i \cdot r_1 \cdot v_{1,r_1} \right),
\end{aligned}$$

where v_{0,r_1} and v_{1,r_1} are defined in Equations (43) and (44).

This means that in moving from the sum defining the prover's claim at the start of round 1 (Expression (42)) to the sum defining the prover's claim at the start of round 2, every term $a_i b_i$ for an $i \in H(0)$ gets multiplied by the same factor, namely $(1 - r_1) \cdot v_{0,r_1}$, and similarly for every term for an $i \in H(1)$. Rather than performing this multiplication for each term individually, it is much faster to first *aggregate* (i.e., sum up) all the terms $a_i \cdot b_i$ in $H(0)$ and $H(1)$ respectively, and just multiply each aggregated value by the relevant factor. This costs only two multiplications *in total across all terms*, rather than one for each of the N terms.

This is the key idea in the sparse-dense sum-check protocol from [STW24, Appendix G]. In round i, there are 2^i relevant groups (one group for each $k \in \{0,1\}^i$, with the k'th group capturing all terms whose binary representations begin with k). At the end of round i, when variable i gets bound to $r_i \in \mathbb{F}$, each term within a group gets multiplied by the same value.

If N is considered too large for linear-in-N prover time to be acceptable, say, because only $T \ll N$ terms $a_i \cdot b_i$ are non-zero, then the sparse-dense sum-check protocol is capable of running in time $O(CT)$ where C is any integer such that $N \leq T^C$.

The sparse-dense sum-check protocol also applies when binding the j'th variable to r_j causes each $\hat{b}$'s value to change by an additive rather than multiplicative factor that depends only on r_j. This is explained in detail in [STW24, Appendix G]. The canonical example to have in mind here is

$$\hat{b}(x) = \sum_{i=1}^{\log N} 2^{i-1} x_i,$$

which arises when applying Generalized-Lasso or Shout to a "range-check" table storing all field elements in $\{0, 1, \ldots, N-1\}$. In this case, changing x_i from c to r_i increases $\hat{b}(x)$ by an additive factor of $2^{i-1}(r_i - c)$.

8.2 Extension to the Booleanity-checking sum-check when $K^{1/d} \gg T$

Overview. Recall that in the Booleanity-checking sum-check (Line 3 of Figure 8), the sum-check protocol is applied to compute

$$\sum_{k \in \{0,1\}^{\log(K)/d},\, j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(r',j)\left(\widetilde{\mathsf{ra}}_i(k,j)^2 - \widetilde{\mathsf{ra}}_i(k,j)\right).$$

Let us first focus on the "sub-sum"

$$\sum_{k \in \{0,1\}^{\log(K)/d},\, j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(r',j)\widetilde{\mathsf{ra}}_i(k,j)^2,$$

as this is the heart of the protocol.

Let

$$\hat{a}(k,j) = \widetilde{\mathsf{ra}}_i(k,j)^2$$

and

$$\hat{b}(k,j) = \widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(r',j).$$

The treatment of the sparse-dense sum-check protocol in [STW24, Appendix G] assumes $\hat{a}$ is multilinear, i.e., $\hat{a} = \widetilde{a}$. This is not the case here, as $\hat{a}$ is the *square* of a multilinear polynomial.

Fortunately, $\widetilde{\mathsf{ra}}_i(k,j)$ satisfies an "ultrastructured" sparseness property: for each $j \in \{0,1\}^{\log T}$, there is at most (in fact, exactly) one $k \in \{0,1\}^{\log(K)/d}$ such that $\widetilde{\mathsf{ra}}_i(k,j) \neq 0$. Exploiting this property ensures that the sparse-dense sum-check protocol still applies to the Booleanity-checking sum-check. At a high level, what's going on is the following.

The treatment in [STW24, Appendix G] exploited that when $\hat{a}(k,j)$ is multilinear, Equation (45) holds, which roughly states that binding the first variable of $\hat{a}$ to r_1 causes the resulting evaluations of $\hat{a}(r_1,y)$ to be a linear combination of $\hat{a}(0,y)$ and $\hat{a}(1,y)$. When $\hat{a}(k,j) = \widetilde{\mathsf{ra}}_i(k,j)^2$, Equation (45) does not hold, but something just as nice does: if k is the unique value in $\{0,1\}^{\log(K)/d}$ with $\widetilde{\mathsf{ra}}_i(k,j) \neq 0$, then assuming wlog that the first entry of k is 0, and letting y denote the remaining $\log(K)/d + \log(T) - 1$ entries of (k,j), it holds that:

$$\widetilde{\mathsf{ra}}_i^2(r_1,y) = \left((1-r_1)\widetilde{\mathsf{ra}}_i(0,y) + r_1\widetilde{\mathsf{ra}}_i(1,y)\right)^2 = (1-r_1)^2\widetilde{\mathsf{ra}}_i(0,y)^2 = (1-r_1)^2\hat{a}(0,y).$$

In other words, binding variable one to r_1 causes $\hat{a}(k,j)$ to get multiplied by a factor $(1-r_1)^2$ that only depends on the first coordinate of k and on r_1.

Algorithm details. As in the overview above, let

$$\hat{a}(k,j) = \widetilde{\mathsf{ra}}_i(k,j)^2$$

and

$$\hat{b}(k,j) = \widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(r',j)$$

and suppose we apply the sum-check protocol to compute

$$\sum_{k \in \{0,1\}^{\log(K)/d},\, j \in \{0,1\}^{\log T}} \hat{a}(k,j) \cdot \hat{b}(k,j). \tag{46}$$

Let $c \geq 1$ be a positive integer such that $cK^{1/(cd)} = o(T)$. For simplicity, assume $\log(K)/d$ is divisible by c. We will break each $k \in \{0,1\}^{\log(K)/d}$ into c chunks each of size $\log(K)/(cd)$, writing

$$k = (k_1, \ldots, k_c) \in \left(\{0,1\}^{\log(K)/(dc)}\right)^c.$$

Similarly, write

$$r = (r_1, \ldots, r_c) \in \left(\mathbb{F}^{\log(K)/(dc)}\right)^c.$$

Evaluating all $\hat{b}(k,j)$ values. The first thing the prover needs to do is compute $\hat{b}(k,j)$ for all $(k,j) \in \{0,1\}^{\log(K)/d} \times \{0,1\}^{\log T}$ such that $\hat{a}(k,j) \neq 0$. In Booleanity-checking where $\hat{b}(k,j) = \widetilde{\mathsf{eq}}(k,r) \cdot \widetilde{\mathsf{eq}}(j,r')$, this can be done with

$$(c+1) \cdot K^{1/(cd)} + T$$

field multiplications as follows. First, the prover computes $c+1$ tables, each of size $K^{1/(dc)}$, storing for each $\ell \in \{1, \ldots, c\}$, the values $\widetilde{\mathsf{eq}}(r_\ell, x)$ as x ranges over $\{0,1\}^{\log(K)/(cd)}$, as well as a table storing $\widetilde{\mathsf{eq}}(r', x)$ as x ranges over the same domain $\{0,1\}^{\log T}$. This requires $c \cdot K^{1/(dc)} + T$ multiplications in total (by Lemma 4.1). Given these tables, for any desired $(k,j) \in \{0,1\}^{\log K} \times \{0,1\}^{\log T}$, $\hat{b}(k,j)$ can be evaluated with one lookup into each of the $c+1$ tables followed by c multiplications.

Building a partial sum tree. Before the start of the protocol, the prover builds a "tree of partial sums". Specifically, for each $k_1 \in \{0,1\}^{\log(K)/(cd)}$, let

$$L(k_1) = \sum_{k=(k_1,k') \in \{0,1\}^{\log(K)/(cd)} \times \{0,1\}^{\log(K)/d - \log(K)/(cd)},\ j \in \{0,1\}^{\log T}} \hat{a}(k,j) \cdot \hat{b}(k,j).$$

The prover builds a binary tree over these leaf values, with each interior node storing the sum of its children. Note that the tree has $2^{\log(K)/(cd)} = K^{1/(cd)} = o(T)$ leaves. This step requires $(c+1)T$ field multiplications in total, because there are T non-zero entries of $\hat{a}(k,j)$ and for each such entry, $\hat{b}(k,j)$ can be computed with c multiplications and then multiplied by $\hat{a}(k,j)$ with a single additional multiplication.

Let us index the internal nodes at distance i from the root by $\{0,1\}^i$ (we will label the root by $\emptyset$). This ensures that the root of the tree, $L(\emptyset)$ computes the entire sum in Expression (46).

Round one message. The round-1 message of the prover can computed via a constant number of field operations (and inversions) given the values stored at the two children of the root. Indeed,

$$s_1(0) = \sum_{k=(0,k') \in \{0,1\} \times \{0,1\}^{\log(K)/d - 1},\ j \in \{0,1\}^{\log T}} \hat{a}(k,j) \cdot \hat{b}(k,j),$$

and this quantity is equal to $L(0)$ (the left child of the root). Similarly, $s_1(2)$ is a constant-time-computable linear combination of $L(0)$ and $L(1)$:

$$s_1(2) = \left(\widetilde{\mathsf{eq}}(2,0)^2 \cdot \widetilde{\mathsf{eq}}(2,r_1) \cdot (1-r_1)^{-1}\right) \cdot L(0) + \left(\widetilde{\mathsf{eq}}(2,1)^2 \widetilde{\mathsf{eq}}(2,r_1) \cdot r_1^{-1}\right) \cdot L(1).$$

This equality holds by the following reasoning. For each $k = (0, k')$ and each $j \in \{0,1\}^{\log T}$ satisfying $\widetilde{\mathsf{ra}}_i(k, j) \neq 0$, it holds that $\widetilde{\mathsf{ra}}_i(k, j) = 1$ and $\widetilde{\mathsf{ra}}_i(2, k', j) = \widetilde{\mathsf{eq}}(2, 0)^2$. Meanwhile,

$$\hat{b}(2, k', j) = \widetilde{\mathsf{eq}}((2, k'), r) \cdot \widetilde{\mathsf{eq}}(r', j) = \widetilde{\mathsf{eq}}(2, r_1) \cdot (1 - r_1)^{-1} \cdot \hat{b}(k, j).$$

Similarly, for each $k = (1, k')$ and each $j \in \{0,1\}^{\log T}$ satisfying $\widetilde{\mathsf{ra}}_i(k, j) \neq 0$, it holds that $\widetilde{\mathsf{ra}}_i(k, j) = 1$ and $\widetilde{\mathsf{ra}}_i(2, k', j) = \widetilde{\mathsf{eq}}(2, 1)^2$, and

$$\hat{b}(2, k', j) = \widetilde{\mathsf{eq}}(2, k', r) \cdot \widetilde{\mathsf{eq}}(r', j) = \widetilde{\mathsf{eq}}(2, r_1) \cdot r_1^{-1} \cdot \hat{b}(k, j).$$

(Even though each round's sum-check prover message s_i is a degree-3 univariate polynomial, Gruen [Gru24, Section 3] shows how a slight modification of the protocol allows the prover to only send $s_i(0)$ and $s_i(2)$ in each round, see Section 6.5).

Round two message. At the end of round 1, the verifier sends the random value r_1. Similar to round 1, the prover's round-2 message is a quickly-computable linear combination of the four values stored at level two of the tree (grandchildren of the root). Indeed,

$$s_2(0) = \sum_{k' \in \{0,1\}^{\log(K)/d-2},\ j \in \{0,1\}^{\log T}} \hat{a}(\mathsf{r}_1, 0, k', j) \cdot \hat{b}(\mathsf{r}_1, 0, k', j).$$

Recall that for each $j \in \{0,1\}^{\log T}$, the prover can quickly ascertain the unique $k'' \in \{0,1\}^{\log(T)-1}$ for which $\hat{a}(\mathsf{r}_1, k'', j) \neq 0$, and quickly compute the factor $m_{k_1'}$ (which depends only on r_1 and k_1') such that

$$\hat{a}(\mathsf{r}_1, k'', j) = m \cdot \hat{a}(k', j),$$

where $k' \in \{0,1\}^{\log K}$ is the unique vector such that $\hat{a}(k', j) \neq 0$. Similarly, we have assumed that the prover can quickly compute the factors v_0, v_1 (which depend on only r_1) such that $\hat{b}(\mathsf{r}_1, 0, k', j) = v_0 \cdot \hat{b}(0, 0, k', j)$ and $\hat{b}(\mathsf{r}_1, 0, k', j) = v_1 \cdot \hat{b}(0, 1, k', j)$. Hence,

$$s_2(0) = m_0 v_0 L(0, 0) + m_0 v_1 L(0, 1).$$

Similarly, $s_2(2)$ is a quickly-computable linear combination of the four nodes at distance two from the root: $L(0, 0)$, $L(0, 1)$, $L(1, 0)$, $L(1, 1)$:

$$s_2(2) = c_{0,0} \cdot L(0, 0) + c_{0,1} L(0, 1) + c_{1,0} L(1, 0) + c_{1,1} L(1, 1),$$

where for each $k_1, k_2 \in \{0, 1\}$,

$$c_{k_1,k_2} = m_{k_1,k_2} v_{k_1,k_2},$$

where

$$m_{k_1,k_2} = \widetilde{\mathsf{eq}}(\mathsf{r}_1, k_1)^2 \cdot \widetilde{\mathsf{eq}}(2, k_2)^2$$

and

$$v_{k_1,k_2} = \widetilde{\mathsf{eq}}(k_1, r_1)^{-1} \cdot \widetilde{\mathsf{eq}}(\mathsf{r}_1, r_1) \cdot \widetilde{\mathsf{eq}}(2, r_2) \cdot \widetilde{\mathsf{eq}}(k_2, r_2)^{-1}.$$

Round $i \leq \log(K)/(cd)$. In general, $s_i(0)$ and $s_i(2)$ are each quickly-computable linear combinations of the nodes at distance i from the root. For example, when $\hat{a} = \widetilde{\mathsf{ra}}_i^2$ and $\hat{b}(k,j) = \widetilde{\mathsf{eq}}(r,k) \cdot \widetilde{\mathsf{eq}}(r',j)$, then:

$$s_2(2) = \sum_{k \in \{0,1\}^i} c_k \cdot L(k),$$

where $c_k = m_k \cdot v_k$, defined as follows:

$$m_k = \widetilde{\mathsf{eq}}(\mathsf{r}_1, \ldots, \mathsf{r}_{i-1}, 2, k)^2$$

and

$$v_k = \widetilde{\mathsf{eq}}(k, (r_1, \ldots, r_i))^{-1} \cdot \widetilde{\mathsf{eq}}(\mathsf{r}_1, \ldots, \mathsf{r}_i, r_1, \ldots, r_i).$$

Each round $i \leq \log(K)/(cd)$ can be implemented in roughly $O(2^i)$ field operations.[9]

Tree-recomputation procedure. The prover performs a tree-recomputation procedure at the end of round $\log(K)/(cd)$. Specifically, the prover computes, for every $j \in \{0,1\}^{\log T}$, the unique $k = (k_2, \ldots, k_c) \in \{0,1\}^{(c-1)\cdot\log(K)/(cd)}$ such that $\hat{a}(\mathsf{r}_1, \ldots, \mathsf{r}_{\log(K)/c}, k, j) \neq 0$, and moreover for this k, the prover can compute both $\hat{a}(\mathsf{r}_1, \ldots, \mathsf{r}_{\log(K)/c}, k, j)$ and $\hat{b}(\mathsf{r}_1, \ldots, \mathsf{r}_{\log(K/c)}, k, j)$.

Indeed, by Lemma 4.1, with $K^{1/(cd)}$ multiplications the prover can compute $\widetilde{\mathsf{eq}}(\mathsf{r}_1, \ldots, \mathsf{r}_{\log(K)/(cd)}, k')$ for all $k' \in \{0,1\}^{\log(K)/(cd)}$. Since $\hat{a} = \widetilde{\mathsf{ra}}_i^2$, each quantity $\hat{a}(\mathsf{r}_1, \ldots, \mathsf{r}_{\log(K)/(cd)}, k, j)$ can be identified with a single lookup into this table followed by a squaring operation. Similarly, for each non-zero $\hat{a}(\mathsf{r}_1, \ldots, \mathsf{r}_{\log(K)/(cd)}, k, j)$, the value $\hat{b}(\mathsf{r}_1, \ldots, \mathsf{r}_{\log(K)/(cd)}, k, j)$ can also be computed with one multiplication given partial products computed and stored during the course of the procedure computing $\hat{b}(k,j)$ for all relevant values of (k,j) while building the partial sum tree prior to the start of round 1. This is $2T + K^{1/(cd)}$ field multiplications in total.

Given these quantities, the prover can construct, with T more multiplications, a new partial-sum tree with T leaves, such that leaf k' stores

$$\sum_{k=(k_1,k',k_3,\ldots,k_c) \in \{0,1\}^{\log(K)/d},\ j \in \{0,1\}^{\log T}} \hat{a}(\mathsf{r}_1, \ldots, \mathsf{r}_{\log(K)/c}, k', k_2, \ldots, k_c) \cdot \hat{b}(\mathsf{r}_1, \ldots, \mathsf{r}_{\log(K)/(cd)}, k', k_2, \ldots, k_c).$$

The next $\log(K)/(cd)$ rounds proceed analogously to the first $\log(K)/(cd)$ rounds, followed by another tree-recomputation step, and so on until the first $\log(K)/d$ rounds are complete.

The final $\log T$ rounds can then be implemented exactly as in the case of small memory sizes (Section 7.3). The total cost for these final $\log T$ rounds is $5dT$ field operations (Section 7.3 implemented those rounds with only $3dT$ multiplications, but $2dT$ multiplications were saved there by exploiting that $K^{1/d} = o(T)$, which does not hold here if $c > 1$).

[9] More precisely, by using a batch-inversion procedure, it's possible to implement round i in with $O(2^i)$ field multiplications and one inversion.

The full Booleanity-checking sum-check. Recall that the Booleanity-checking sum-check actually applies the sum-check protocol to compute

$$\sum_{k \in \{0,1\}^{\log(K)/d},\ j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(r',j)\left(\widetilde{\mathsf{ra}}_i(k,j)^2 - \widetilde{\mathsf{ra}}_i(k,j)\right).$$

We have explained above how to apply the sum-check protocol to compute

$$\sum_{k \in \{0,1\}^{\log(K)/d},\ j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(r',j)\widetilde{\mathsf{ra}}_i(k,j)^2. \tag{47}$$

It suffices to now explain how to apply the sum-check prover to compute

$$\sum_{k \in \{0,1\}^{\log(K)/d},\ j \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r,k)\widetilde{\mathsf{eq}}(r',j)\widetilde{\mathsf{ra}}_i(k,j), \tag{48}$$

since in the Booleanity-checking sum-check the prover's message in each round is just the difference between the corresponding message for Expressions (47) and (48).

Walking through the algorithm described above for proving Expression (47) and modifying it appropriately for Expression (48), it is easy to see that the sum-check prover for Expression (48) only needs to do $O(cK^{1/(cd)})$ additional field multiplications on top of what the prover for Expression (48) does. In other words, "handling" Expression (48) in addition to Expression (47) does not substantially change the prover cost.

Cost summary for Booleanity-checking when $K^{1/d} \gg T$. In total, across all $i = 1, \ldots, d$, the Booleanity-checking prover performs the following field multiplications:

- $c \cdot K^{1/(cd)} + T$ multiplications in pre-computation to evaluate $\widetilde{\mathsf{eq}}$ polynomials at appropriate points.
- $d(c+1)T$ more multiplications to build the d partial-sum trees prior to round one.
- $O(dK^{1/(cd)})$ to compute the first $\log(K)/(dc)$ rounds worth of messages given these trees.
- $3dT + O(dK^{1/(cd)})$ field multiplications every time the partial-sum tree is rebuilt and the next $\log(K)/(cd)$ rounds of messages are computed.
- $5dT$ field multiplications in the final $\log(T)$ rounds.

Since the partial sum tree for each $i = 1, \ldots, d$ is rebuilt $c-1$ times, this means the prover performs

$$O(dcK^{1/(dc)}) + (d(c+1) + 3d(c-1) + 5d + 1)T = O(dcK^{1/(dc)}) + (4dc + 3d + 1)\,T$$

field multiplications in total.

For example, if $c = 2$ and $d = 2$, this is $O(K^{1/4}) + 23T$ field multiplications. This is an appropriate setting of parameters when $K = 2^{64}$ as arises in Jolt, as in this case $K^{1/4} = 2^{16}$, which is substantially less than typical values of T.

8.3 Shout's read-checking sum-check

Recall that the read-checking sum-check within Shout (Line 3 in Figure 7) applies the sum-check protocol to confirm that $\widetilde{\mathsf{rv}}(r_{\text{cycle}})$ equals:

$$\sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathsf{ra}}(k, r_{\text{cycle}}) \cdot \widetilde{\mathsf{Val}}(k) \tag{49}$$

[STW24, Appendix G] showed that the $\widetilde{\mathsf{Val}}$ polynomials corresponding to the lookup tables arising in the Jolt RISC-V zkVM satisfy the conditions necessary to apply the sparse-dense sum-check protocol to compute

$$\sum_{k \in \{0,1\}^{\log K}} \hat{a}(k) \cdot \widetilde{\mathsf{Val}}(k) \tag{50}$$

for any sparse, multilinear polynomial $\hat{a}$. It is easily seen that for any $r_{\text{cycle}} \in \mathbb{F}^{\log T}$, $\widetilde{\mathsf{ra}}(k, r_{\text{cycle}})$ has at most T non-zero evaluations over $k \in \{0,1\}^{\log K}$, since $\widetilde{\mathsf{ra}}(k, j)$ is a one-hot vector for each fixed $j \in \{0,1\}^{\log T}$. Hence, the results of [STW24, Appendix G] apply directly.

If C is an integer such that $K^{1/C} = o(T)$, then the resulting total prover cost for the read-checking sum-check is, up to low-order terms, $2CT$ field multiplications.

The $\widetilde{\mathsf{ra}}$-virtualization sum-check for $d > 1$ proceed identically to the case of small memories (Section 7.2), requiring d^2T multiplications up to low-order terms.

8.4 The $\widetilde{\mathsf{raf}}$-evaluation sum-check and Hamming-weight-one check when $K \gg T$

As pointed out in Section 7.3, the $\widetilde{\mathsf{raf}}$-evaluation sum-check of Figure 8 is identical to the read-checking sum-check in the core Shout PIOP, just with $\widetilde{\mathsf{Val}}(k)$ replaced with

$$\sum_{i=1}^{d} \sum_{\ell=0}^{\log(K)/d-1} 2^{(i-1)\cdot \log(K)/d + \ell} \cdot k_{i,\ell},$$

which is the MLE of the function that maps k to $\text{int}(k)$. In other words, the $\widetilde{\mathsf{raf}}$-evaluation sum-check is equivalent to checking reads into the table whose k'th entry is $\int(k)$. As further observed in Section 7.3, in applications the prover can set r_{cycle} in the core Shout PIOP equal to r'_{cycle} (the random point at which the verifier wishes to evaluate $\widetilde{\mathsf{raf}}$ via the $\widetilde{\mathsf{raf}}$-evaluation sum-check). Then running the $\widetilde{\mathsf{raf}}$-evaluation sum-check and the read-checking sum-check in a batched manner per Section 6.2.1 is equivalent to replacing the lookup table $\widetilde{\mathsf{Val}}(k)$ with $\widetilde{\mathsf{Val}}(k) + z \cdot \widetilde{\text{int}}(k)$. The first $\log K$ rounds of the batched sum-check can be implemented by separately applying the read-checking sum-check to the two tables with MLEs given by $\widetilde{\mathsf{Val}}$ and $\widetilde{\text{int}}$, with the latter costing $CT + o(T)$ multiplications for the prover. The final $\log T$ rounds of the batched protocol are no more expensive than the read-checking sum-check alone, as the value $\widetilde{\mathsf{Val}}(r''')$ in the read-checking sum-check (where r''' is the verifier's random challenges chosen over the course of the first $\log K$ rounds) is simply replaced in the batched sum-check with $\widetilde{\mathsf{Val}}(r''') + z \cdot \widetilde{\text{int}}(r''')$.

Hamming weight one check. Checking that each committed address has Hamming weight one (Line 4 of Figure 8) can be directly implemented with $O(dK^{1/d})$ field multiplications after the prover computes an array storing $\widetilde{\mathsf{eq}}(j, r''_{\text{cycle}})$ for all $j \in \{0,1\}^{\log T}$ (which, for appropriate choice of r''_{cycle}, will have already been computed elsewhere within Shout). If $c > 1$ so $O(dK^{1/d})$ time is not acceptable, the original sparse-dense sum-check protocol applies with $\hat{a}(k) = \widetilde{\mathsf{ra}}_i(k, r''_{\text{cycle}})$ and $\hat{b} = 1$ and achieves cost $cT + O(CK^{1/C})$ field multiplications, where $C = cd$.

8.5 Cost summary for Shout when $K \gg T$

In summary, for gigantic structured memories where $K^{1/C} = o(T)$, the Shout prover incurs the following costs up to low-order terms:

- $2T$ multiplications for evaluating $\widetilde{\mathsf{rv}}(r_{\text{cycle}})$ before applying the read-checking sum-check.
- $\left(2C + d^2\right) T$ multiplications for the read-checking sum-check.
- $(4C + 3d)\, T$ for Booleanity checking.[10]
- CT for the $\widetilde{\mathsf{raf}}$-evaluation sum-check.
- cT for the Hamming-weight-one-checking sum-check if $c > 1$.

In total, this is at most $\left(7C + d^2 + 3d + c + 2\right) T$ multiplications.

8.6 A Shout variation with a linear prover dependence on d

A variation of Equation (15) yields (a variation of) Shout that simply applies the sum-check protocol to confirm that:

$$\widetilde{\mathsf{rv}}(r_{\text{cycle}}) = \sum_{\substack{k=(k_1,\ldots,k_d)\in\left(\{0,1\}^{\log(K)/d}\right)^d \\ j\in\{0,1\}^{\log T}}} \widetilde{\mathsf{eq}}(r_{\text{cycle}}, j) \left(\prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(k_i, j)\right) \cdot \widetilde{\mathsf{Val}}(k).$$

For very large values of d, a nuisance is that the final $\log T$ rounds of this protocol cause the prover to incur roughly d^2T field multiplications. Here, we describe an alternative application of the sum-check protocol that avoids quadratic dependence on d (but has a worse leading constant, and hence is only preferable when d is very large).

Let $j_0, j_1, \ldots, j_d$ each consist of $\log T$ variables. Let $\widetilde{\mathsf{eq}}(j_0, j_1, \ldots, j_d)$ denote the multilinear extension of the function that evaluates to 1 if all $d+1$ vectors $j_0, \ldots, j_d$ are equal, and zero otherwise. Then

$$\widetilde{\mathsf{eq}}(j_0, j_1, \ldots, j_d) = \prod_{i=1}^{\log T} \left(\left(\prod_{k=0}^{d}(1 - j_{k,i})\right) + \left(\prod_{k=0}^{d} j_{k,i}\right)\right). \tag{51}$$

Now, in our modified version of Shout, replace Equation (15) with:

[10] Here, we avoid T multiplications by setting $r' = r_{\text{cycle}}$ within the Booleanity-checking sum-check, and thereby reusing a table of $\widetilde{\mathsf{eq}}$ evaluations between the first bullet and this bullet, same as in Section 7.3.

$$\widetilde{\mathsf{rv}}(r_{\text{cycle}}) = \sum_{\substack{k=(k_1,\ldots,k_d)\in\left(\{0,1\}^{\log(K)/d}\right)^d \\ j_1,\ldots,j_d\in\{0,1\}^{\log T}}} \widetilde{\mathsf{eq}}(r_{\text{cycle}}, j_1, j_2, \ldots, j_d) \cdot \left(\prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(k_i, j_i)\right) \cdot \widetilde{\mathsf{Val}}(k). \tag{52}$$

Note that this invocation of the sum-check protocol has $\log(K) + d\log T$ rounds rather than $\log(K)+\log(T)$ rounds in the standard variant of Shout (i.e., applying sum-check to check Equation (15)). However, the total proof size remains $O(\log(K) + d\log(T))$ field elements, as each of the final $d\log T$ rounds in the new variant has the prover send a degree-2 polynomial (whereas the unmodified standard variant sends a degree-$(d+1)$ polynomial in each of the final $\log T$ rounds).

Theorem 8.1. *Shout (Figure 7) with Equation* (15) *replaced by Equation* (52) *is perfectly complete and has soundness error* $O((\log(K) + d\log(T))/|\mathbb{F}|)$.

Proof. The proof is identical to Theorem 5.4, except that we must establish that Equation (52) holds. To see this, observe that the right hand side of Equation (52) is a multilinear polynomial in r_{cycle}. So it suffices to establish that the right hand side and left hand side agree at all inputs $r_{\text{cycle}} \in \{0,1\}^{\log T}$. Observe that for any term of the sum on the right hand side, if $j_1, \ldots, j_d \in \{0,1\}^{\log T}$, then $\widetilde{\mathsf{eq}}(r_{\text{cycle}}, j_1, j_2, \ldots, j_d) = 0$ unless

$$j_1 = j_2 = \cdots = j_d = r_{\text{cycle}}, \tag{53}$$

and if this equality does hold then $\widetilde{\mathsf{eq}}(r_{\text{cycle}}, j_1, j_2, \ldots, j_d) = 1$.

If Equation (53) does hold, then $\left(\prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(k_i, j_i)\right) = 1$ if the register with binary representation $k = (k_1, \ldots, k_d)$ is read at cycle r_{cycle}, and $\left(\prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(k_i, j_i)\right) = 0$ otherwise. Hence, the right hand side equals $\widetilde{\mathsf{Val}}(k)$ where k is the register read at cycle r_{cycle}. The result follows. □

8.7 Fast prover implementation

First $\log K$ **rounds.** The first $\log K$ rounds of this sum-check invocation proceed identically to the standard variant of Shout (in these rounds, the prover message is no different between the two variants). The reader may be initially surprised by this because Expression (52) has $K \cdot T^d$ terms while Expression (15) only has $K \cdot T$ terms. However, per the proof of Theorem 8.1, the factor $\widetilde{\mathsf{eq}}(r_{\text{cycle}}, j_1, j_2, \ldots, j_d)$ evaluates to 0 over $\left(\{0,1\}^{\log T}\right)^d$ unless $j_1 = j_2 = \cdots = j_d$. This means that in the first $\log K$ rounds (where only variables of k are being bound, not variables of $j_1, \ldots, j_d$), $j_1, \ldots, j_d$ effectively act as a single entity $j \in \{0,1\}^{\log T}$, and this leads to the same prover messages as in the original Shout protocol for these rounds.

Final $d\log T$ **rounds.** Let $r = (r_1, \ldots, r_d) \in \left(\mathbb{F}^{\log(K)/d}\right)^d$ denote the randomness chosen by the verifier across the first $\log K$ rounds of sum-check. Then the final $d\log T$ rounds compute the following sum:

$$\widetilde{\mathsf{Val}}(r) \cdot \sum_{j_1,\ldots,j_d \in \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r_{\text{cycle}}, j_1, j_2, \ldots, j_d) \left(\prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(r_i, j_i) \right). \tag{54}$$

We bind the first variable of $j_1, j_2, \ldots, j_d$ in sequence, followed by the second variable of each, and so on. Let us call the first d rounds "Stage 1" (i.e., when we bind the first variable of each of $j_1, j_2, \ldots, j_d$), the next d rounds Stage 2, and so forth.

The key insight for a fast prover. While there are T^d terms in Expression (54), round-over-round almost all of them are zero. Specifically, $\widetilde{\mathsf{eq}}(r_{\text{cycle}}, j_1, j_2, \ldots, j_d)$ is a product of $\log(T)$ terms, one per variable in each j_i–intuitively, it tests equality of $j_1, \ldots, j_d$ "bitwise". Since $\widetilde{\mathsf{eq}}$ checks equality of each bit independently, it turns out that in each round of the protocol, the prover need only consider terms that agree in all remaining bits after the bit currently being bound. Sharing prefix and suffix products across the rounds gives $O(dT/2^S)$ work in Stage S. Summing over stages gives $O(\sum_{S=1}^{\log T} dT/2^S) = O(dT)$ time in total.

8.7.1 Detailed prover algorithm description for $d = 2$

To simplify notation, we begin with a description of the prover algorithm in the case $d = 2$. Further, let $r' = r_{\text{cycle}}$.

Consider the first round within Stage i. This is round $t = 2(i-1)+1$ within the final $d \log T$ rounds of the sum-check applied to compute Expression (52), and round $t' = \log(K) + t$ within the entire modified-Shout protocol.

Let $w_1 = (w_{1,1}, \ldots, w_{1,i-1}) \in \mathbb{F}^{i-1}$ denote the vector of random values chosen for the first $i-1$ coordinates of j_1 across the already-completed $i-1$ stages, and similarly let $w_2 = (w_{2,1}, \ldots, w_{2,i-1})$.

Simplifying the prover's message definition. The prover's prescribed message $s_{t'}$ in round t' of our modified version of Shout satisfies:

$$\begin{aligned} s_{t'}(c) = \widetilde{\mathsf{Val}}(r) \cdot \sum_{\substack{j_1' \in \{0,1\}^{\log(T)-i} \\ j_2' \in \{0,1\}^{\log(T)-i+1}}} & \widetilde{\mathsf{eq}}\left(r', (w_1, c, j_1'), (w_2, j_2')\right) \\ & \cdot \widetilde{\mathsf{ra}}_1(r_1, w_1, c, j_1') \cdot \widetilde{\mathsf{ra}}_2(r_2, w_2, j_2'). \end{aligned}$$

Clearly, $\widetilde{\mathsf{eq}}\left(r', (w_1, c, j_1'), (w_2, j_2')\right) = 0$ unless $j_{2,>1}' = j_1'$. So this sum simplifies to

$$\begin{aligned} \widetilde{\mathsf{Val}}(r) \cdot \sum_{\substack{j_1' \in \{0,1\}^{\log(T)-i} \\ b \in \{0,1\}}} & \widetilde{\mathsf{eq}}\left(r', (w_1, c, j_1'), (w_2, b, j_1')\right) \\ & \cdot \widetilde{\mathsf{ra}}_1(r_1, w_1, c, j_1') \cdot \widetilde{\mathsf{ra}}_2(r_2, w_2, b, j_1'). \end{aligned} \tag{55}$$

For any vector $x \in \mathbb{F}^{\log T}$, let $x_{>i}$ denote the last $\log(T) - i$ coordinates of x. By Equation (51),

$$\widetilde{\mathsf{eq}}\left(r', (w_1, c, j_1'), (w_2, b, j_1')\right) = \left(\prod_{\ell=1}^{i-1} \left(r'_\ell w_{1,\ell} w_{2,\ell} + (1 - r'_\ell)(1 - w_{1,\ell})(1 - w_{2,\ell})\right)\right) \cdot \widetilde{\mathsf{eq}}(r'_i, c, b) \cdot \widetilde{\mathsf{eq}}(r'_{>i}, j_1', j_1').$$

Hence, letting

$$C = \widetilde{\mathsf{Val}}(r) \cdot \left(\prod_{\ell=1}^{i-1} \left(r'_\ell w_{1,\ell} w_{2,\ell} + (1 - r'_\ell)(1 - w_{1,\ell})(1 - w_{2,\ell})\right)\right), \tag{56}$$

Expression (55) equals:

$$C \cdot \sum_{j_1' \in \{0,1\}^{\log(T)-i}, b \in \{0,1\}} \widetilde{\mathsf{eq}}(r'_i, c, b) \cdot \widetilde{\mathsf{eq}}(r'_{>i}, j_1', j_1') \cdot \widetilde{\mathsf{ra}}_1(r_1, w_1, c, j_1') \cdot \widetilde{\mathsf{ra}}_2(r_2, w_2, b, j_1'). \tag{57}$$

Furthermore, note that for $j_1' \in \{0,1\}^{\log(T)-i}$,

$$\widetilde{\mathsf{eq}}(r'_{>i}, j_1', j_1') = \widetilde{\mathsf{eq}}(r'_{>i}, j_1'),$$

where the polynomial $\widetilde{\mathsf{eq}}$ on the right hand side refers to Equation (3). Hence, Expression (57) equals:

$$C \cdot \sum_{j_1' \in \{0,1\}^{\log(T)-i}, b \in \{0,1\}} \widetilde{\mathsf{eq}}(r'_i, c, b) \cdot \widetilde{\mathsf{eq}}(r'_{>i}, j_1') \cdot \widetilde{\mathsf{ra}}_1(r_1, w_1, c, j_1') \cdot \widetilde{\mathsf{ra}}_2(r_2, w_2, b, j_1'). \tag{58}$$

Similarly, turning our attention to the prover's message in round $t + 1$,

$$s_{t'+1}(c) = C \cdot \widetilde{\mathsf{eq}}(r'_i, w_{1,i}, c) \cdot \sum_{j_2' \in \{0,1\}^{\log(T)-i}} \widetilde{\mathsf{eq}}(r'_{>i}, j_2') \cdot \widetilde{\mathsf{ra}}_1(r_1, w_1, w_{1,i}, j_2') \cdot \widetilde{\mathsf{ra}}_2(r_2, w_2, c, j_2'). \tag{59}$$

Prover algorithm and cost accounting for $d = 2$. The evaluation-point accounting in this paragraph assumes characteristic greater than two, so that $0, 1, 2$ are distinct and multiplication by 2 uses addition. The general-d coefficient algorithm below works in every characteristic. Before the final $d \log T$ rounds of Shout's read-checking sum-check protocol begin, the prover computes and stores $\log(T) - 1$ arrays $E_{\log(T)-1}, \ldots, E_1$, where E_i contains all evaluations $\widetilde{\mathsf{eq}}(r'_{>i}, j_{>i})$ as $j_{>i}$ ranges over $\{0,1\}^{\log(T)-i}$. Also, at the start of these final $d \log T$ rounds, the prover will have already computed two arrays, say A_1 and A_2, that store all evaluations of the form $\widetilde{\mathsf{ra}}_1(r_1, j)$ and $\widetilde{\mathsf{ra}}_2(r_2, j)$ as j ranges over $\{0,1\}^{\log T}$. As per the standard linear-time sum-check prover algorithm (Section 6.2), the prover will bind A_1 and A_2 (see Equation (32)) as appropriate round-over-round. Per Section 7.2.1, if $K^{1/d} = o(T)$, the cost of binding these arrays round-over-round is a low-order cost, so we ignore it in our accounting below.

The prover can maintain the value C from Equation (56) round-over-round with only a constant number of field multiplications per round.

During the first round in Stage i, with prover message given by Expression (58). Note that for $c = 0$, $\widetilde{\mathsf{eq}}(r'_i, c, b) = 0$ if $b = 1$, so for $c = 0$ $b = 1$ need not be considered at all by the prover. In addition, $\widetilde{\mathsf{eq}}(r'_{>i}, j'_1)$ can be obtained with a single lookup into E_i. The prover can compute Expression (58) for $c = 0$ with $2T/2^i + O(1)$ field multiplications. Expression (58) for $c = 2$ can be computed with $4T/2^i + O(1)$ multiplications (twice as many as for $c = 0$ because both $b = 0$ and $b = 1$ must be considered when $c = 2$).

The prover's message for second round of Stage i (Expression (59)) requires at most $4T/2^i$ field multiplications, with $2T/2^i$ required for $c = 0$ and the same number required for $c = 2$.

Hence, each stage $i = 1, \ldots, \log(T)$ requires $10T/2^i$ field multiplications for the prover. Summing across all i, this is at most $10T$ multiplications. Accounting also for the $T/2$ multiplications to compute the arrays $E_{\log(T)-1}, \ldots, E_1$, in total across all stages i this is $10.5 \cdot T$ field multiplications.

An optimization when $K^{1/d}$ is $o(\sqrt{T})$. When $K^{1/d} = o(\sqrt{T})$, a significant optimization is possible, similar to the one described in Section 7.2.1 that renders binding the arrays A_1 and A_2 a low-order cost. At the start of the final $d \log T$ rounds of sum-check, for each $\ell \in \{1, 2\}$, $\widetilde{\mathsf{ra}}_\ell(r_\ell, j)$ takes on only $K^{1/d}$ distinct values as j ranges over $\{0,1\}^{\log T}$. Hence, all possible products of the form $\widetilde{\mathsf{ra}}_1(r_1, j_1) \cdot \widetilde{\mathsf{ra}}_2(r_2, j_2)$ as j_1 and j_2 range over $\{0, 1, 2\} \times \{0,1\}^{\log(T)-1}$ can be computed and stored in a table of size $O(K^{2/d})$, thereby saving the prover the need to incur T field multiplications to compute such quantities arising in Expression (57) for round $\log(T) + 1$. Similar optimizations apply for several rounds, with the size of the lookup table of "pre-computed products of $\widetilde{\mathsf{ra}}_1$ and $\widetilde{\mathsf{ra}}_2$ evaluations" growing (rapidly) round-over-round. After a few rounds, the lookup tables would have size larger than T and this optimization is no longer helpful.

Depending on how much smaller $K^{1/d}$ is than $\sqrt{T}$, this can save up to about half of the $10T$ field multiplications accounted for above. This yields a total number of multiplications of roughly $5.5 \cdot T$.

8.7.2 General d: prover messages and cost accounting

We now describe the same calculation for any $d \geq 2$. Each stage fixes one more cycle bit in each of the d factors, one factor per round. Let $t = \log T$. At the start of stage i, the first $i - 1$ cycle bits in factor ℓ have been fixed to challenges $w_\ell = (w_{\ell,1}, \ldots, w_{\ell,i-1})$. The remaining Boolean cycle bits must agree across the factors for the equality polynomial to be nonzero. Write $h \in \{0,1\}^{t-i}$ for their common value after the current bit. It will help to name the partially evaluated factors and the equality weight on these remaining bits:

$$F_\ell(c, h) = \widetilde{\mathsf{ra}}_\ell(r_\ell, w_\ell, c, h), \qquad E(h) = \widetilde{\mathsf{eq}}(r'_{>i}, h).$$

In round M, the current bit in factors $1, \ldots, M-1$ has already been fixed to $w_{1,i}, \ldots, w_{M-1,i}$. We group the terms into three products: C_i contains the contributions from completed stages, P_M

contains the factors processed earlier in this stage, and S_M^b contains those still to come. Specifically,

$$C_i = \widetilde{\mathsf{Val}}(r) \prod_{z=1}^{i-1} \left(r'_z \prod_{\ell=1}^{d} w_{\ell,z} + (1 - r'_z) \prod_{\ell=1}^{d} (1 - w_{\ell,z}) \right), \tag{60}$$

$$P_M(h) = E(h) \prod_{\ell<M} F_\ell(w_{\ell,i}, h), \qquad S_M^b(h) = \prod_{\ell>M} F_\ell(b, h), \quad b \in \{0,1\}. \tag{61}$$

For $M < d$, all unbound current bits must equal a common Boolean bit b. Thus the round polynomial is

$$s_{\log K + d(i-1)+M}(c) = C_i \sum_{b\in\{0,1\}} \widetilde{\mathsf{eq}}(r'_i, w_{1,i}, \ldots, w_{M-1,i}, c, b) \cdot \sum_{h\in\{0,1\}^{t-i}} P_M(h) F_M(c,h) S_M^b(h). \tag{62}$$

The single occurrence of b in the equality factor suffices because b is Boolean, so repeated powers of b or $1-b$ equal themselves. We keep this equality factor inside the sum over b, since its value can differ for $b = 0$ and $b = 1$. For the last round, no Boolean current bit remains to be summed:

$$s_{\log K + di}(c) = C_i \widetilde{\mathsf{eq}}(r'_i, w_{1,i}, \ldots, w_{d-1,i}, c) \sum_{h\in\{0,1\}^{t-i}} P_d(h) F_d(c,h). \tag{63}$$

Both expressions follow by factoring the bitwise equality polynomial in Equation (51). Each has degree at most two in c.

Linear-time implementation. Let $N = T/2^i$, the number of possible values of h. We can reuse products across the d rounds of a stage. First, compute S_M^0 and S_M^1 by working backwards from the last factor, multiplying in one factor at a time. Doing this for every h and both choices of b costs $2(d-2)N$ multiplications. Next, start with $P_1 = E$. After receiving each challenge, multiply the newly fixed factor into this product to obtain P_{M+1}. Across the stage, this costs $(d-1)N$ multiplications. The equality factors do not depend on h, so they need only be computed once per round. Maintaining their products in the same way costs $O(d)$ multiplications per stage.

We specify each quadratic message by its constant and quadratic coefficients. If the incoming claim is H and the message is $a_0 + a_1 c + a_2 c^2$, its evaluations at 0 and 1 must sum to H, so $a_1 = H - 2a_0 - a_2$ in every characteristic. For $M < d$, put

$$\alpha_0 = (1 - r'_i) \prod_{\ell<M} (1 - w_{\ell,i}), \quad \alpha_1 = r'_i \prod_{\ell<M} w_{\ell,i}, \quad D_M(h) = F_M(1,h) - F_M(0,h).$$

Equation (62) gives

$$a_0 = C_i \alpha_0 \sum_h P_M(h) F_M(0,h) S_M^0(h),$$

$$a_2 = C_i \left(\alpha_1 \sum_h P_M(h) D_M(h) S_M^1(h) - \alpha_0 \sum_h P_M(h) D_M(h) S_M^0(h) \right).$$

The first sum costs $2N$ multiplications. The two sums for a_2 share $P_M D_M$, so together cost $3N$. Public scalar factors cost $O(1)$ per round. In the last round, omit the suffix products: the constant coefficient costs N multiplications and the quadratic coefficient costs another N. Binding one variable in each of the d factor tables costs dN per stage. Including both suffix choices and all variable bindings, these final $d \log T$ rounds therefore cost at most

$$\begin{aligned} &\frac{T}{2} + \sum_{i=1}^{\log T} \big(2(d-2) + (d-1) + 5(d-1) + 2 + d\big) \frac{T}{2^i} + O(d \log T) \\ &\qquad \leq (9d - 7.5)T + O(d \log T). \end{aligned} \tag{64}$$

The initial $T/2$ term builds the suffix equality tables. The address-bound tables $\widetilde{\mathsf{ra}}_\ell(r_\ell, \cdot)$ are prepared during the first $\log K$ rounds, whose costs are additional. The cycle rounds use $O(dT)$ field operations and space.

Binding with small memories. Let $n = K^{1/d}$ and fix d. Section 7.2.1's binding optimization reduces the dT term to $O(d\sqrt{nT})$ multiplications: use tables of the n possible initial values times all 2^q prefix weights for the first q stages, and ordinary binding afterwards, with 2^q near $\sqrt{T/n}$. Consequently, when $n = o(T)$ the bound becomes

$$(8d - 7.5)T + O(d\sqrt{nT} + d \log T) = (8d - 7.5)T + o(T). \tag{65}$$

This optimization counts multiplications; forming entries from these tables uses additions. Ordinary binding remains available for the $O(dT)$ bound on total field operations.

Final-round optimization. In Equation (63), write the message as $L(c)U(c)$, where L is the known linear equality factor including C_i, and $U(c) = \sum_h P_d(h) F_d(c, h)$ is linear. If $L(1) \neq 0$, compute $U(0)$ with N multiplications and recover $U(1)$ from $L(0)U(0) + L(1)U(1) = H$. If $L(1) = 0$ but $L(0) \neq 0$, compute $U(1)$ instead and recover $U(0)$. If both are zero, the message is identically zero. This is Gruen's optimization (Section 6.5), with no division by zero. It saves $T + O(1)$ multiplications across the stages, giving $(9d - 8.5)T + O(d \log T)$ with ordinary binding, or $(8d - 8.5)T + o(T)$ when d is fixed and $n = o(T)$. It also uses at most one field inversion per stage.

Precomputation of short products, as in the $d = 2$ case, can further reduce the prefix and suffix work. Such tables must include both Boolean suffix choices and the challenges already received. These additional savings are not needed for the explicit bounds above.

9 Fast Twist prover implementation

Recall from Figure 9 that the core Twist PIOP consists of three sum-check invocations: the read-checking sum-check, the write-checking sum-check, and the $\widetilde{\mathsf{Val}}$-evaluation sum-check.

9.1 $\widetilde{\mathsf{Val}}$-evaluation sum-check

Let us start by considering the $\widetilde{\mathsf{Val}}$-evaluation sum-check, as it is the simplest. This invocation applies the sum-check protocol to compute:

$$\sum_{j' \in \{0,1\}^{\log T}} \widetilde{\mathsf{Inc}}(j') \cdot \widetilde{\mathsf{wa}}(r_{\text{address}}, j') \cdot \widetilde{\mathsf{LT}}(j', r_{\text{cycle}}).$$

We explain in Section 9.2 that, with $3T/2$ field multiplications, the prover can compute a table storing all evaluations of $\widetilde{\mathsf{LT}}(j', r_{\text{cycle}})$ as j' ranges over $\{0,1\}^{\log T}$. Similar to the procedure for computing all evaluations of $\widetilde{\mathsf{ra}}(\tau, j)$ described in Section 7.2.1, the prover can compute all $\widetilde{\mathsf{wa}}(r_{\text{address}}, j')$ evaluations with K field multiplications.

Given the above two tables, the standard linear-time sum-check prover algorithm (Section 6.2) performs $6T$ additional field multiplications. Hence, total prover time in a straightforward implementation of the $\widetilde{\mathsf{Val}}$-evaluation sum-check is $K + 7.5 \cdot T$ field multiplications.

9.2 Computing the Less-Than evaluation table

In the $\widetilde{\mathsf{Val}}$-evaluation sum-check in Twist (Figure 9), the prover needs to compute $\widetilde{\mathsf{LT}}(j', r)$ for all $j' \in \{0,1\}^{\log T}$ for $r = r_{\text{cycle}} \in \mathbb{F}^{\log T}$. Here, $\widetilde{\mathsf{LT}}$ is the multilinear extension of the $\mathsf{LT}(x, y)$ function defined in Section 4.2, which indicates whether or not x is the binary representation of an integer that is strictly less than (the integer with binary representation given by) y.

We explain here how the prover can do this with $3T/2$ field multiplications. We use simple, explicit expressions for $\widetilde{\mathsf{LT}}$ given in [STW24].

Let $x = (x_1, \ldots, x_{\log T})$ and $y = (y_1, \ldots, y_{\log T})$. If x and y are binary representations of integers between 0 and $T-1$, we think of x_1 and y_1 as the "low-order bit" of the two binary representations, and $x_{\log T}$ and $y_{\log T}$ as the high-order bit, consistent with the convention in Section 4.2.

For $i = 1, \ldots, \log T$, let $x_{>i} = (x_{i+1}, \ldots, x_{\log T})$ and $x_{\geq i} = (x_i, \ldots, x_{\log T})$. Define

$$\widetilde{\mathsf{LT}}_i(x, y) = (1 - x_i) \cdot y_i \cdot \widetilde{\mathsf{eq}}(x_{>i}, y_{>i}). \tag{66}$$

It is easy to see that

$$\widetilde{\mathsf{LT}}(x, y) = \sum_{i=1}^{\log T} \widetilde{\mathsf{LT}}_i(x, y). \tag{67}$$

Note that LT_i does not depend on the first (i.e., low-order) $i-1$ bits of x. Accordingly, while Equation (66) expresses LT_i as a function of all $\log T$ bits of x and y, we can equivalently think of LT_i as a function only of the last $\log(T) - (i-1)$ bits of x and y. We will follow this convention henceforth.

Suppose the prover has already computed a table E_i defined as follows:

$$E_i \text{ stores all } T/2^i \text{ evaluations of } \widetilde{\mathsf{eq}}(j'_{>i}, r_{>i}) \text{ as } j'_{>i} \text{ ranges over } \{0,1\}^{\log(T)-i}. \tag{68}$$

Suppose further that the prover has also already computed a table D_i of size $T/2^{i-1}$ such that for all $x \in \{0,1\}^{\log(T)-(i-1)}$,

$$D_i[x] = \sum_{\ell=i}^{\log T} \widetilde{\mathsf{LT}}_\ell(x, r_{\geq \ell}). \tag{69}$$

Via standard observations [VSBW13] (see Lemma 4.1), the tables $E_{\log(T)}, \ldots, E_1$ can all be computed with $T/2$ field multiplications in total, with $E_{\log T}$ containing the single empty-product value. Furthermore, given table E_i, the table D_i (which recall has size $2 \cdot T/2^i$) can be computed with just $T/2^i$ field multiplications. To see this, observe that the following holds.

Let $x = (x_i, \ldots, x_{\log T}) \in \{0,1\}^{\log(T)-(i-1)}$ and let $x_{>i} = (x_{i+1}, \ldots, x_{\log T})$. Then:

- If $x_i = 1$, $D_i[x] = D_{i+1}[x_{>i}]$. This is because

$$\widetilde{\mathsf{LT}}_i(x, r_{\geq i}) = (1 - x_i) \cdot r_i \cdot \widetilde{\mathsf{eq}}(x_{>i}, r_{>i}) = 0.$$

- If $x_i = 0$ then $D_i[x] = D_{i+1}[x_{>i}] + r_i \cdot E_i[x_{>i}]$.

The table D_1 is exactly the table we want, containing all T evaluations of $\widetilde{\mathsf{LT}}(j', r)$ as j' ranges over $\{0,1\}^{\log T}$. As explained above, D_1 can be computed with $3T/2$ multiplications in total. $T/2$ of these are devoted to computing all of the E_i tables, and an additional T multiplications are devoted to computing all of the D_i tables.

9.3 Read-checking and write-checking sum-checks

Achieving a fast prover in the read-checking and write-checking sum-checks requires new techniques. Let us begin by focusing on the read-checking sum-check, which in the case $d = 1$ confirms that $\widetilde{\mathsf{rv}}(r')$ equals:

$$\sum_{(k,j) \in \{0,1\}^{\log K} \times \{0,1\}^{\log T}} \widetilde{\mathsf{eq}}(r', j) \cdot \widetilde{\mathsf{ra}}(k, j) \cdot \widetilde{\mathsf{Val}}(k, j). \tag{70}$$

9.3.1 Overview

We call our Twist read-checking and write-checking prover algorithm the "local algorithm" because it has the following major advantage. Call a memory operation 2^i-*local* if it accesses a memory cell that was previously accessed within the last 2^i cycles. Locality reduces the aggregate prover work by reducing the number of distinct cells in aligned blocks of cycles. Section 9.3.3 bounds the total work by counting the distinct cells in these blocks, including cells accessed for the first time and cells accessed after long gaps.

In our local algorithm, we bind the $\log T$ variables of the cycle j first, starting with the "low-order" bit j_1 of j and proceeding towards the high-order bit $j_{\log T}$. This is in contrast to the read-checking algorithm for Shout, which binds the $\log(K)$ variables of the register address k first, rather than the $\log T$ variables of j. Intuitively, this allows the prover to benefit from local memory accesses during the first $\log K$ rounds of sum-check because round i "coalesces" the values at any given register k across a contiguous chunk of 2^i time steps. So, if fewer than 2^i distinct registers were read or written during these 2^i time steps, then the prover does less than 2^i work to process those time steps in round i. This leads to less than T total prover work in round i, resulting in a prover time that is less than $T \log K$ across those $\log K$ rounds. After round $\log K$, there are only T terms to be summed, and the standard linear-time sum-check achieves $O(T)$ *total* time for those rounds (as opposed to $O(T)$ time *per round*).

9.3.2 Details of the local algorithm for read-checking

Consider the standard linear-time sum-check prover implementation (Section 6.2). Our local-algorithm prover can be viewed as simply running this standard procedure, but optimized for the specific structure of this sum-check instance. Recall that for the local algorithm, we restrict our attention to the case that $d = 1$.

Tracking increments instead of values. Naively implemented, the standard algorithm requires $K \cdot T$ space, with the bottleneck being storing all $K \cdot T$ evaluations of $\mathsf{Val}(k, j)$ for $k \in \{0,1\}^{\log K}$ and $j \in \{0,1\}^{\log T}$. However, we can avoid this by exploiting the same observation that leads to fast commitments: rather than explicitly storing all $\widetilde{\mathsf{Val}}(k, j)$ values for $(k, j) \in \{0,1\}^{\log K} \times \{0,1\}^{\log T}$, the prover can instead just store, for each $j \in \{0,1\}^{\log T}$, the unique register k that was written at cycle j, and the corresponding non-zero increment, namely the difference between the value written at cycle j and the value stored at k just prior to that write.

Under this definition, for all $k \in \{0,1\}^{\log K}$ and $j \in \{0,1\}^{\log T}$,

$$\widetilde{\mathsf{Val}}(k, j) = \sum_{j' < j} \widetilde{\mathsf{Inc}}(j') \cdot \widetilde{\mathsf{wa}}(k, j'), \tag{71}$$

where $j' < j$ is shorthand for $\mathsf{LT}(j', j) = 1$, i.e., for j' being the binary representation of an integer $\text{int}(j')$ that is strictly less than $\text{int}(j)$. Note that Equation (71) is *not* an equality of formal polynomials, but rather an equality that holds pointwise at all $(k, j) \in \{0,1\}^{\log K} \times \{0,1\}^{\log T}$. Nonetheless, Equation (71) implies that for any $r_1, \ldots, r_i \in \mathbb{F}$ and any $j \in \{0,1\}^{\log(T)-i}$,

$$\widetilde{\mathsf{Val}}(k, r_1, \ldots, r_i, j) = \sum_{j' \in \{0,1\}^{\log T}} \widetilde{\mathsf{Inc}}(j') \cdot \widetilde{\mathsf{wa}}(k, j') \cdot \widetilde{\mathsf{LT}}(j', r_1, \ldots, r_i, j). \tag{72}$$

Indeed, the right hand side of Equation (72) is a multilinear polynomial in the $\log(K) + \log(T)$ variables $(k, r_1, \ldots, r_i, j)$, and by Equation (71) and the definition of LT, the right hand side and left hand side agree whenever $(k, r_1, \ldots, r_i, j) \in \{0,1\}^{\log K} \times \{0,1\}^{\log T}$.

Simplifying Expression (72)**.** Break the T cycles $j \in \{0,1\}^{\log T}$ into $T/2^i$ "chunks" each of size 2^i. For each $j \in \{0,1\}^{\log(T)-i}$, chunk j refers to the collection of all cycles in $\{0,1\}^{\log T}$ whose last $\log(T) - i$ bits equal j, i.e., all cycles of the form $(\hat{j}, j)$ for some $\hat{j} \in \{0,1\}^i$.

We say that chunk $\bar{j}$ is strictly less than chunk j (denoted $\bar{j} < j$) if $\widetilde{\mathsf{LT}}(\bar{j}, j) = 1$, i.e., if $\bar{j}$ is the binary representation of an integer strictly less than the integer represented by j. Similarly, we say that chunk $\bar{j}$ is strictly greater than chunk j (denoted $\bar{j} > j$) if $\bar{j}$ is the binary representation of an integer strictly greater than j.

If $j', j'' \in \{0,1\}^{\log T}$ are respectively in chunks $\bar{j}$ and j such that $\bar{j} < j$, then $\widetilde{\mathsf{LT}}(j', j'') = 1$. Meanwhile, if $\bar{j} > j$ then $\widetilde{\mathsf{LT}}(j', j'') = 0$.

Let $j^* = (0^i, j)$ denote the first cycle in chunk j, where 0^i denotes the all-zeros vector of length i. Then a cycle $j' \in \{0,1\}^{\log T}$ is in a chunk $\bar{j} < j$ if and only if $j' < j^*$. Combining this with the preceding paragraph and Fact 4.1, we conclude that:

$$\widetilde{\mathsf{Val}}(k, r_1, \ldots, r_i, j'') = \widetilde{\mathsf{Val}}(k, j^*) + \sum_{\hat{j} \in \{0,1\}^i,\ j = (\hat{j}, j'')} \widetilde{\mathsf{Inc}}(j) \cdot \widetilde{\mathsf{wa}}(k, j) \cdot \widetilde{\mathsf{LT}}(j, r_1, \ldots, r_i, j'')$$

where

$$\widetilde{\mathsf{Val}}(k, j^*) = \sum_{j' \in \{0,1\}^{\log T} \colon j' < j^*} \widetilde{\mathsf{Inc}}(j') \cdot \widetilde{\mathsf{wa}}(k, j').$$

It follows from the definition of $\widetilde{\mathsf{LT}}$ (Equation (67)) that for any $j = (\hat{j}, j'')$, $\widetilde{\mathsf{LT}}(j, r_1, \ldots, r_i, j'') = \widetilde{\mathsf{LT}}(\hat{j}, r_1, \ldots, r_i)$. Hence, we conclude that

$$\widetilde{\mathsf{Val}}(k, r_1, \ldots, r_i, j'') = \widetilde{\mathsf{Val}}(k, j^*) + \sum_{\hat{j} \in \{0,1\}^i \colon j = (\hat{j}, j'')} \widetilde{\mathsf{Inc}}(j) \cdot \widetilde{\mathsf{wa}}(k, j) \cdot \widetilde{\mathsf{LT}}(\hat{j}, r_1, \ldots, r_i). \tag{73}$$

Ensuring the prover runs in $O(T)$ time per round. In round i, for fixed $j' \in \{0,1\}^{\log(T)-(i-1)}$, let us refer to the vector

$$\left\{\widetilde{\mathsf{Val}}(k, r_1, \ldots, r_{i-1}, j') \colon k \in \{0,1\}^{\log K}\right\}$$

as *row j' of $\widetilde{\mathsf{Val}}$ in round i*. The prover will proceed iteratively through the rows in lexicographic order, starting with row $0^{\log(T)-i}$ and proceeding to row $1^{\log(T)-i}$.

Per the above, the prover can maintain an array I that in round i stores all non-zero values of the form:

$$\sum_{\hat{j} \in \{0,1\}^i \colon j = (\hat{j}, j'')} \widetilde{\mathsf{Inc}}(j) \cdot \widetilde{\mathsf{wa}}(k, j) \cdot \widetilde{\mathsf{LT}}(\hat{j}, r_1, \ldots, r_i) \tag{74}$$

and

$$\sum_{\hat{j} \in \{0,1\}^i \colon j = (\hat{j}, j'')} \widetilde{\mathsf{Inc}}(j) \cdot \widetilde{\mathsf{wa}}(k, j) \tag{75}$$

for $j'' \in \{0,1\}^{\log(T)-i}$.

Observe that Expresion (74) is precisely the summation appearing in the right-hand side of Equation (73), thus ensuring that when processing row j'', the prover can indeed store in its memory all K values $\widetilde{\mathsf{Val}}(k, r_1, \ldots, r_i, j'')$ as k ranges over $\{0,1\}^{\log K}$.

Since there is only one non-zero increment per cycle, it follows easily from Fact 4.1 that, for any fixed $j'' \in \{0,1\}^{\log(T)-i}$, there are at most 2^i non-zero values of Expression (74) (respectively, Expresion (75)).

To update the Expression (74) values round over round, the prover computes the following:

$$\begin{aligned}
&\sum_{\hat{j} \in \{0,1\}^i \colon j = (\hat{j}, j'')} \widetilde{\mathsf{Inc}}(j) \cdot \widetilde{\mathsf{wa}}(k, j) \cdot \widetilde{\mathsf{LT}}(\hat{j}, r_1, \ldots, r_i) \\
&\qquad = \left(r_i \cdot \sum_{\hat{j} \in \{0,1\}^{i-1} \colon j = (\hat{j}, 0, j'')} \widetilde{\mathsf{Inc}}(j) \cdot \widetilde{\mathsf{wa}}(k, j) \right) \\
&\qquad + \left(\sum_{b \in \{0,1\}} \widetilde{\mathsf{eq}}(b, r_i) \cdot \sum_{\hat{j} \in \{0,1\}^{i-1} \colon j = (\hat{j}, b, j'')} \widetilde{\mathsf{Inc}}(j) \cdot \widetilde{\mathsf{wa}}(k, j) \cdot \widetilde{\mathsf{LT}}(\hat{j}, r_1, \ldots, r_{i-1}) \right)
\end{aligned}$$

where the parenthesized expressions can be computed from the Expresion (74) and (75) values from the previous round using one multiplications per non-zero value. Expression (75) values are updated by simply summing pairs of values from the previous round.

In summary, for the first $\log(K)$ rounds of the read-checking sum-check, with T multiplications per round, the prover can ensure that in each round i, it can iterate over rows j' one-by-one and when processing row j' ensure that it has in its memory the K values

$$\widetilde{\mathsf{Val}}(k, r_1, \ldots, r_{i-1}, j') \colon k \in \{0,1\}^{\log K}. \tag{76}$$

Computing other necessary arrays and worst-case cost accounting. Via standard techniques (see Lemma 4.1), with K field multiplications across all of the first $\log K$ rounds, the prover can maintain at each round i an array A storing all evaluations $\widetilde{\mathsf{eq}}(x, r_1, \ldots, r_i)$ as x ranges over $\{0,1\}^i$. Since only one register is read per cycle, at the end of round i, each row j'' of $\widetilde{\mathsf{ra}}(\cdot, r_1, \ldots, r_i, j'')$ has at most 2^i non-zero entries. Each such entry is the *sum* of the entries of A corresponding to reads of that cell within the block; it need not be a single entry of A when addresses repeat. These sums can be maintained by sparse binding, in $O(T)$ field operations per round. This suffices for the $O(K + T \log K + T)$ worst-case bound without the further multiplication-count optimizations below.

Similarly, with standard techniques, the prover can across all rounds i maintain an array storing all evaluations $\widetilde{\mathsf{eq}}(r', r_1, \ldots, r_i, j'')$ as j'' ranges over $\{0,1\}^{\log(T)-i}$. This costs $2T$ field multiplications in total (T to build the size-T array at the start of the protocol, and T more to bind it round-over-round over the course of the protocol). These are the arrays (along with I) needed to run the standard linear-time sum-check proving algorithm described in Section 6.2. However, a significant optimization is possible to avoid these $2T$ field multiplications, and more significantly, to also save one field multiplication per relevant evaluation point c for each of the first $\log K$ rounds.

An optimization. The optimizations of Dao and Thaler [DT24] can be adapted to speed up the prover further.

Specifically, assuming $K = o(T)$ so $\sqrt{KT} = o(T)$, the prover builds two different arrays of size at most $2\sqrt{KT}$, defined as follows. Let

$$r'_L = (r'_1, \ldots, r'_h) \quad \text{and} \quad r'_R = (r'_{h+1}, \ldots, r'_{\log T}), \qquad h = \lceil (\log K + \log T)/2 \rceil.$$

The first array E, at the end of round i, stores the following values:

$$E[j''] = \widetilde{\mathsf{eq}}(r'_L, r_1, \ldots, r_i, j'') \quad \text{for } j'' \in \{0,1\}^{h-i}. \tag{77}$$

Intuitively, $E(j'')$ captures the contribution of the "left part" j'' (of length $h - i$) of a cycle index $j = (j'', j''') \in \{0,1\}^{\log(T)-i}$ to the value $\widetilde{\mathsf{eq}}(r', r_1, \ldots, r_i, j)$.

The second array E', in round i, stores the following values

$$E'[x, j'''] = A(x) \cdot \widetilde{\mathsf{eq}}(r'_R, j''') \quad \text{for } x \in \{0,1\}^i \text{ and } j''' \in \{0,1\}^{\log T - h}. \tag{78}$$

Repeated reads of the same cell require aggregation of these table entries. To make this explicit, consider round s, let $h = \lceil (\log K + \log T)/2 \rceil$, and split the remaining cycle bits into j'' of length $h - s$ and j''' of length $\log T - h$. For an evaluation point $c \in \{0, 2, 3\}$, precompute

$$G^c_s(x, j''') = \widetilde{\mathsf{eq}}((r_1, \ldots, r_{s-1}, c), x)\widetilde{\mathsf{eq}}(r'_R, j'''), \qquad x \in \{0,1\}^s.$$

For each cell k read in block (j'', j'''), form

$$W_s^c(k, j'', j''') = \sum_{\substack{x \in \{0,1\}^s \\ \mathsf{ra}(k,x,j'',j''')=1}} G_s^c(x, j''').$$

Thus $W_s^c = \widetilde{\mathsf{eq}}(r'_R, j''')\widetilde{\mathsf{ra}}(k, r_1, \ldots, r_{s-1}, c, j'', j''')$. Each summand is a table lookup. Repeated reads add terms to the same W_s^c, using additions and no further multiplications. The prover computes

$$\mathsf{agg}_{j''} = \sum_{j''',k} W_s^c(k, j'', j''')\widetilde{\mathsf{Val}}(k, r_1, \ldots, r_{s-1}, c, j'', j''')$$

with one multiplication per distinct cell in each block, then combines these aggregates with the left equality weights $\widetilde{\mathsf{eq}}(r'_L, (r_1, \ldots, r_{s-1}, c, j''))$. This yields the prescribed $s_s(c)$. Across the first $\log K$ rounds, the lookup tables and left equality weights cost $O(\sqrt{KT})$ multiplications, since their sizes form geometric sequences. The tables for the constant number of evaluation points only change this low-order term. This accounting ignores additions, as in our cost model; rebuilding the aggregates by scanning reads uses $O(T \log K)$ additions. For a bound on total field operations that benefits from locality, use sparse binding instead, as below.

Worst-case cost accounting. In total, maintaining the arrays E (Expression (77)) and E' (Expression (78)) across the first $\log K$ rounds requires $O(\sqrt{KT}) = o(T)$ field multiplications. Maintaining the array I (Expressions (74) and (75)) costs at most $2^i \cdot (T/2^i) = T$ multiplications per round, as there are $T/2^i$ rows in round i and computing one row given the previous row requires at most 2^i multiplications. This yields a total of $T \log K$ multiplications to maintain the array I over the first $\log K$ rounds. We call these *write-induced multiplications.*

Given these arrays, the prover in round $i = 1, \ldots, \log K$ performs at most $\left(T/2^i\right) \cdot 2^i + O(\sqrt{TK}) = T + o(T)$ field multiplications for each of the relevant evaluation points $s_i(c)$. Since the read-checking sum-check expression is cubic in the cycle variables j, there are three evaluation points, namely $c \in \{0, 2, 3\}$. This means there are $3T \log K + o(T)$ field multiplications, which we call *read-induced multiplications.*

In total, this means the first $\log K$ rounds of the sum-check protocol applied to Expression (70) incur at most the following number of field multiplications:

$$4T \log K + o(T).$$

After round $\log K$, the number of terms being summed has fallen from KT down to T, and the standard linear-time sum-check proving algorithm (appropriately optimized [DT24, Gru24]) requires at most $6T$ total field multiplications to implement all subsequent rounds. In fact, when $K = o(T)$, this falls to roughly $5T$ due to binding of the E array taking much less than T field operations, as in each round i, there are two different algorithms the prover can choose between to bind E, one taking time 2^i, which leverages that at the start of the protocol E contains only values in $\{0, 1\}$, and the other taking time $T \cdot K/2^i$.

Hence, in total, for small memories the number of prover field multiplications is upper bounded by roughly:

$$(4 \log K + 6)\, T + o(T).$$

9.3.3 A refined time bound for local memory accesses

The dominant term in the (worst-case) time bound for our main prover algorithm is $4T \log K$ field multiplications. This comes from performing 4 multiplications per cycle for each of the first $\log K$ rounds. One of these four multiplications is *write-induced*: it comes from updating the array I: there are only $T/2^i$ "rows" of I stored in round i, but computing one row given the previous row can require 2^i field multiplications. This means that every write operation, in the worst case, contributes one field multiplication per round, for each of the first $\log K$ rounds.

The other (at most) three field multiplications per cycle per round are read-induced. Each of the $T/2^i$ rows of E at the end of round i has at most 2^i distinct values, which means round $(i+1)$ incurs at most $2^i \cdot (T/2^i)$ field multiplications per evaluation point $c \in \{0, 2, 3\}$. This means that every read operation, in the worst case, contributes three field multiplications per round for the first $\log K$ rounds.

Refined cost analysis for write-induced operations. At the end of round i, updating the array I requires at most 2^i field multiplications. However, a tighter bound holds. For any $j' \in \{0,1\}^{\log(T)-i}$ consider cycles between $\text{int}(j') \cdot 2^i$ and $(\text{int}(j')+1) \cdot 2^i$. These are the cycles whose high-order $\log(T) - i$ bits equal j'. If at most Z distinct registers were written during these cycles, then in round i, computing row j' of I from the preceding row requires only Z field multiplications. In particular, if there were D duplicate writes during these cycles (meaning a write to a register that was already written during these cycles) then Z equals $2^i - D$.

Refined cost analysis for read-induced operations. Similarly, if there were D "duplicate reads" during these cycles, then the number of field multiplications in round i due to read operations performed during these cycles falls from a worst-case bound of $3 \cdot 2^i$ to $3 \cdot (2^i - D)$.

From short gaps to a bound on total work. Recall that round s groups cycles into consecutive blocks of size 2^s, starting at cycle 0. Nearby accesses can fall on opposite sides of a block boundary. For example, cycles 7 and 8 are one cycle apart, but first share a block when its size reaches 16. To account for this, we count the distinct cells in all blocks of a given size, then sum over block sizes.

Assume $d = 1$ and $2 \le K \le T$. Fix a gap length $D = 2^i$, where $0 \le i \le \log K$. Call an access *exceptional* if it is the first access to its cell, or if the preceding access to that cell was more than D cycles earlier. Let B be the number of exceptional accesses among the $2T$ reads and writes.

Consider the first access to a given cell within a block of 2^s cycles. If it is not exceptional, it must occur within the first D cycles of the block. Otherwise, the preceding access to that cell would also lie within the block, contradicting our choice of the first access. There are at most two accesses per cycle. Thus each block contains at most $2D$ distinct cells whose first access in the block is not exceptional. Let Z_s be the number of distinct cells summed over all blocks of size 2^s; a cell appearing in two blocks is counted twice. Since there are $T/2^s$ blocks and $2T$ accesses in total,

$$Z_s \le \min\{2T,\ B + 2DT/2^s\}.$$

This also bounds the number of read entries and write entries the prover processes at this stage. Combining adjacent blocks and updating their stored values takes $O(Z_s)$ field operations, with a constant factor to account for both blocks in each pair.

For the first i stages, use $Z_s \le 2T$. For the remaining stages, use the second bound and sum the geometric series $\sum_{s>i} 2DT/2^s \le 2T$. This gives

$$\sum_{s=1}^{\log K} Z_s \le 2Ti + B \log K + 2T.$$

Adding the $O(K+T)$ work to initialize the tables and finish the remaining rounds, the core prover's total work is

$$O\big(K + T(i+1) + B \log K\big).$$

The three terms account for initialization, accesses with short gaps, and exceptional accesses, respectively. In particular, if $B \log K = O(T(i+1))$, the bound becomes $O(K + T(i+1))$. The one-hot checks add $O(K+T)$ work for $d = 1$, including the additional check for small-characteristic fields in Section 5.5.

Explicit multiplication bound. The preceding lookup-based implementation also permits a quantitative refinement in the paper's multiplication cost model, which does not charge for additions. Let R_s and W_s count distinct read cells and distinct written cells, respectively, summed over aligned blocks of 2^s cycles, and put $L = \log K$. The first L rounds of the read and write sum-checks together cost at most

$$\sum_{s=1}^{L} (3R_s + 4W_s) + T + O(\sqrt{KT}) \tag{79}$$

multiplications: three message evaluations per distinct read and per distinct write, one shared increment-table update per distinct write, and at most T multiplications to fold the scalar $\widetilde{\mathsf{Inc}}$ table used in the write sum-check. This uses evaluation points $0, 2, 3$ in characteristic greater than three and assumes $K = o(T)$ for the lookup-table term to be low order. Since $R_s, W_s \le T$ and $R_s, W_s \le Z_s$, Equation (79) is at most

$$\min\big\{7TL,\ 7Ti + 7B(L-i) + 14T(1-2^{i-L})\big\} + T + O(\sqrt{KT}).$$

The remaining rounds and the value-evaluation sum-check are additional. The multiplication bound and the bound on total field operations use the two implementations described above: lookup aggregation for the former, sparse binding for the latter.

Implementing the write-checking sum-check prover is essentially identical to the read-checking case, so we omit further details for brevity.

9.4 Cost summary

Local algorithm for $d = 1$. If $K = o(T)$, then using the local proving algorithm, the entire core Twist PIOP can be implemented with the following number of field multiplications, up to low-order terms:

- $7.5 \cdot T$ from the $\widetilde{\mathsf{Val}}$-evaluation sum-check.
- $(4 \log K + 6)T$ from the read-checking sum-check.
- An additional $(3 \log K + 5)T$ from the write-checking sum-check.

This is at most $(7\log(K) + 18.5)\,T$ field operations to process T reads and T writes. For $K = 32 = 2^5$, this is $53.5 \cdot T$ field multiplications. For $K = 2^{20}$, it is $158.5 \cdot T$.

Locality can reduce this work. Equation (79) replaces the $7T \log K$ contribution by a count of distinct read and written cells in aligned blocks. Section 9.3.3 separately gives $O(K + T(i+1) + B\log K)$ total field work using sparse binding, where B counts first accesses and accesses after gaps longer than 2^i cycles.

Checking correctness of one-hot encodings (Figure 8) for all $2T$ committed addresses must be done in addition to the above costs. Per Section 7.3, this costs $6dT + O(dK^{1/d})$ field multiplications (up to low-order terms, all of this extra cost is coming from Booleanity-checking).

The alternative algorithm. The alternative algorithm has similar (in fact, slightly better) costs in the worst case to the local algorithm when $d = 1$, but generalizes more nicely to $d > 1$ because it binds the variables of k before the variables of j, and the degree in the variables of k is independent of d for all of the polynomials that the sum-check protocol is applied to within Twist. Specifically, the costs are:

- $7.5T + K$ from the $\widetilde{\mathsf{Val}}$-evaluation sum-check, using the straightforward implementation of Section 9.1.
- $(3\log(K) + d^2 + 2d + 1)T$ from the read-checking sum-check.
- An additional $(2\log(K) + d^2 + 2d + 2)T$ from the write-checking sum-check.

In total, this is at most $(5\log(K) + 2d^2 + 4d + 10.5)T$ field multiplications, plus the stated lower-order terms. For $d = 1$, the leading term is $(5\log(K) + 16.5)T$, compared with $(7\log(K) + 18.5)T$ for the local algorithm in its worst case. Booleanity checking adds $6dT$ to either algorithm when required by the commitment scheme.

10 Discussion and cost summary

As with offline memory-checking techniques such as Spice, Twist and Shout can be formulated as polynomial IOPs for the memory-checking problem, and the PIOP can be turned into a SNARK by combining it with any suitable polynomial commitment scheme. Recall that Twist and Shout actually refer to a family of PIOPs parametrized by an integer parameter $d > 0$ that helps control commitment key size (in the case of elliptic curve commitment schemes) or commitment time (in the case of hashing-based commitment schemes). The higher d is, the more non-zero values the Twist and Shout provers commit to, and the more field work the Twist and Shout provers do. With elliptic curves commitment schemes d will typically be between 1 and 8, and with hashing-based commitments d will typically be between 1 and 16.

10.1 Proof size and verifier costs

Twist and Shout avoid invoking a grand product argument. They invoke the sum-check protocol a constant number of times, leading to proofs of size $O(d \cdot (\log(T) + \log(K)))$, where T is the number of memory operations and K the size of the memory. When combined with a commitment scheme that has logarithmic-size evaluation proofs such as HyperKZG, Zeromorph, or Dory, they

yield SNARKs for memory-checking whose proofs consist of $O(d(\log(T) + \log(K)))$ field and group elements. Via standard batching techniques, the verifier only needs to perform a single evaluation proof verification, which involves a constant number of MSMs of size $O(\log(T) + \log(K))$ and a constant number of pairing evaluations.

10.2 Prover time

Commitment costs. For read operations, both the Twist and Shout provers commit only to the address ra specified via (d-dimensional) one-hot encoding. These commitments are unavoidable, as the memory-checking problem formulation itself requires the address to be committed. The *values* returned by any read operation, captured by the multilinear polynomial $\widetilde{\mathsf{rv}}$, need *not* be committed: $\widetilde{\mathsf{rv}}$ is a virtual polynomial.

For write operations, the Twist prover commits to the address and the value written (or, more precisely, a value we call the write *increment*, a notion we will define later. For purposes of cost analysis, what is important is that if write values are "small", then increments are small too).

Field work. We give a variety of algorithms for quickly implementing the prover in the various sum-check invocations throughout the Twist and Shout PIOPs. Which algorithm to use depends on various aspects including: is the number of memory operations T significantly bigger than the memory size K? In the case of Twist, are reads and writes to memory local? Here, we provide a concise summary of the prover times we achieve in different contexts. We explicitly state Booleanity-checking costs in each regime (i.e., the cost of proving that various committed values lie in $\{0, 1\}$), as Booleanity-checking can be omitted when using a commitment scheme like Binius [DP23, DP24] that itself enforces Booleanity of committed values.

Shout with $K = o(T)$. We hide additive terms of $O(K)$ and $O(K^{1/d} \log K)$ in this summary. The core Shout PIOP with $d = 1$ (Figure 5) costs T field multiplications for the prover (Section 7.1). The core Shout PIOP for $d > 1$ (Figure 7) costs $(d^2 + 1)T$ field multiplications (Section 7.2). Booleanity-checking (Section 7.3) costs an additional $3dT$ field multiplications, with other parts of the one-hot-encoding checking PIOP in Figure 8 contributing only low-order costs for the prover. This translates to $4T$ multiplications in total when $d = 1$, $11T$ multiplications when $d = 2$, and so forth.

Shout for gigantic structured memories. Let $C = cd$ be a constant such that $CK^{1/C} = o(T)$.Then the structured-memory prover analysis of Section 8.5, applied to the general-d core Shout PIOP and combined with one-hot-encoding-checking, costs at most $\left(7C + d^2 + 3d + c + 2\right) T$ field multiplications. For example, when $C = 4$, $d = 2$, and $c = 2$, this translates to $42T$ multiplications. Of these, $(4C + 3d)T$ are due to Booleanity checking. See Section 8.5 for details.

In Section 8.6, we describe a variation of Shout with prover work that has a linear rather than quadratic dependence on d. For fixed d and $K^{1/d} = o(T)$, its final cycle rounds cost at most $(8d - 7.5)T + o(T)$ multiplications, or $(8d - 8.5)T + o(T)$ with the final-round optimization described there. These counts cover the cycle rounds alone; address rounds and one-hot checks are additional.

Twist costs. Again we assume $K = o(T)$ and $K^{1/d} \log K = o(T)$ and hence suppress additive terms of $O(K)$ and $O(K^{1/d} \log K)$. The core Twist PIOP (Figure 9) costs $(5 \log(K) + 2d^2 + 4d + 10.5)T$ field multiplications in the worst case, up to the lower-order terms stated in Section 9.4. For $d = 1$, the separate local algorithm can reduce this cost for local memory accesses. Section 9.3.3 gives its bound in terms of the gaps between accesses to the same cell. Booleanity-checking costs an additional $6dT$ field multiplications (again, other parts of the one-hot-encoding checking PIOP are low-order costs).

Comparing to prior works. Figures 1–3 compare prover costs for Twist and Shout with Spice, Lasso, and LogUpGKR using elliptic-curve-based commitment schemes.

Two important settings for the Jolt zkVM are: lookups into gigantic structured tables (i.e., how Lasso is used in the baseline Jolt implementation to prove correct execution of primitive RISC-V instructions), and reads and writes into 32 RISC-V registers (two such reads and one such write occur in Jolt for nearly every cycle of the RISC-V CPU).

For structured tables, Shout with $d = 2$ commits $2T$ rather than Lasso's at least $16T$ nonzero values, and uses $42T$ rather than at least $144T$ field multiplications: reductions of at least $8\times$ and $144/42 \approx 3.4\times$, respectively. At $d = 4$, the corresponding reductions are at least $4\times$ and $144/59 \approx 2.4\times$. At $d = 8$, Shout commits $8T$ values, while the displayed multiplication bounds are $147T$ for Shout and at least $144T$ for Lasso.

For 32 read/write registers and $d = 1$, the leading terms in Figure 1 give about $11/3 \approx 3.7\times$ fewer nonzero committed values and $80/47.5 \approx 1.7\times$ fewer field multiplications than Spice, when $K = o(T)$. Figure 4 compares committed bits with hashing-based commitments. For 32 registers, Twist commits $128T$ bits versus Spice's $421T$, about $3.3\times$ less. For structured lookup tables of size 2^{64}, Shout commits $256T$ bits versus Lasso's at least $928T$, at least $3.6\times$ less.

Comparison of Shout to FLI. Shout with $c = 1$ has quite a bit in common with FLI [GM24]: They both represent addresses via one-hot encoding, commit to the one-hot encodings with curve-based commitments (at the cost of one group operation per address for the prover), and observe that given the one-hot encodings, the validity of the lookups can then easily be expressed via R1CS. Unlike Shout, FLI then directly applies a Nova-type folding scheme [KST22] to fold these R1CS instances. Shout instead applies the sum-check protocol to directly prove satisfaction of the R1CS. Inspired by Lasso [STW24], FLI also considers lookups into giant, decomposable tables. In this context, our improvements over FLI are analogous to our improvements over Lasso: by avoiding table decompositions, Shout lowers the amount of committed data (cf. Section 2.3).

Comparison of Twist to Proofs for Deep Thought. Bünz and Chen [BC25] present an incremental verifiable computation (IVC) framework for read/write memory (while FLI targets read-only memory). Their solution involves techniques reminiscent of Twist's use of increments. However, to check correctness of the increments they invoke Spice [SAGL18] and LogUp [Hab22]. The focus of Twist and Shout is precisely to eliminate the overheads of these prior memory-checking arguments. Hence, Bünz and Chen's work can be viewed as targeting how to efficiently "link" the state of memory between adjacent shards in an instance of IVC, but they incur overheads similar to the baselines we consider and improve upon.

10.3 Remarks on design choices

Benefits of avoiding grand product arguments. Beyond the concrete cost improvements detailed in Figures 1–4, avoiding grand products yields qualitative benefits. Grand-product-based memory checking requires committing to timestamps and range-checking them, adding protocol complexity. The most efficient grand product argument [Tha13] achieves only $O(\log^2 n)$ proof size; achieving $O(\log n)$ via Setty-Lee [SL20] requires committing to partial products that are *arbitrary field elements*—an order of magnitude more expensive than small values. Twist and Shout achieve $O(\log T)$ proof size while committing only to the problem instance itself (addresses, values, and increments), which is information-theoretically optimal.

On hashing-based commitment schemes. We discuss hashing-based polynomial commitment schemes (Binius [DP23], FRI-Binius [DP24], Blaze [BCF+24]) for completeness. These offer plausible post-quantum security and compatibility with small-characteristic fields supporting faster arithmetic. However, curve-based schemes yield proofs an order of magnitude smaller. In practice, many projects using hashing-based commitments "wrap" proofs inside curve-based SNARKs for small final proof sizes, sacrificing post-quantum security. Our experiments use Dory as it currently offers the best concrete performance for applications prioritizing proof size, and avoids complications that arise when building zkVMs over the small-characteristic fields used by Binius, FRI-Binius, and Blaze.

Forward-looking directions. All concrete results in this paper use curve-based multilinear polynomial commitments over 256-bit fields. Looking forward, we expect two largely orthogonal improvements.

First, we are optimistic about the prospects of a future implementation of Twist, Shout, and Jolt that replaces Dory with a lattice-based commitment scheme like Hachi [NOZ26], to achieve post-quantum security while *also* obtaining a large (4× or more) prover speedup by allowing the use of 128-bit instead of 256-bit fields. This estimate is based on (i) profiling evidence that Jolt prover time is dominated by prime-field multiplications in the current implementation, and (ii) microbenchmarks indicating that 128-bit field multiplications can be $\approx 6\times$ faster than 256-bit Montgomery multiplications (see Section 6.1 for 256-bit cost baselines [Mon85, Xop]). Such lattice commitments have many of the same properties of curve-based commitments exploited by Twist and Shout (e.g., "pay-per-bit" commitment costs, allowing very fast commitments to large, sparse vectors).

Second, subsequent work has shown that Jolt with Twist and Shout is amenable to streaming prover implementations using space that grows sublinearly with the number of cycles proved [NTZ25]. Realizing this potential requires careful systems work (data layout, batching strategies, and I/O patterns); this work is well underway, but we leave a full implementation and experimental evaluation to the future.

11 Performance evaluation

In this section, we evaluate the concrete performance of Twist and Shout. We integrated Twist and Shout into the Jolt zkVM [AST24], with Twist replacing Spice [SAGL18] and Shout replacing Lasso [STW24] as the memory checking arguments underlying the zkVM.

Our evaluation contains two parts. First, we compare end-to-end performance for RV32IM Jolt (RISC-V with the Multiplication extension and 32-bit registers) with Spice and Lasso (henceforth $\text{Jolt}_{\mathcal{SL}}$) against Jolt with Twist and Shout (henceforth $\text{Jolt}_{\mathcal{TS}}$), measuring prover-time and proof-size improvements over the prior state of the art. Second, we extend $\text{Jolt}_{\mathcal{TS}}$ to support the RV64IMAC instruction set (RISC-V with the Multiplication extension and others, and *64-bit* registers) and run it on a 32-core CPU. The purpose of this second part is to assess how fast $\text{Jolt}_{\mathcal{TS}}$ can be, especially at larger cycle counts. The first comparison is the primary apples-to-apples evaluation: it changes the memory-checking machinery inside Jolt while keeping the surrounding prover stack fixed as much as possible.

Summary of findings. The first part of our experimental analysis shows that Twist and Shout substantially improve prover speed and proof size as predicted by our analytic cost estimates. Proof size falls by about 5×, to roughly 50 KB, and prover time falls by about 3× (except at tiny cycle counts, i.e., about a million cycles and below). For reference, simple tasks like verifying a single standard digital signature require hundreds of thousands of cycles. We have not made an effort to optimize $\text{Jolt}_{\mathcal{TS}}$ for such small cycle counts, and indeed $\text{Jolt}_{\mathcal{TS}}$ can be substantially sped up in this regime by increasing d and modifying the manner in which polynomials are committed to better balance the cost of computing commitments versus computing evaluation proofs.

	Prover time (s)			Proof size (kB)	
Guest program	$\text{Jolt}_{\mathcal{SL}}$	$\text{Jolt}_{\mathcal{TS}}$	speedup	$\text{Jolt}_{\mathcal{SL}}$	$\text{Jolt}_{\mathcal{TS}}$
Fibonacci	289.54	86.75	3.34×	267.7	48.8
SHA2 chain	309.63	96.43	3.21×	271.4	49.0
SHA3 chain	314.68	97.94	3.21×	271.4	49.0
BTreeMap	297.00	98.78	3.01×	280.6	49.8

Table 1: Representative RV32IM end-to-end benchmarks at 2^{25} cycles. The $\text{Jolt}_{\mathcal{TS}}$ proof sizes include the standard compression of BN254 G_T elements [NBS08, Lee21].

Our analytic cost estimates "ignored" $O(\sqrt{K^{1/d} \cdot T})$ $\mathbb{G}_T$ scalar multiplications and pairings performed by the Dory prover, with the justification that this quantity grows sublinearly with T. Even though a single pairing or scalar multiplication is concretely expensive, as T grows they still become a vanishing fraction of total prover time. Our experiments confirm this is reasonable, at least when the cycle count is not tiny. For $T \geq 2^{26}$, Dory evaluation proofs are well under 20% of total prover, and every 4-fold increase in cycle count roughly halves the fraction of prover time spent on these scalar multiplications and pairings.

A final qualitative finding is that Twist for RAM makes up a very small fraction of the Jolt prover time, only about 5%. This may seem surprising given that the $O(T \log K)$ field operations required for Twist in the worst case is a dominant term in the Jolt prover's asymptotic runtime. There are two reasons that RAM accesses are concretely fast to prove. First, the Twist-for-RAM prover "pays nothing" on cycles that do not invoke a LOAD or STORE operation (i.e., cycles that do not access RAM). Second, the Twist prover runs in time less than $O(T \log K)$ when memory accesses are "local".

In the second part of our experiments, we show that on a 32-core CPU, the Jolt prover can prove over one million RV64IMAC cycles per second at the largest benchmarked cycle counts.

On proof size comparisons. Our 5× proof size improvement compares $\text{Jolt}_{\mathcal{TS}}$ (with G_T point compression [NBS08, Lee21]) to $\text{Jolt}_{\mathcal{SL}}$ (with HyperKZG). The improvement stems primarily from *fewer sum-check rounds*—the vast majority of $\text{Jolt}_{\mathcal{SL}}$'s proof size comes from sum-check transcripts in the GKR-based grand product arguments, not from the polynomial commitment scheme. Instantiating $\text{Jolt}_{\mathcal{SL}}$ with Dory would yield even larger proofs, so our improvement would exceed 5×.

On prover-time impact of the PCS choice. Our decision to instantiate $\text{Jolt}_{\mathcal{SL}}$ with HyperKZG is also conservative for prover time, not only for proof size. While HyperKZG evaluation proofs require committing to random field elements, which is slow, in our $\text{Jolt}_{\mathcal{SL}}$ implementation evaluation proofs are heavily batched and hence their computation is a minor fraction of total prover time relative to the field-arithmetic dominated GKR/sum-check work. Consequently, using HyperKZG does not materially increase prover time in our measurements, while it strictly improves proof size; allowing $\text{Jolt}_{\mathcal{SL}}$ to use Dory would therefore only make $\text{Jolt}_{\mathcal{SL}}$ look worse on proof size (and not meaningfully better on prover time) in our comparison.

Zero-knowledge overhead. The measurements above focus on the succinctness-oriented prover path, but the Jolt prover can be made zero-knowledge via BlindFold-style masking as discussed in Section 1. In our implementation, enabling this zero-knowledge mode does not materially change prover time and adds roughly 3 KB to proof size, so it does not affect the relative comparisons between $\text{Jolt}_{\mathcal{SL}}$ and $\text{Jolt}_{\mathcal{TS}}$.

On codebase versions. For the RV32IM comparison, we benchmark the last RV32IM-supporting revisions of both $\text{Jolt}_{\mathcal{SL}}$ and $\text{Jolt}_{\mathcal{TS}}$, with $\text{Jolt}_{\mathcal{TS}}$ obtained by swapping Spice/Lasso for Twist/Shout within the same Jolt prover stack. After producing these RV32IM benchmarks, subsequent systems optimization effort focused on extending and optimizing $\text{Jolt}_{\mathcal{TS}}$ for RV64IMAC, and the RV32IM code path has not been kept in sync. Consequently, the $\approx 3\times$ prover-time reduction reported for the RV32IM experiments should be interpreted as a *conservative* snapshot of Twist/Shout performance within Jolt.

Prover space efficiency. A striking feature of $\mathsf{Jolt}_{\mathcal{TS}}$ is its low memory footprint: at the largest benchmarked cycle count, the prover uses approximately 350 bytes per cycle proved. In particular, Jolt can prove over a million RV64IMAC cycles using under two gigabytes of prover space. This space efficiency suggests that Jolt proofs can be produced in resource-constrained environments such as web browsers or mobile devices, and is more than sufficient for many important applications. For reference, proving knowledge of a valid ECDSA digital signature with Jolt takes under 250,000 cycles, while SHA3-hashing 40 KB of data requires about one million cycles.

This space efficiency stems directly from Twist and Shout's design: by avoiding grand products and the associated need to commit to timestamps, partial products, and range-check witnesses, the prover's memory footprint scales linearly with cycle count with a small constant factor. As mentioned earlier, subsequent work [NTZ25] has shown that $\mathsf{Jolt}_{\mathsf{TS}}$ is highly amenable to a *streaming prover* implementation that uses space *sublinear* in the number of CPU cycles T, with minimal slowdown relative to an implementation using space linear in T. An implementation of a streaming Jolt prover is well underway.

Additional discussion. Jolt$_{\mathcal{TS}}$ exhibits "economies of scale". This means the bigger the cycle count T, the faster the Jolt prover is *per cycle*. In the current experiments, we show an economy of scale that's primarily derived from the fact that Dory evaluation proof computation time (and multi-pairings performed during commitment) grow sub-linearly with T, and hence represent a vanishing fraction of prover costs as T increases. In general, however, other economies of scale can be leveraged. These arise from the dimension parameter d in the Twist and Shout instances (larger d leads to faster evaluation proofs but more proving time devoted to committing and to various sum-check invocations), and the table-structure parameter C_{tbl} determining the number of field operations $O(C_{\text{tbl}}K^{1/C_{\text{tbl}}} + C_{\text{tbl}}T)$ performed by the Shout prover for instruction execution. This experimental parameter is distinct from the shorthand $C = cd$ used in the generic structured-Shout cost summary. For example, in Shout for primitive instruction execution for RV32IM, d is fixed to 8 and C_{tbl} to 4 for all cycle counts larger than 2^{24}. For very large cycle counts T (e.g., $T \geq 2^{30}$), we could profitably reduce d and C_{tbl}, and see even faster prover times (measured in cycles proved per second) as a result. Similarly, and as mentioned earlier, many potential concrete optimizations to Jolt at very small cycle counts (e.g., $T \leq 2^{20}$) remain available for exploration.

12 Additional experimental results

12.1 Comparison of Lasso and Spice to Twist and Shout

12.1.1 Experimental setup

Jolt$_{\mathcal{SL}}$ is instantiated using the HyperKZG [ZSC24] polynomial commitment scheme.[11] Jolt$_{\mathcal{TS}}$ is instantiated using the Dory polynomial commitment scheme. Both Jolt$_{\mathcal{SL}}$ and Jolt$_{\mathcal{TS}}$ are instantiated using the BN254 elliptic curve.

We use the term *guest program* to refer to the program that the Jolt prover proves it ran correctly. We benchmarked Jolt on the following guest programs, varying the inputs such that the length of the proven execution trace ranges from 2^{18} to 2^{25} RISC-V cycles. More precisely, the guest programs were compiled to RV32IM (RISC-V 32-bit Base Instruction Set, plus the "M" Standard Extension for Integer Multiplication and Division). In Jolt, the number of memory operations T proven by each of its constituent memory checking instances equals the length of the execution trace.

- **Fibonacci**: Naïvely computes the n'th number in the Fibonacci sequence, modulo 2^{128}.
- **SHA2 chain**: Computes an iterated chain of SHA2 hashes, i.e. `SHA2`(`SHA2`$(\ldots(x))$.
- **SHA3 chain**: Computes an iterated chain of SHA3 hashes, i.e. `SHA3`(`SHA3`$(\ldots(x))$.
- **BTreeMap**: A series of insertions, deletions, and reads performed on an instance of Rust's `BTreeMap` data structure.

All benchmarks were run on a 2023 M3 Max MacBook Pro (16-core CPU, 128 GiB RAM) unless otherwise indicated. Both Jolt$_{\mathcal{SL}}$ and Jolt$_{\mathcal{TS}}$ were benchmarked with multithreading.[12] Full

[11] Were Jolt$_{\mathcal{SL}}$ to be instantiated with Dory, it would not result in a substantially faster prover, and would lead to slightly larger proofs.

[12] The benchmarked code for Jolt$_{\mathcal{SL}}$ and Jolt$_{\mathcal{TS}}$ is available at `https://github.com/a16z/jolt/tree/bench/`

benchmark results are provided in Tables 2 and 3. Reported cycle counts are rounded to the next power of two.

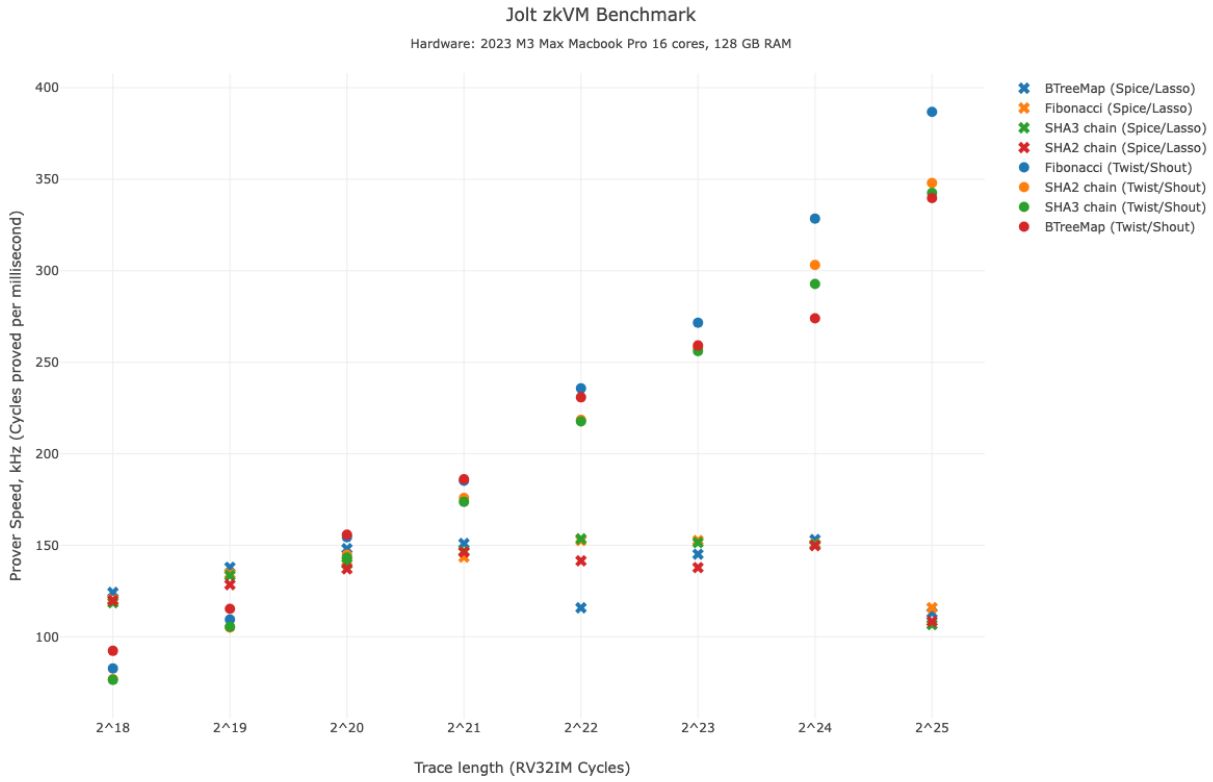


Figure 10: Jolt prover speed, comparing Spice/Lasso to Twist/Shout

The *dimension* parameter d for each Twist and Shout instance is set dynamically so that $K^{1/d} \leq 256$, where K denotes the relevant memory size for that instance. For most realistic workloads, this leads to the following parameter settings:

- Twist for register reads and writes: $d = 1$.
- Twist for RAM reads and writes: $2 \leq d \leq 4$ ($d = 4$ supports up to 16 GB of guest memory).
- Shout for instruction execution lookups: $d = 8$.
- Shout for bytecode lookups: $2 \leq d \leq 3$ ($d = 3$ supports bytecode sizes up to 64 MB).

All benchmarks reported in this section use $d = 2$ for both Twist instances, with the exception of the BTreeMap guest program, which uses $d = 3$ for RAM when the cycle count Y exceeds 2^{23} ($T \geq 2^{24}$).

`jolt-classic-rv32` and `https://github.com/a16z/jolt/tree/bench/twist-shout-rv32`, respectively. These branches correspond to the last RV32IM-supporting snapshot used for the RV32IM comparison; subsequent optimization work has focused on the RV64IMAC Jolt$_{\mathcal{TS}}$ implementation.

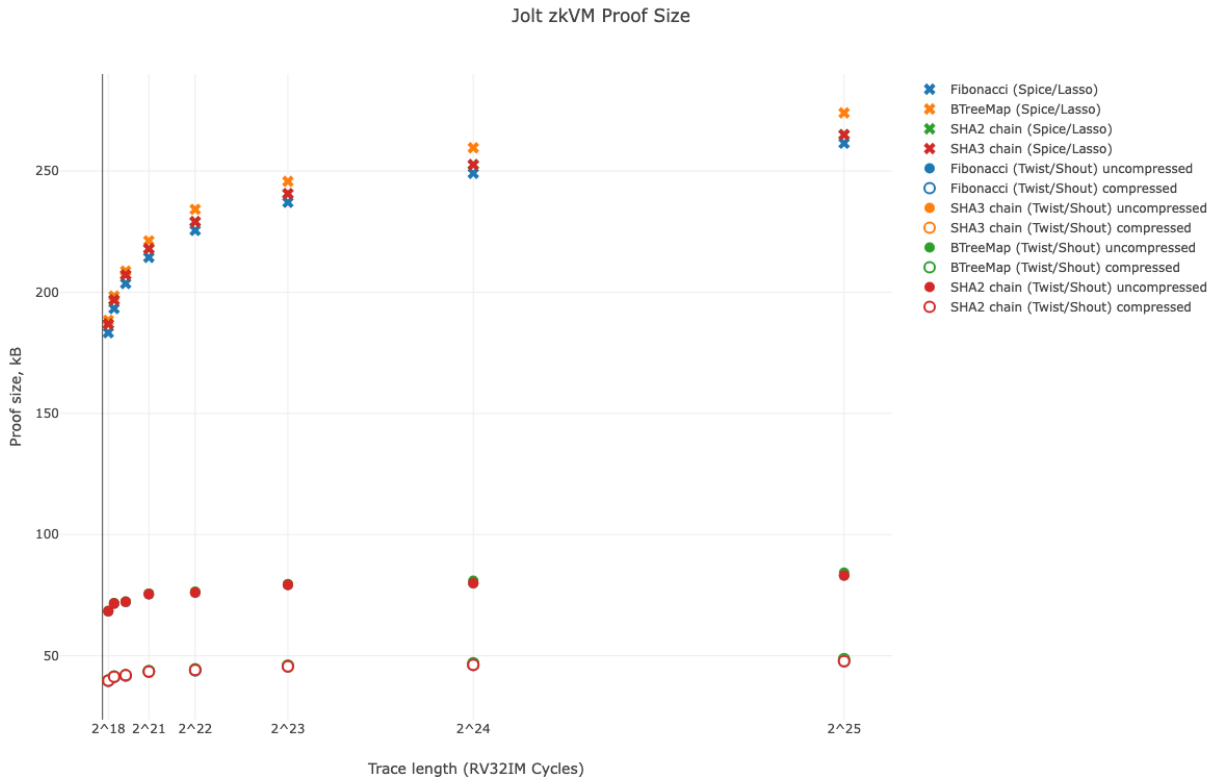


Figure 11: Jolt proof size, comparing Spice/Lasso to Twist/Shout

12.1.2 Prover speed details

Figure 10 compares the speed of the Jolt$_{\mathcal{SL}}$ and Jolt$_{\mathcal{TS}}$ provers across various cycle counts. For shorter cycle counts, Jolt$_{\mathcal{TS}}$ is bottlenecked by the concretely expensive multi-pairings and scalar multiplications arising in the Dory commitment and opening proof protocols (and we see that Jolt$_{\mathcal{SL}}$ is faster than Jolt$_{\mathcal{TS}}$ for cycle counts $\leq 2^{19}$). However, these multi-pairings are of size $\Theta(\sqrt{TK^{1/d}})$, so their cost scales sublinearly in the cycle count T. We observe that $T = 2^{20}$ is the threshold at which Jolt$_{\mathcal{TS}}$ becomes faster than Jolt$_{\mathcal{SL}}$, with the gap widening as T grows larger, reaching a 3× improvement over Jolt$_{\mathcal{SL}}$ at $T = 2^{25}$. While Jolt$_{\mathcal{TS}}$ prover speed improves with larger T (mainly due to the sublinear growth of Dory proving costs), we observe that Jolt$_{\mathcal{SL}}$ is relatively flat at around 150 kHz for $2^{20} \leq T \leq 2^{24}$, with a slight drop-off at $T = 2^{25}$, where Jolt$_{\mathcal{SL}}$ begins to use swap memory.

The sublinear cost of the Dory multi-pairings and scalar multiplications can be seen directly in Figure 12, with Table 4 providing a more granular breakdown of prover time. While both the Dory commitment and evaluation proof computation involve multi-pairings (and scalar multiplications) of size $\Theta(\sqrt{TK^{1/d}})$, committing also involves a linear-size MSM. This is why the relative cost of the evaluation proof shrinks more quickly compared to commitment: committing is a linear-size MSM *plus* a multi-pairing of size $\Theta(\sqrt{TK^{1/d}})$, while evaluation proofs involve only $\Theta(\sqrt{TK^{1/d}})$ pairings

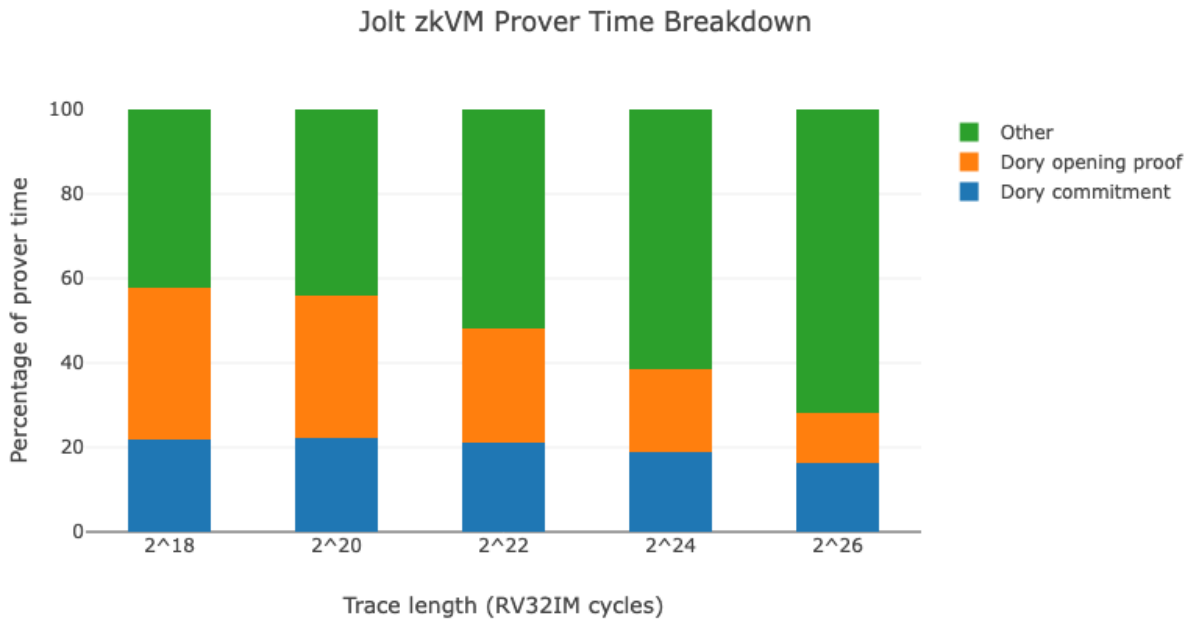


Figure 12: Fraction of Jolt prover costs due to Dory in $\text{Jolt}_{\mathcal{TS}}$ (benchmarked on SHA2 chain guest program)

and scalar multiplications.

Table 4 also shows that Twist and Shout are concretely efficient: in aggregate, all the Twist and Shout sum-checks in Jolt comprise less than half of the total prover time.

12.1.3 Proof size details

Figure 11 compares the proof size for $\text{Jolt}_{\mathcal{SL}}$ and $\text{Jolt}_{\mathcal{TS}}$ across various cycle counts. For $\text{Jolt}_{\mathcal{TS}}$ we report two proof sizes: the solid circles represent proof sizes as currently implemented, while the hollow circles represent the projected proof sizes after a $3\times$ compression [NBS08, Lee21] of BN254 G_T elements (which arise in Dory commitments and opening proofs). Even without G_T point compression, $\text{Jolt}_{\mathcal{TS}}$ proofs are approximately $3\times$ smaller than $\text{Jolt}_{\mathcal{SL}}$ proofs, and with compression the proofs are $5\times$ smaller, comprising around 50 KB. These concrete costs reflect superior asymptotics: the grand product proofs in Spice and Lasso consist of $\Theta\left(\log^2(T)\right)$ field elements, while $\text{Jolt}_{\mathcal{TS}}$ proofs contain $\Theta\left(\log(TK^{1/d})\right)$ field elements.

Guest program	cycle count	Prover time (s)	Proof size (kB)
Fibonacci	2^{18}	2.18	187.711
	2^{19}	3.90	197.895
	2^{20}	7.43	208.495
	2^{21}	14.62	219.511
	2^{22}	27.49	230.943
	2^{23}	54.96	242.791
	2^{24}	111.35	255.055
	2^{25}	289.54	267.735
SHA2 chain	2^{18}	2.19	191.359
	2^{19}	4.08	201.543
	2^{20}	7.65	212.143
	2^{21}	14.34	223.159
	2^{22}	29.65	234.591
	2^{23}	60.89	246.439
	2^{24}	112.07	258.703
	2^{25}	309.63	271.383
SHA3 chain	2^{18}	2.21	191.359
	2^{19}	3.93	201.543
	2^{20}	7.50	212.143
	2^{21}	14.29	223.159
	2^{22}	27.32	234.591
	2^{23}	55.40	246.439
	2^{24}	111.63	258.703
	2^{25}	314.68	271.383
BTreeMap	2^{18}	2.11	192.991
	2^{19}	3.80	203.175
	2^{20}	7.08	213.775
	2^{21}	13.89	226.527
	2^{22}	36.21	239.799
	2^{23}	57.81	251.647
	2^{24}	109.58	265.855
	2^{25}	297.00	280.583

Table 2: Jolt$_{\mathcal{SL}}$ benchmarks, RV32IM

Guest program	cycle count	Prover time (s)	Pf. size (kB)	Compressed pf. size (kB)
Fibonacci	2^{18}	3.17	69.876	40.550
	2^{19}	4.79	73.148	42.142
	2^{20}	6.79	73.828	42.774
	2^{21}	11.32	77.100	44.366
	2^{22}	17.79	77.780	44.998
	2^{23}	30.89	81.052	46.590
	2^{24}	51.07	81.732	47.222
	2^{25}	86.75	85.004	48.814
SHA2 chain	2^{18}	3.41	70.020	40.694
	2^{19}	4.99	73.292	42.286
	2^{20}	7.24	73.972	42.918
	2^{21}	11.93	77.244	44.510
	2^{22}	19.19	77.924	45.142
	2^{23}	32.53	81.196	46.734
	2^{24}	55.35	81.876	47.366
	2^{25}	96.43	85.148	48.958
SHA3 chain	2^{18}	3.43	70.020	40.694
	2^{19}	4.97	73.292	42.286
	2^{20}	7.32	73.972	42.918
	2^{21}	12.07	77.244	44.510
	2^{22}	19.27	77.924	45.142
	2^{23}	32.75	81.196	46.734
	2^{24}	57.30	81.876	47.366
	2^{25}	97.94	85.148	48.958
BTreeMap	2^{18}	2.84	70.124	40.798
	2^{19}	4.55	73.396	42.390
	2^{20}	6.73	74.076	43.022
	2^{21}	11.27	77.452	44.718
	2^{22}	18.17	78.236	45.454
	2^{23}	32.37	81.508	47.046
	2^{24}	61.23	82.898	48.068
	2^{25}	98.78	86.274	49.764

Table 3: Jolt$_{\mathcal{TS}}$ benchmarks, RV32IM

cycle count		2^{18}	2^{20}	2^{22}	2^{24}	2^{26}
Total prover time (s)		3.46	7.25	17.67	52.12	183.01
Dory commitment	seconds	0.76	1.62	3.75	9.76	29.82
	% of total	22.0	22.4	21.2	18.7	16.3
Dory opening proof	seconds	1.24	2.44	4.76	10.27	21.45
	% of total	35.8	33.7	26.9	19.7	11.7
Spartan	seconds	0.06	0.24	0.92	4.02	16.88
	% of total	1.8	3.3	5.2	7.7	9.2
Shout (instruction exec)	seconds	0.30	0.59	1.67	5.58	22.54
	% of total	8.7	8.1	9.4	10.7	12.3
Shout (bytecode)	seconds	0.06	0.12	0.40	1.61	6.50
	% of total	1.7	1.7	2.2	3.1	3.6
Twist (RAM)	seconds	0.08	0.17	0.49	1.69	6.84
	% of total	2.2	2.3	2.8	3.2	3.7
Twist (registers)	seconds	0.05	0.17	0.55	2.09	8.97
	% of total	1.5	2.3	3.1	4.0	4.9
Other	seconds	0.91	1.89	5.14	17.10	70.01
	% of total	26.4	26.1	29.1	32.8	38.3

Table 4: Jolt$_{\mathcal{TS}}$ prover time breakdown (benchmarked on SHA2 chain guest program)

12.2 Jolt with Twist and Shout for RV64IMAC

Figures 13 and 14 display prover time and proof size when Jolt$_{\mathcal{TS}}$ is applied to the RV64IMAC instruction set and the prover is run on an AMD Threadripper PRO 7975WX with 32 cores, 768 GB DDR5 RAM. The main findings are that the prover proves over one million cycles per second once the cycle count T is about 2^{27} or higher, and that the proof size is only slightly larger than for the RV32IM instruction set.

Support for 64-bit registers is important for two reasons: (1) Performance: Many applications can be executed with far fewer cycles using 64-bit registers. For example, a 256-bit multiplication requires 6× fewer cycles with 64-bit registers [vex25]. (2) Language support: Some high-level programming languages like Go don't currently compile to RV32IM. So if you want to prove execution of something like Geth (an Ethereum client written in Go), you need RISC-V with 64-bit registers.

An attractive property of Jolt$_{\mathcal{TS}}$ is that it is only about 1.5 times slower per cycle when proving 64-bit registers (RV64IMAC) than 32-bit ones (RV32IMAC). This is thanks to its use of large-characteristic fields (which are preferred by elliptic curve commitment schemes), which allows Jolt, and Twist and Shout, to store (even the product of two) register values in a single field element without overflow. In contrast, most other zkVMs work over 64-bit or even 31-bit fields, forcing them to break register values into limbs, commit each limb separately, and prove each committed limb is well-formed. Thus, other zkVMs are typically roughly 2× slower per cycle when doubling register size.

Prover memory usage Figure 15 displays the peak memory usage (maximum resident set size, as reported by `time -l`) of the Jolt$_{\mathcal{TS}}$-for-RV64IMAC prover across the same four guest programs. The cycle counts reported here are RV64IMAC cycles, as in Section 12.2.

Over the benchmarked range, peak memory usage grows less than proportionally with the trace length T. This is because there are constant overheads (e.g., precomputed Dory parameters) that are significant at small instance sizes but become a diminishing fraction of total memory as T grows. For example, at $T = 2^{28}$ the prover uses approximately 91–98 GB of RAM depending on the guest program, which is well under twice the roughly 68–81 GB used at $T = 2^{27}$. Full benchmark results are provided in Table 5.

Guest program	cycle count	Peak memory (GB)
Fibonacci	2^{18}	0.57
	2^{19}	1.04
	2^{20}	1.79
	2^{21}	3.25
	2^{22}	5.24
	2^{23}	10.33
	2^{24}	14.25
	2^{25}	29.23
	2^{26}	40.09
	2^{27}	75.42
	2^{28}	91.54
SHA2 chain	2^{18}	0.59
	2^{19}	1.02
	2^{20}	1.67
	2^{21}	3.11
	2^{22}	5.10
	2^{23}	10.05
	2^{24}	14.28
	2^{25}	26.84
	2^{26}	41.48
	2^{27}	67.78
	2^{28}	98.18
SHA3 chain	2^{18}	0.65
	2^{19}	1.14
	2^{20}	1.72
	2^{21}	3.31
	2^{22}	6.07
	2^{23}	11.25
	2^{24}	15.98
	2^{25}	31.72
	2^{26}	40.56
	2^{27}	81.32
	2^{28}	92.33
BTreeMap	2^{18}	0.63
	2^{19}	1.08
	2^{20}	1.78
	2^{21}	3.28
	2^{22}	5.51
	2^{23}	10.86
	2^{24}	14.53
	2^{25}	26.37
	2^{26}	42.14
	2^{27}	75.59
	2^{28}	93.34

Table 5: Jolt$_{\mathcal{TS}}$ prover peak memory usage, RV64IMAC

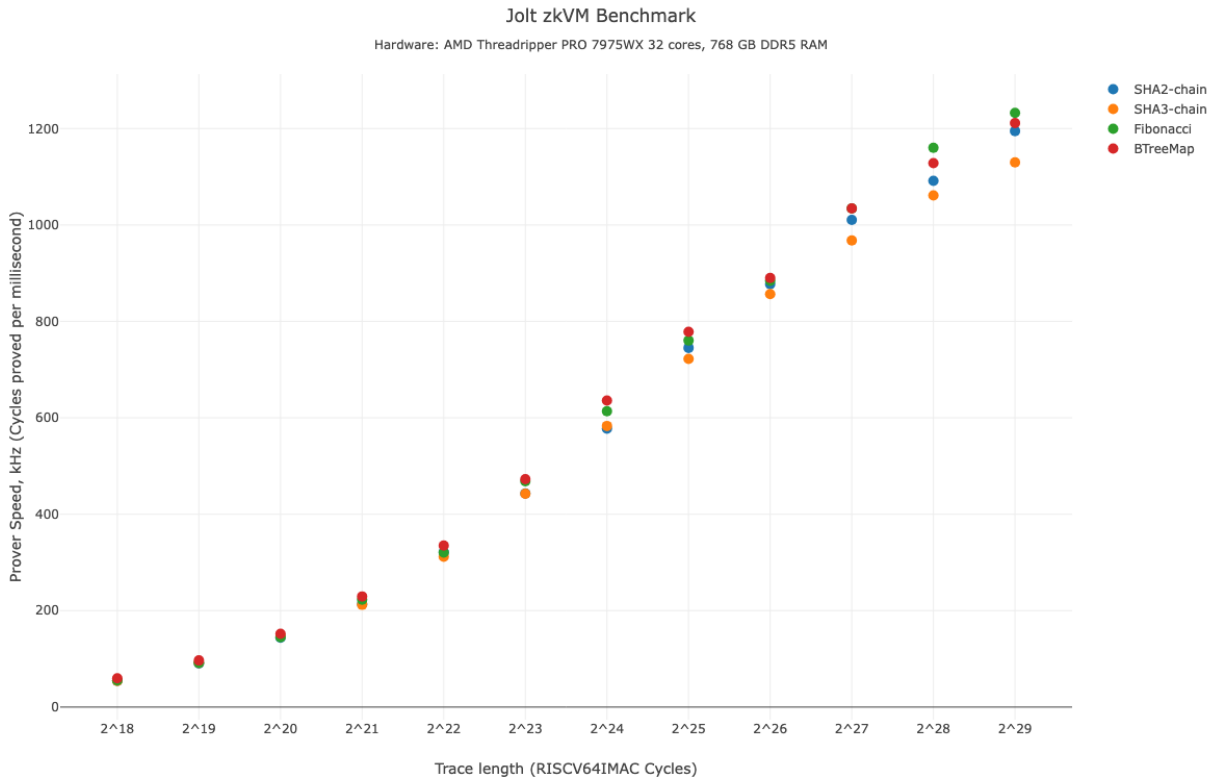


Figure 13: Jolt$_{\mathcal{TS}}$ prover speed, RV64IMAC. Benchmarked on an AMD Threadripper PRO 7975WX with 32 cores, 768 GB DDR5 RAM



Figure 14: Jolt$_{\mathcal{TS}}$ proof size, RV64IMAC



Figure 15: Jolt$_{\mathcal{TS}}$ prover peak memory usage, RV64IMAC

A Faster SNARKs for non-uniform computation

This appendix describes SpeedySpartan and Spartan++, fast SNARKs for non-uniform circuits. A distinguishing aspect of both is that they incur minimal commitment costs: the prover simply commits to its witness, and the rest of the prover's work is field operations in the sum-check protocol and the cost to prove a polynomial evaluation (which, for appropriate polynomial commitment schemes, can be sub-linear in the size of the circuit). This improves significantly on Spartan and BabySpartan (and other prior SNARKs) for non-uniform circuits.

A.1 Overview

In this overview, let us consider the simple and clean setting of proving satisfiability of an arithmetic circuit consisting of fan-in two addition and multiplication gates over field $\mathbb{F}$. This treatment extends easily to more general constraint systems.

Suppose that the circuit has size T and consider two $\log(T)$-variate multilinear polynomials $\widetilde{z}_A$ and $\widetilde{z}_B$ that respectively capture the left and right inputs to each gate of the circuit and a third polynomial $\widetilde{z}$ that captures the output of each gate. The first two polynomials, $\widetilde{z}_A$ and $\widetilde{z}_B$ are *virtual polynomials*: they are not explicitly committed/sent by the SpeedySpartan prover. The third polynomial $\widetilde{z}$ *is* explicitly committed/sent by the prover.

Overview of SpeedySpartan. SpeedySpartan applies the sum-check protocol once to confirm that $\widetilde{z}$ respects the operation of its gate. That is, if $j \in \{0,1\}^{\log(T)}$ is the label of a multiplication gate, then the sum-check confirms that $\widetilde{z}(j) = \widetilde{z}_A(j) \cdot \widetilde{z}_B(j)$, and similarly if j is an addition gate then the sum-check confirms that $\widetilde{z}(j) = \widetilde{z}_A(j) + \widetilde{z}_B(j)$.

At the end of this sum-check, the verifier has to evaluate each of $\widetilde{z}_A$, and $\widetilde{z}_B$ and $\widetilde{z}$ at a random point. Since $\widetilde{z}$ was explicitly committed by the prover, the evaluation of $\widetilde{z}$ can be obtained via the evaluation argument of the polynomial commitment scheme. Since $\widetilde{z}_A$ and $\widetilde{z}_B$ are virtual polynomials the verifier cannot obtain their evaluations directly. SpeedySpartan applies the sum-check protocol a second time to reduce the task of computing the necessary evaluations of $\widetilde{z}_A$ and $\widetilde{z}_B$ to the task of evaluating $\widetilde{z}$ at single point. In particular, this second sum-check invocation is simply the Shout read-checking sum-check (see Figure 7). That is, the left input $\widetilde{z}_A(j)$ to gate j is equal to the value *output* by some other gate j_L, and similarly the right input $\widetilde{z}_B(j)$ to j is the output of some gate j_R. This is the same as a *lookup* into the table whose entries are given by $\widetilde{z}$. The wires of the circuit (i.e., the identities of the left and right inputs to each gate) determine the addresses that are looked up. In the context of SpeedySpartan, these addresses depend only on the circuit wires and hence can be committed (as in Shout) by an honest party in pre-processing. This is a complete overview of the SpeedySpartan protocol.

Overview of Spartan++. Recall that Spartan is a SNARK for R1CS and that SuperSpartan extends it to CCS [STW23], a generalization of Plonkish, R1CS, and algebraic intermediate representations (AIR). Spartan++ is essentially just SuperSpartan [STW23], but with an improved sparse polynomial commitment scheme used to commit to "constraint matrices". Specifically, if the prover is claiming to know a solution vector z to an R1CS instance

$$Az \circ Bz = Cz,$$

(where, unlike in SpeedySpartan, A, B, and C, may have many non-zeros per row), then Spartan has the multilinear extension polynomials $\widetilde{A}$, $\widetilde{B}$, and $\widetilde{C}$ committed in pre-processing via Spark. In the online phase of Spartan, the sum-check protocol is applied several times, and at the end of these invocations, the Spartan verifier needs to evaluate $\widetilde{A}$, $\widetilde{B}$, and $\widetilde{C}$ at a random point r (in the case of CCS, the verifier needs evaluations of $\widetilde{M_0}, \ldots, \widetilde{M_{t-1}}$ for a chosen value of t). This evaluation in Spartan is provided by the prover along with a Spark evaluation proof. Spark uses the Lasso lookup argument internally, to perform lookups into a structured table storing all multilinear Lagrange basis polynomials evaluated at r. Spartan++ is the same, except we give a much faster sparse polynomial commitment scheme, based on Shout rather than Lasso. We call this scheme Spark++. Unlike Lasso, Shout allows the values returned by the lookups to be virtual, which eliminates the primary bottleneck for the Spark prover (namely, committing to the values returned by the lookups; these values are random since r is random, and hence very slow to commit to).

A.2 Details of SpeedySpartan

A.2.1 Detailed protocol description

Our presentation below borrows text from BabySpartan [ST23]. Like BabySpartan [ST23], our focus here is on Plonkish constraint systems. For simplicity and easier adoption, we focus on degree-2 Plonkish, but the protocol readily applies to arbitrary degree Plonkish with custom gates. In particular, we use the definition from Plonk [GWC19, §6], which we state below.

Definition A.1 (Plonkish)**.** Consider a finite field $\mathbb{F}$. Let the public parameters consist of size bounds $m, n, \ell \in \mathbb{N}$, with $\ell < n$. The Plonkish structure consists of: (1) vectors $q_m, q_l, q_r, q_o, q_c \in \mathbb{F}^m$, and (2) a set of vectors $(a, b, c) \in [n]^m$ specifying the left, right, and output indices in a vector containing public inputs and outputs (IO) and the witness. An instance $x \in \mathbb{F}^\ell$ consists of public IO and is satisfied by a witness $w \in \mathbb{F}^{n-\ell}$ if the following holds for all $i \in [m]$. $(q_l)_i \cdot z_{a_i} + (q_r)_i \cdot z_{b_i} + (q_o)_i \cdot z_{c_i} + (q_m)_i \cdot (z_{a_i} \cdot z_{b_i}) + (q_c)_i = 0$, where $z = (w, x)$ and z_{a_i} denotes the entry of z at index provided by the ith entry of vector a.

Note that Plonkish is a special case of CCS [STW23]. Observe that the vector z can be viewed as a lookup table and its entries are "looked up" at indices provided by vectors a, b, and c. In the language of CCS, we can represent these indices as three sparse matrices. Let $A, B, C \in \mathbb{F}^{m \times n}$ denote those matrices. In particular, for all $i \in [m]$, the ith row of A (similarly B and C) is the unit vector that encodes the position specified by the ith entry of vector a (similarly b and c). In other words, each row of A, B, C contains a single non-zero entry of 1 so that the matrix-vector products Az, Bz, Cz "lookup" the correct entry of z. Thus, we can express the Plonkish satisfiability check as follows:

$$q_l \circ Az + q_r \circ Bz + q_o \circ Cz + q_m \circ Az \circ Bz + q_c = 0$$

SpeedySpartan can use any multilinear polynomial commitment scheme. In a preprocessing phase, the verifier (or other honest party) commits to polynomials that encode the circuit structure (i.e., sparse matrices A, B, C) using a multilinear polynomial commitment scheme. In the online phase, the prover begins by committing to its purported witness $\widetilde{w}$. The prover then reduces the circuit satisfiability check to claims about evaluations of $\widetilde{w}$ and polynomials committed in the preprocessing phase. Finally, the prover uses the polynomial evaluation argument of the polynomial commitment scheme to prove those claimed evaluations.

Preprocessing phase. Let m, n be powers of two, $s = \log m$, and $\ell = n/2$. The input to this step includes sparse matrices $A, B, C \in \mathbb{F}^{m \times n}$, where each row of the sparse matrix is a unit vector, and vectors $q_l, q_r, q_o, q_m, q_c \in \mathbb{F}^m$. The preprocessing phase outputs a set of polynomials for which the verifier has a query access.

- Output $(\widetilde{q_l}, \widetilde{q_r}, \widetilde{q_o}, \widetilde{q_m}, \widetilde{q_c}, \widetilde{\mathsf{addr}}_A, \widetilde{\mathsf{addr}}_B, \widetilde{\mathsf{addr}}_C)$, where for $M \in \{A, B, C\}$, addr_M is the vector of size $m \cdot d \cdot n^{1/d}$ listing the d-dimensional one-hot encodings of addresses provided in M per Section 4.5 (i.e., viewing each row of M as the one-hot encoding of an address into a lookup table of size n). $\widetilde{\mathsf{addr}}_M$ is sent as d separate polynomials $\widetilde{\mathsf{addr}}_{M,1}, \ldots, \widetilde{\mathsf{addr}}_{M,d}$ each of size $m \cdot n^{1/d}$.

Online phase. The prover is given as input a purported witness vector $w \in \mathbb{F}^{n-\ell}$ and a public IO vector $x \in \mathbb{F}^\ell$. The verifier is given as input $x \in \mathbb{F}^\ell$.

1. $\mathcal{P} \to \mathcal{V}$: A purported witness polynomial $\widetilde{w}$.
2. $\mathcal{V} \to \mathcal{P}$: pick $\tau \in \mathbb{F}^s$ at random and send it to $\mathcal{P}$.
3. $\mathcal{V} \leftrightarrow \mathcal{P}$: Let f be defined as per Equation (80) (degree-2 Plonkish) or (81) (arithmetic circuits). $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to confirm that

$$0 = \sum_{x \in \{0,1\}^s} \mathsf{eq}(\tau, x) \cdot f(x).$$

   At the end of the sum-check protocol, the verifier is left with checking whether

$$c = \mathsf{eq}(\tau, r) \cdot f(r),$$

   where c and r are determined over the course of the sum-check protocol.

4. $\mathcal{P} \to \mathcal{V}$: Let $z = (w, x)$, and $z_A = A \cdot z$, $z_B = B \cdot z$ and $z_C = C \cdot z$. Send v_A, v_B, v_C defined as follows:

$$v_A \leftarrow \widetilde{z}_A(r),\ v_B \leftarrow \widetilde{z}_B(r),\ v_C \leftarrow \widetilde{z}_C(r).$$

5. $\mathcal{V}$: Let $e = \mathsf{eq}(\tau, r)$, and obtain the following values by querying polynomials committed during pre-processing: $v_l \leftarrow \widetilde{q_l}(r), v_r \leftarrow \widetilde{q_r}(r), v_o \leftarrow \widetilde{q_o}(r), v_m \leftarrow \widetilde{q_m}(r), v_c \leftarrow \widetilde{q_c}(r)$. Reject if

$$c \neq e \cdot (v_l \cdot v_A + v_r \cdot v_B + v_o \cdot v_C + v_m \cdot v_A \cdot v_B + v_c).$$

6. $\mathcal{V} \leftrightarrow \mathcal{P}$: Apply the core Shout PIOP with addresses from the preprocessing phase $(\mathsf{addr}_A, \mathsf{addr}_B, \mathsf{addr}_C)$ to validate claims v_A, v_B, v_C. For each $M \in \{A, B, C\}$, use the two-stage protocol in Figure 7 with $T = m$, $K = n$, lookup-table polynomial $\widetilde{z}$, and address factors $\widetilde{\mathsf{addr}}_{M,1}, \ldots, \widetilde{\mathsf{addr}}_{M,d}$. Its read-checking identity is

$$v_M = \sum_{k \in \{0,1\}^{\log n}} \widetilde{M}(r, k) \cdot \widetilde{z}(k).$$

   The subsequent address-virtualization sum-check reduces the required evaluation of $\widetilde{M}$ to evaluations of the preprocessed address factors. Obtain the required evaluation of $\widetilde{z}$ from $\widetilde{w}$ and the public IO.

Figure 16: The description of SpeedySpartan PIOP. Let d denote the parameter chosen for Shout.

Figure 16 depicts the SpeedySpartan PIOP in full. Below, we describe via prose the resulting SNARK one obtains by combining the PIOP with a multilinear polynomial commitment scheme.

Preprocessing phase. In a preprocessing step, the verifier (or another trusted party) computes commitments to the multilinear extensions of the matrices $A, B, C \in \mathbb{F}^{m \times n}$. Each row in these matrices is a unit vector, i.e., each row contains a single entry equal to 1 (the rest are zeros). In other words, each matrix provides the one-hot encodings of m addresses into a table of size n.

For a chosen parameter $d \geq 2$, the commitment to each sparse matrix encodes m lookup addresses using d-dimensional one-hot encodings (Section 4.5). Since this commitment is created by the verifier (or another trusted party), there is no need to ensure, as part of Shout, that the underlying vectors are one-hot encodings. That is, Booleanity-checking and additional checks from Figure 8 are not needed. This is analogous to how in Spartan's preprocessing step [Set20], there is no need to prove the correctness of timestamps in its use of offline memory checking procedure.

Online phase. Without loss of generality, let m and n be powers of 2 and that $\ell = n/2$. Let $s = \log m, s' = \log n$. For a purported witness $w \in \mathbb{F}^{n-\ell}$ and public IO $x \in \mathbb{F}^{\ell}$, let $z = (w, x) \in \mathbb{F}^n$. We can view the vector z as a function with the signature $\{0,1\}^{s'} \to \mathbb{F}$. Similarly, we can view the sparse matrices A, B, C as functions with the following signature: $\{0,1\}^s \times \{0,1\}^{s'} \to \mathbb{F}$. Crucially, in our target setting, each row of these sparse matrices is a unit vector (i.e., there is a single non-zero entry of 1 and the rest are zeros).

For all $x \in \{0,1\}^s$ and $M \in \{A, B, C\}$, let

$$\widetilde{z_M}(x) = \left(\sum_{y \in \{0,1\}^{s'}} \widetilde{M}(x, y) \cdot \widetilde{z}(y) \right)$$

Also, let

$$f(x) = (\widetilde{q_l}(x) \cdot \widetilde{z}_A(x) + \widetilde{q_r}(x) \cdot \widetilde{z}_B(x) + \widetilde{q_o}(x) \cdot \widetilde{z_C}(x) + \widetilde{q_m}(x) \cdot \widetilde{z}_A(x) \cdot \widetilde{z}_B(x) + \widetilde{q_c}(x)) \tag{80}$$

The prover begins by sending a commitment to its purported witness $\widetilde{w}$. To check satisfiability, the verifier selects a random $\tau \in \mathbb{F}^s$, and applies the sum-check protocol to the polynomial

$$g(x) = \widetilde{\mathsf{eq}}(\tau, x) \cdot f(x),$$

using it to confirm that

$$0 = \sum_{x \in \{0,1\}^s} g(x).$$

If z satisfies the constraint system then this equality is guaranteed to hold, and if z does not satisfy the constraint system then this equality will fail to hold with probability at least $1 - O(s)/|\mathbb{F}|$.We call this the "Spartan-sum-check" since this sum-check invocation is exactly how Spartan begins [Set20].

At the end of this invocation of the sum-check protocol, the verifier has to evaluate $\widetilde{z}_A(r)$, $\widetilde{z}_B(r)$ and $\widetilde{z_C}(r)$, where $r \in \mathbb{F}^s$ is a random point chosen over the course of the sum-check protocol. Hence,

the invocation of the sum-check protocol reduced the task of checking that z satisfies the constraint system, to the task of evaluating the multilinear extensions of z_A, z_B, and z_C each at a random point $r \in \mathbb{F}^s$.

Recall that by design, each row of A, B, C has a single non-zero entry (i.e., viewing z as a lookup table, each row of A, B, and C is the one-hot encoding of some address of z). So, we invoke the Shout protocol to prove the desired evaluations of $\widetilde{z}_A, \widetilde{z}_B$, and $\widetilde{z_C}$ at r. In more detail, by using the Shout read-checking sum-check (Equation (15)), the prover reduces the claims about $\widetilde{z}_A(r), \widetilde{z}_B(r)$, and $\widetilde{z_C}(r)$ into evaluations claims about the lookup table $\widetilde{z}$ and polynomials committed in the preprocessing phase.[13] The prover then proves the evaluations of these committed polynomials using a polynomial evaluation argument.

Note that multiple invocations of the sum-check protocol and polynomial evaluation arguments, arising from running Shout for each of $\widetilde{z}_A, \widetilde{z}_B, \widetilde{z_C}$, can be batched together using standard techniques (see Sections 6.2.1 and 6.1.1 for details).

A.2.2 The special case of arithmetic circuit satisfiability

Arithmetic circuit satisfiability trivially reduces to the following special case of Plonkish:

$$f(x) = (1 - \widetilde{q}_m(x)) \cdot (\widetilde{z}_A(x) + \widetilde{z}_B(x)) + \widetilde{q}_m(x) \cdot \widetilde{z}_A(x) \cdot \widetilde{z}_B(x) - \widetilde{z}_C(x) \tag{81}$$

Here, $q_m(x) = 1$ if gate x is a multiplication gate, and equals 0 otherwise. The value $z_C(x)$ is the output of gate x. For the cost analysis below, gate outputs occupy a contiguous, power-of-two-sized block of z, so evaluations of $\widetilde{z}_C$ come directly from the witness commitment and public inputs; only $\widetilde{z}_A$ and $\widetilde{z}_B$ require lookups. For simplicity and ease of comparing to prior works, we focus on this special case when assessing the costs of SpeedySpartan.

A.2.3 SpeedySpartan: Analysis of costs

Polynomial evaluation proofs and the choice of commitment scheme. SpeedySpartan's speedups relative to prior work are largest when polynomial evaluation proofs are not a major contributor to prover time. There are several reasons this could be the case. One is that SpeedySpartan can be instantiated with a commitment scheme that comes with very fast evaluation proofs. Another, discussed in Appendix A.2.5, is that in applications where many different instances of (one or more) constraint systems arise, folding techniques can ensure only a single evaluation proof needs to be produced across all instances.

We now discuss attractive choices of polynomial commitment schemes that lead to fast SpeedySpartan prover even in settings that are not amenable to folding techniques.

Commitment schemes with fast evaluation proof computation. For fast evaluation proof computation, the most extreme choice is Hyrax [WTS+18]. In Hyrax, the prover simply performs a linear number of field multiplications, which are required anyway to compute the requested evaluation. Moreover, if the committed polynomial is sparse (as is the case in SpeedySpartan), the

[13] More precisely, if $z = (w, x)$ for the public vector x, then the prover commits to $\widetilde{w}$, and $\widetilde{z}(r)$ can be evaluated efficiently with one evaluation query to $\widetilde{w}$. See [Set20, STW23] for details.

Hyrax prover's field work grows linearly with the sparsity of the polynomial (i.e., the number of non-zeros). More precisely, if M out of N entries of the committed vector are non-zero, the Hyrax prover's work during evaluation proof computation is $O(M + \sqrt{N})$ field operations.

This is a perfect fit to ensure a very fast SpeedySpartan prover. On top of this, Hyrax is transparent and has sublinear (square-root) size commitment key. The downside of Hyrax is that its commitment size and evaluation proof size are fairly large: they are each $\sqrt{N}$ field and group elements where N is the size of the committed polynomial. In summary, Hyrax is an attractive choice for SpeedySpartan when prover time is a top priority and sublinear-but-large verifier costs are acceptable.

Another commitment scheme with fast evaluation proof computation is Dory [Lee21]. Indeed, Dory can be seen as a refinement of both Hyrax and Bulletproofs/IPA [BBB$^+$18, BCC$^+$16], and inherits the fast evaluation proofs from Hyrax.[14] Dory evaluation proofs entail the same field work as in Hyrax evaluation proof computation, plus some extra cryptographic work that is asymptotically sublinear. This cryptographic work consists of roughly $16\sqrt{N}$ scalar multiplications in $\mathbb{G}_1$ or $\mathbb{G}_2$ of a pairing-friendly group, and 6 multi-pairings of size $\sqrt{N}/2^i$ for each $i = 1, \ldots, \log N$.

Because pairings are concretely expensive (one pairing is equivalent in cost to about 4000 group operations), these evaluation proofs can be the concrete prover bottleneck (i.e., more expensive than committing or sum-check-proving in SpeedySpartan) unless N is not too small. This effect is somewhat amplified in SpeedySpartan, become some polynomials arising in the protocol (which are committed in pre-processing) are of slightly superlinear size $m \cdot n^{1/d}$. These polynomials have only m non-zero coefficients, so they do not require superlinear time to commit to even in pre-processing, but this does mean that the $O(\sqrt{N})$ cryptographic costs arising in Dory evaluation proof computation apply with $N = m \cdot n^{1/d}$. So while $O(\sqrt{N}) = O(\sqrt{m} \cdot n^{1/2d})$ is still sub-linear in m and n (and hence evaluation proof computation is asymptotically a low-order prover cost), it can be a concrete SpeedySpartan prover bottleneck if the circuit is small *and* the parameter d is small. If this is the case, per Section 6.1.1 one can always further lower the cost of Dory evaluation proof computation by a factor of $\log T$ with a constant-factor increase in verifier costs, by increasing the commitment size from one target group element to $\log T$ group elements.

Calculations with our analytic cost model (Section 6.1.2) show that Dory evaluation proofs represent a small fraction of total prover time (at most 30%) so long as m and n are at least 2^{26} and $d \geq 3$.

Challenges with HyperKZG or Bulletproofs. HyperKZG [ZSC24] and Bulletproofs/IPA [BCC$^+$16, BBB$^+$18] have expensive evaluation proof computation, as evaluation proofs require the prover to commit to a linear number of random field elements. As discussed above, this issue is amplified in the context of SpeedySpartan because some committed polynomials have size $m \cdot n^{1/d}$ (though they have sparsity only m). This means the commitment key size when using HyperKZG or Bulletproofs is slightly superlinear $m \cdot n^{1/d}$, and the evaluation proofs require committing to $O(m \cdot n^{1/d})$ random field elements. On top of that, unlike Hyrax and Dory, the field work required to compute evaluation proofs grows with the size $m \cdot n^{1/d}$ of the committed polynomial and not only its sparsity m. For these reasons, HyperKZG and Bulletproofs appear to be poor fits for

[14] Essentially, the Dory committer uses pairings to *commit to a Hyrax commitment*, thereby obviating the $\sqrt{n}$ commitment size of Hyrax. For evaluation proofs, the Dory prover uses a protocol reminiscent of Bulletproofs/IPA to succinctly prove that it (a) knows a Hyrax commitment that opens the Dory commitment and (b) knows a valid Hyrax evaluation proof. It thereby obviates the $\sqrt{n}$ evaluation proof size of Hyrax.

SpeedySpartan.

Detailed costs for SpeedySpartan. As a running example throughout this section, we consider the case of $d = 2$ and Hyrax as the choice of polynomial commitment scheme. This leads to a very fast SpeedySpartan prover and achieves non-trivial verifier costs of $O(m^{1/2}n^{1/4})$. In this running example, we set $m = n = 2^{24}$ for concreteness. We also assume that the entries of the witness vector z are all "small" (see Section 2.1) and hence fast to commit to. This is indeed the case in many applications (where the entries of z often represent bit or byte decompositions, or 32-bit values).

Size of proving key. When Hyrax (or Dory) is used in SpeedySpartan, the (transparently generated) commitment key consists of $\sqrt{m \cdot n^{1/d}}$ group elements. For example, if $m = n = 2^{24}$ and $d = 2$, this is only 2^{18} group elements, translating to MBs of space.

Pre-processing. Pre-processing in SpeedySpartan requires committing to $3dm$ unit vectors each of length $n^{1/d}$, plus 5 vectors of length m. With Hyrax, committing to the unit vectors requires dm group operations, while committing to the other 5 vectors (assuming they have small entries per Section 2.1) requires roughly $5m$ group operations.

Online phase prover costs. In SpeedySpartan, during the online phase, the prover only commits to the witness vector z and nothing else (more precisely, the prover commits to the MLE of its purported witness vector w of length $n - \ell$). Thus, if the witness vector contains "small" field elements, then the prover only commits to small field elements.

Then the SpeedySpartan prover applies the sum-check protocol several times. Once is the Spartan sum-check, and the other is Shout's read-checking sum-check, applied twice in parallel (once for each matrix A and B). The Spartan sum-check can be implemented with $9m$ field multiplications. This is the cost of the standard linear-time sum-check proving algorithm (Section 6.2) combined with optimizations of Dao and Thaler [DT24] and Gruen [Gru24] (see Section 6.5). As a modest additional optimization, we also exploit here that $q_m(x) \in \{0, 1\}$ for all $x \in \{0, 1\}^{\log m}$. This ensures that binding the array storing q_m evaluations produces only $2^{O(i)}$ distinct values in round i, which ensures that the prover only incurs $2^{O(i)}$ multiplications to bind the array in that round.

Applying Theorem 7.2 with $T = m$ and $K = n$, each invocation of Shout costs at most $(d^2 + 1)m + 5n + o(m)$ field multiplications when $n^{1/d} = o(m)$. Thus the two lookups and the Spartan sum-check together cost

$$(2d^2 + 11)m + 10n + o(m).$$

For $d = 2$ and $m = n$, this is $29m + o(m)$ field multiplications. These bounds use the two-stage Shout protocol of Figure 7.

The SpeedySpartan prover must also provide an evaluation for each committed polynomial. Per the discussion in Section A.2.3, we focus on settings where this is not a significant contributor to prover time.

	Plonk (small proof)	Plonk (fast prover)	BabySpartan	SpeedySpartan ($d = 2$)	SpeedySpartan ($d = 3$)
Committed field elements (excluding pre-processing)	$8m$ random $3m$ small	$6m$ random $3m$ small	$3m + n$ small	n small	n small
Field multiplications	$54m \log m$	$54m \log m$	$33m + 24n$	$19m + 10n$	$29m + 10n$
Total field multiplications (approximate)	$54m \log m + 546m$	$54m \log m + 414m$	$51m + 30n$	$19m + 16n$	$29m + 16n$
Total field multiplications (for $m = n = 2^{24}$)	$1842 \cdot 2^{24}$	$1710 \cdot 2^{24}$	$81 \cdot 2^{24}$	$35 \cdot 2^{24}$	$45 \cdot 2^{24}$

Figure 17: Prover costs for SpeedySpartan compared with two versions of Plonk [GWC19] and BabySpartan when applied to arithmetic circuits of fan-in two (n is the total witness size and m is the number of constraints (i.e., $n - m$ is the number of auxiliary, unconstrained inputs). For simplicity we assume $m = n$ when stating Plonk's costs. When translating committed field elements to field multiplications, we treat each small committed field element as costing one group operation, each random committed field element as costing 11 group operations, and each group operation as costing 6 field operations. See Section 6.1.2 for justification of these translations. The qualitative comparison between protocols is insensitive to these precise choices.

Verifier costs. The proof size is $O(d \log(n + m))$ field elements, plus the polynomial evaluation proof provided for each committed polynomial. If desired, standard techniques can batch these evaluation proofs at minimal cost to the prover, ensuring that only a single evaluation proof is provided and verified (see for example [GLH+24, Section 5]). The verifier's runtime is $O(d \log(n+m))$ field operations plus the cost of checking the polynomial evaluation proof(s).

A.2.4 Comparison of SpeedySpartan with Plonk and BabySpartan

Figure 17 reports quantitative prover costs for Plonk, BabySpartan, and SpeedySpartan. The reported costs apply when these protocols are combined with any elliptic-curve commitment scheme like Hyrax and Dory. The reported costs also apply to HyperKZG and Bulletproofs/IPA, but these commitment schemes are poor fits for SpeedySpartan due to high evaluation proof computation time and commitment key size (§A.2.3).

We use the reported costs from Plonk [GWC19]. We omit other natural baselines such as HyperPlonk [CBBZ23b] as BabySpartan is strictly faster than these baselines. For all schemes, we ignore the cost of evaluation proof computation. This is justified for SpeedySpartan by the discussion above (i.e., we are focused either on choices of commitment schemes for which this is not a significant prover cost, or settings where this cost is insignificant for other reasons).

The comparison can be summarized as follows. With $d = 2$, SpeedySpartan improves over the field multiplications performed by the Plonk prover by roughly 50×, and improves the commitment costs by about 75×. For commitment costs, the large improvement comes both because SpeedySpartan commits to about 9× fewer field elements than Plonk, and because all of the values committed by the SpeedySpartan prover are small, while 2/3 of the values committed by the Plonk prover are random (recall from Sections 2.1 and 6.1.2 that random values take an order of magnitude more time to commit to than small ones). SpeedySpartan with $d = 2$ improves over the commitment cost of BabySpartan by about 4×, and over the field work of BabySpartan by about 2×.

With $d = 3$, the field work of SpeedySpartan increases about 30% relative to $d = 2$, and the online

commitment costs remain the same. The reason to consider higher values of d is that commitment key size, evaluation proof size, evaluation proof computation time (in the case of Dory), and pre-processing time all decrease as d grows.

A.2.5 Obtaining a folding scheme from SpeedySpartan

Like how an "early stopping" (Super)Spartan leads to HyperNova [KS24a], SpeedySpartan leads to a folding scheme for Plonkish with attractive characteristics: the folding scheme can fold multiple Plonkish instances defined with respect to Plonkish constraint systems that do *not* necessarily share the same circuit structure. Compared to a naive solution that achieves this property, a SpeedySpartan-based solution has far better costs. In fact, the costs are similar to HyperNova: the prover simply commits to its witness and incurs some field operations in the sum-check protocol. The verifier circuit size is logarithmic in the circuit sizes, which is similar to HyperNova's. We leave it to the future work to improve the size of the verifier circuit with ideas from NeutronNova to use an early-stopping sum-check protocol [KS24b].

A.3 Details of Spartan++

A.3.1 Spark++: A faster commitment scheme for sparse polynomials

Commitment phase. Let p be a T-sparse multilinear polynomial to be committed. This means p is a polynomial in ℓ variables, so p has size 2^ℓ,but there are only $T = o(2^\ell)$ values of $x \in \{0,1\}^\ell$ such that $p(x) \neq 0$. Let $S = \{x \in \{0,1\}^\ell : p(x) \neq 0\}$. For simplicity, let us assume that $p(x) \in \{0,1\}$ for all $x \in \{0,1\}^\ell$; the cost comparison below considers this binary case. For general sparse polynomials, preprocessing also commits to the nonzero values, as described below.

Consider a memory of size $K = 2^\ell$. Associate each index of memory with an input $x \in \{0,1\}^\ell$. For parameter $d \geq 1$, the commitment to p consists of commitments to the d-dimensional one-hot encoding of each $x \in S$. That is, the commitment to p consists of d committed polynomials $\widetilde{\mathsf{ra}}_1, \ldots, \widetilde{\mathsf{ra}}_d$, where for each $j \in \{0,1\}^{\log T}$, $\widetilde{\mathsf{ra}}_1(\cdot, j), \ldots, \widetilde{\mathsf{ra}}_d(\cdot, j)$ provides the d-dimensional one-hot encoding of the j'th element of S.

In the application to Spartan++, an honest party constructs these commitments during preprocessing from the distinct elements of S. Thus the online prover need not prove correctness of the one-hot encodings or of the support list.

Evaluation phase. Suppose the verifier requests $p(r)$. By multilinear Lagrange interpolation and the assumption that $p(x) \in \{0,1\}$ for all $x \in \{0,1\}^\ell$, we can write:

$$p(r) = \sum_{k_1,\ldots,k_d \in \{0,1\}^{\ell/d}, j \in \{0,1\}^{\log T}} \left(\prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(k_i, j) \right) \cdot \widetilde{\mathsf{eq}}(k, r).$$

In other words, if we consider the lookup table of size $K = 2^\ell$ whose k'th entry (for $k \in \{0,1\}^\ell$) is $\widetilde{\mathsf{eq}}(k, r)$, and we consider S to be a list of T addresses for lookups into this table, then $p(r)$ is simply

$$\sum_{j \in \{0,1\}^{\log T}} \widetilde{\mathsf{rv}}(j), \tag{82}$$

where $\widetilde{\mathsf{rv}}$ is the MLE of the values returned by the lookups.

Based on Equation (82), to prove the correct value of $p(r)$, the prover has two options. One is to apply the sum-check protocol to compute Expression (82), at the end of which the verifier has to evaluate $\widetilde{\mathsf{rv}}(r')$ for a random $r' \in \mathbb{F}^{\log T}$. The verifier can then accomplish this using Shout's read-checking sum-check applied to the table above. When the field has characteristic other than two, the other option is to observe that Expression (82) equals $T \cdot \widetilde{\mathsf{rv}}(2^{-1}, \ldots, 2^{-1})$ (this observation also arose surrounding Equation (11) in Section 5.3), and apply Shout's read-checking sum-check to compute this particular evaluation of $\widetilde{\mathsf{rv}}$.

Note that the table to which Shout is applied above is MLE-structured. Indeed, for this table $\widetilde{\mathsf{Val}}(k) = \widetilde{\mathsf{eq}}(k, r)$, which can clearly be evaluated at any point $k \in \mathbb{F}^\ell$ in $O(\log K) = O(\ell)$ time, via Equation (3).

For this particular lookup table, the two reductions can also be combined into a direct sum-check:

$$p(r) = \sum_{j \in \{0,1\}^{\log T}} \prod_{i=1}^{d} \widetilde{\mathsf{ra}}_i(r_i, j),$$

where $r = (r_1, \ldots, r_d)$ and each r_i contains ℓ/d coordinates. On Boolean j, the product is exactly $\widetilde{\mathsf{eq}}(k_j, r)$ for the corresponding support address k_j. This sum-check ends with one query to each committed $\widetilde{\mathsf{ra}}_i$.

General nonzero values. For arbitrary p, enumerate its support as $k_1, \ldots, k_T$ and let $v(j) = p(k_j)$. Preprocessing commits to the multilinear extension of v in addition to the address encodings. Then

$$p(r) = \sum_{j \in \{0,1\}^{\log T}} \widetilde{v}(j) \cdot \widetilde{\mathsf{rv}}(j),$$

where the lookups still return $\widetilde{\mathsf{eq}}(k_j, r)$. A sum-check reduces this weighted sum to evaluations of $\widetilde{v}$ and $\widetilde{\mathsf{rv}}$; the first is obtained from its preprocessing commitment and the second through Shout. Padding uses zero values in v. This handles arbitrary sparse constraint matrices without additional online commitments.

Costs. Committing to the sparse polynomial p entails committing to d vectors, each of length $TK^{1/d}$, with T entries equal to 1 and the rest equal to 0. The evaluation phase involves applying Shout to T lookups into a table of size $K = 2^\ell$. This is a canonical table to which the sparse-dense sum-check protocol applies (Section 8.1), enabling the Shout prover to run in time $O(CT)$ where C is such that $K \leq T^C$.

A.3.2 Details of Spartan++

Recall that Spartan++ is essentially just SuperSpartan, but with the Spark sparse polynomial commitment scheme replaced with Spark++. Here, we spell out the details. For ease of exposition, we focus on R1CS (a special case of CCS) and Spartan (a special case of SuperSpartan).

A special case of R1CS capturing arithmetic circuits. In Spartan++ the prover only commits to the values (i.e., outputs) w of multiplication gates. If there are M multiplication gates, then $w \in \mathbb{F}^M$, and the prover begins the protocol by committing to the multilinear extension $\widetilde{w}$.

Let $z = (w, x) \in \mathbb{F}^n$ where $x \in \mathbb{F}^{n-M}$ is public input. Ensuring that w indeed contains the correct value of every multiplication gate entails proving that z satisfies M rank-one constraints. Specifically, if a_i and b_i are the left and right inputs to multiplication gate i, then the i'th constraint is simply

$$a_i \cdot b_i = w_i.$$

Let us express this constraint system as $Az \circ Bz = w$ where A and B are appropriate matrices in $\mathbb{F}^{M \times n}$. Note that, unlike in SpeedySpartan, any particular row of A and B may have many non-zero entries (addition gates in the circuit lead to non-zero entries in A and B, but addition gates do not increase the number of rows). The protocol supports arbitrary field coefficients in these matrices via the weighted version of Spark++. For the concrete cost comparison below, we restrict to matrices whose nonzero entries are all 1; eliminating addition gates does not guarantee this property for every arithmetic circuit.

The Spartan++ protocol. In pre-processing, an honest party commits to $\widetilde{A}$ and $\widetilde{B}$ using Spark++.

In the online phase, as in Spartan, the prover first applies the standard zero-check PIOP (Section 4.4) to confirm that all constraints are satisfied. That is, letting $a = Az$ and $b = Bz$, the verifier picks a random $r \in \mathbb{F}^{\log M}$ and checks that

$$0 = \sum_{x \in \{0,1\}^{\log M}} \widetilde{\mathsf{eq}}(r, x) \cdot \left(\widetilde{a}(x) \cdot \widetilde{b}(x) - \widetilde{w}(x) \right). \tag{83}$$

At the end of the sum-check protocol, the verifier has to evaluate $\widetilde{a}(r')$, $\widetilde{b}(r')$, $\widetilde{w}(r')$ and $\widetilde{\mathsf{eq}}(r, r')$ where $r' \in \mathbb{F}^{\log M}$ is the randomness chosen round-over-round during the sum-check protocol. The evaluation $\widetilde{w}(r')$ can be obtained from the commitment to $\widetilde{w}(r')$, and $\widetilde{\mathsf{eq}}(r, r')$ can be computed in $O(\log M)$ time. But $\widetilde{a}$ and $\widetilde{b}$ are not committed—they are effectively virtual polynomials. Hence, $\widetilde{a}(r')$ and $\widetilde{b}(r')$ are computed with an additional application of the sum-check protocol, applied to compute the right hand sides of the following two expressions for $\widetilde{a}(r')$ and $\widetilde{b}(r')$:

$$\widetilde{a}(r') = \sum_{j \in \{0,1\}^{\log n}} \widetilde{A}(r', j) \cdot \widetilde{z}(j),$$

and

$$\widetilde{b}(r') = \sum_{j \in \{0,1\}^{\log n}} \widetilde{B}(r', j) \cdot \widetilde{z}(j).$$

At the end of these two instances of sum-check (which can be run in parallel and batched per Section 6.2.1), the verifier needs to evaluate $\widetilde{A}(r', r'')$, $\widetilde{B}(r', r'')$ and $\widetilde{z}(r'')$ where r'' denotes the randomness chosen over the course of the two parallel sum-check instances. The evaluation $\widetilde{z}(r'')$ is obtained from the commitment to $\widetilde{w}$ and the public input. The evaluations of $\widetilde{A}$ and $\widetilde{B}$ are obtained via Spark++.

Prover Costs. Assume for simplicity that A and B have the same number of non-zero entries and that this number is T. We count nonzero matrix entries explicitly, since eliminating addition gates can increase matrix density.

Given the vectors $a = Az$ and $b = Bz$, the first sum-check (to compute Expression (83)) costs $5M$ field operations (see [DT24] for full details), while the other two sum-check invocations (to compute $\widetilde{a}(r')$ and $\widetilde{b}(r')$) together cost $M + 5n$ multiplications. In total this is $6M + 5n$ multiplications for the prover.

It remains to account for the sparse-polynomial evaluations. Use the direct sum-check above for each of $\widetilde{A}$ and $\widetilde{B}$, with $K = Mn$. For one such evaluation, the prover prepares arrays of all $\widetilde{\mathsf{ra}}_i(r_i, j)$ using $O(dK^{1/d})$ multiplications and table lookups. Computing the claimed sum costs at most $(d-1)T$ multiplications. For this multiplication count, assume the field has characteristic greater than d. Use message evaluations at $0, \ldots, d$, generating each affine factor's values by repeated additions and recovering one message evaluation from the round claim. The standard sum-check prover for a product of d multilinear factors then costs at most d^2T multiplications: in a round with N Boolean suffixes, computing d message evaluations costs $d(d-1)N$ multiplications and binding the d arrays costs dN.

Consequently, the two sparse-polynomial evaluations cost at most

$$2(d^2 + d)T + O(d(Mn)^{1/d})$$

field multiplications. With $n \approx M$, $d = 4$, and $(Mn)^{1/4} = o(T)$, the total field-work bound is therefore

$$11M + 40T + o(T).$$

The online prover commits only to its witness. Polynomial evaluation proofs for the witness and preprocessed polynomials are additional; their costs depend on the commitment scheme, as discussed in Appendix A.2.3. The displayed bound is for the binary-matrix case. General coefficients use the weighted sum-check described above.

For comparison, the analytic cost model for Spartan is as follows. Recall that Spartan applies Spark to $\widetilde{A}$ and $\widetilde{B}$, and that Spark applies Lasso internally. In this application of Lasso, the prover commits to 2 random field elements per non-zero entry of A and B. This translates to about 22 group operations per non-zero, which is roughly equivalent to $22 \cdot 6 = 132$ field operations. On top of the commitment costs, the prover also performs 24 field operations within Lasso. So this yields 156 field operations per non-zero matrix entry. Since there are $2T$ non-zeros between the two matrices, this is about $312T$ field operations in total to implement Lasso-within-Spark-within-Spartan. When $M \leq T$ and $n \approx M$, this $312T$ term alone is about six times the $11M + 40T$ field-work bound above. This is an analytic comparison under the stated binary-matrix and evaluation-proof assumptions, rather than a measured end-to-end speedup.

References

[AB09] Sanjeev Arora and Boaz Barak. *Computational complexity: a modern approach*. Cambridge University Press, 2009.

[And17] Andrew Waterman, Krste Asanovic. The RISC-V instruction set manual. `https://riscv.org/wp-content/uploads/2017/05/riscv-spec-v2.2.pdf`, 2017.

[AST24] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, 2024.

[AW09] Scott Aaronson and Avi Wigderson. Algebrization: A new barrier in complexity theory. *ACM Transactions on Computation Theory (TOCT)*, 1(1):1–54, 2009.

[BBB+18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2018.

[BC25] Benedikt Bünz and Jessica Chen. Proofs for deep thought: Accumulation for large memories and deterministic computations. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 269–301. Springer, 2025.

[BCC+16] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, 2016.

[BCF+24] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D Rothblum, and Hadas Zeilberger. Blaze: Fast SNARKs from interleaved RAA codes. Cryptology ePrint Archive, 2024.

[BCG+18] Jonathan Bootle, Andrea Cerulli, Jens Groth, Sune Jakobsen, and Mary Maller. Arya: Nearly linear-time zero-knowledge proofs for correct program execution. In *Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, 2018.

[BCGT13] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, and Eran Tromer. Fast reductions from RAMs to delegatable succinct constraint satisfaction problems: Extended abstract. In *Proceedings of the Innovations in Theoretical Computer Science (ITCS)*, 2013.

[BEG+91] Manuel Blum, Will Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. Checking the correctness of memories. In *Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, 1991. Journal version in Algorithmica, 1994.

[BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, 2020.

[BG12] Stephanie Bayer and Jens Groth. Efficient zero-knowledge argument for correctness of a shuffle. In *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, pages 263–280, 2012.

[BSCG+13] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. SNARKs for C: Verifying program executions succinctly and in zero knowledge. In *Proceedings of the International Cryptology Conference (CRYPTO)*, August 2013.

[CBBZ23a] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, 2023.

[CBBZ23b] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, 2023.

[CTY11] Graham Cormode, Justin Thaler, and Ke Yi. Verifying computations with streaming interactive proofs. *Proc. VLDB Endow.*, 5(1):25–36, 2011.

[Dor24] Daniel Dore. TaSSLE: Lasso for the commitment-phobic. Cryptology ePrint Archive, 2024.

[DP23] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. Cryptology ePrint Archive, Paper 2023/1784, 2023. `https://eprint.iacr.org/2023/1784`.

[DP24] Benjamin E Diamond and Jim Posen. Polylogarithmic proofs for multilinears over binary towers. Cryptology ePrint Archive, 2024.

[DT24] Quang Dao and Justin Thaler. More optimizations to sum-check proving. Cryptology ePrint Archive, Paper 2024/1210, 2024.

[EFG22] Liam Eagen, Dario Fiore, and Ariel Gabizon. cq: Cached quotients for fast lookups. Cryptology ePrint Archive, 2022.

[EHB22] Youssef El Housni and Gautam Botrel. EdMSM: multi-scalar-multiplication for SNARKs and faster montgomery multiplication. *Cryptology ePrint Archive*, 2022.

[FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.

[FS01] Jun Furukawa and Kazue Sako. An efficient scheme for proving a shuffle. In *Proceedings of the International Cryptology Conference (CRYPTO)*, pages 368–387, 2001.

[GL07] Jens Groth and Steve Lu. A non-interactive shuffle with pairing based verifiability. In *Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, pages 51–67, 2007.

[GLH+24] Yanpei Guo, Xuanming Liu, Kexi Huang, Wenjie Qu, Tianyang Tao, and Jiaheng Zhang. Deepfold: Efficient multilinear polynomial commitment from reed-solomon code and its application to zero-knowledge proofs. Cryptology ePrint Archive, 2024.

[GM24] Albert Garreta and Ignacio Manzur. FLI: Folding lookup instances. Cryptology ePrint Archive, Paper 2024/1531, 2024.

[GPR21] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo–a turing-complete stark-friendly cpu architecture. Cryptology ePrint Archive, 2021.

[Gro26] Jens Groth. Permutation proofs déjà vu. `https://blog.nexus.xyz/permutation-proofs-deja-vu/`, January 2026. Nexus Blog, January 22, 2026.

[Gru24] Angus Gruen. Some improvements for the piop for zerocheck. Cryptology ePrint Archive, 2024.

[GW20a] Ariel Gabizon and Zachary Williamson. Proposal: The TurboPlonk program syntax for specifying SNARK programs, 2020.

[GW20b] Ariel Gabizon and Zachary J Williamson. plookup: A simplified polynomial protocol for lookup tables. Cryptology ePrint Archive, 2020.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. ePrint Report 2019/953, 2019.

[Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530, 2022.

[KS24a] Abhiram Kothapalli and Srinath Setty. HyperNova: Recursive arguments for customizable constraint systems. In *Annual International Cryptology Conference*, pages 345–379, 2024.

[KS24b] Abhiram Kothapalli and Srinath Setty. NeutronNova: Folding everything that reduces to zero-check. Cryptology ePrint Archive, 2024.

[KS25] Darya Kaviani and Srinath Setty. Vega: Low-latency zero-knowledge proofs over existing credentials. Cryptology ePrint Archive, Paper 2025/2094, 2025.

[KST22] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In *Proceedings of the International Cryptology Conference (CRYPTO)*, 2022.

[KT23] Tohru Kohrita and Patrick Towa. Zeromorph: Zero-knowledge multilinear-evaluation proofs from homomorphic univariate commitments. Cryptology ePrint Archive, 2023.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, pages 177–194, 2010.

[Lee21] Jonathan Lee. Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In *Theory of Cryptography Conference*, pages 1–34. Springer, 2021.

[LFKN90] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. In *Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, October 1990.

[Lip89] Richard J Lipton. *Fingerprinting sets*. Princeton University, Department of Computer Science, 1989.

[Lip90] Richard J Lipton. Efficient checking of computations. In *Annual Symposium on Theoretical Aspects of Computer Science*, pages 207–215. Springer, 1990.

[Mon85] Peter L Montgomery. Modular multiplication without trial division. *Mathematics of computation*, 44(170):519–521, 1985.

[MvOV01] Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 2001.

[NBS08] Michael Naehrig, Paulo S. L. M. Barreto, and Peter Schwabe. On compressible pairings and their computation. In *Proceedings of the International Conference on Cryptology in Africa (AFRICACRYPT)*, pages 371–388, 2008.

[Nef01] C. Andrew Neff. A verifiable secret shuffle and its application to e-voting. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, pages 116–125, 2001.

[NOZ26] Ngoc Khanh Nguyen, George O'Rourke, and Jiapeng Zhang. Hachi: Efficient lattice-based multilinear polynomial commitments over extension fields. Cryptology ePrint Archive, Paper 2026/156, 2026.

[NTZ25] Vineet Nair, Justin Thaler, and Michael Zhu. Proving CPU executions in small space. Cryptology ePrint Archive, Paper 2025/611, 2025.

[OLB24] Alex Ozdemir, Evan Laufer, and Dan Boneh. Volatile and persistent memory for zkSNARKs via algebraic interactive proofs. Cryptology ePrint Archive, Paper 2024/979, 2024.

[PH23] Shahar Papini and Ulrich Haböck. Improving logarithmic derivative lookups using GKR. Cryptology ePrint Archive, 2023.

[PK22] Jim Posen and Assimakis A Kattis. Caulk+: Table-independent lookup arguments. Cryptology ePrint Archive, 2022.

[Rob91] John Michael Robson. An o (t log t) reduction from ram computations to satisfiability. *Theoretical Computer Science*, 82(1):141–149, 1991.

[SAGL18] Srinath Setty, Sebastian Angel, Trinabh Gupta, and Jonathan Lee. Proving the correct execution of concurrent services in zero-knowledge. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, October 2018.

[Set20] Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Proceedings of the International Cryptology Conference (CRYPTO)*, 2020.

[SL20] Srinath Setty and Jonathan Lee. Quarks: Quadruple-efficient transparent zkSNARKs. Cryptology ePrint Archive, Report 2020/1275, 2020.

[ST23] Srinath Setty and Justin Thaler. BabySpartan: Lasso-based SNARK for non-uniform computation. Cryptology ePrint Archive, 2023.

[STW23] Srinath Setty, Justin Thaler, and Riad Wahby. Customizable constraint systems for succinct arguments. Cryptology ePrint Archive, 2023.

[STW24] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with Lasso. Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT), 2024.

[Tha13] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In *Proceedings of the International Cryptology Conference (CRYPTO)*, 2013.

[Tha22] Justin Thaler. Proofs, arguments, and zero-knowledge. *Foundations and Trends in Privacy and Security*, 4(2–4):117–660, 2022.

[vex25] vex_0x. Tweet regarding cycle counts for 64-bit vs. 32-bit registers. `https://x.com/vex_0x/status/1917658212980891810`, Aug 2025. Accessed: 2025-10-02.

[VSBW13] Victor Vu, Srinath Setty, Andrew J. Blumberg, and Michael Walfish. A hybrid architecture for verifiable computation. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2013.

[WSR+15] Riad S. Wahby, Srinath Setty, Zuocheng Ren, Andrew J. Blumberg, and Michael Walfish. Efficient RAM and control flow in verifiable outsourced computation. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2015.

[WTS+18] Riad S. Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zkSNARKs without trusted setup. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2018.

[Xop] XopMC. secp256k1-x64-avx: very fast (not secure) implementation of arithmetic on curve secp256k1 on x86_64. `https://github.com/XopMC/secp256k1-x64-avx`. README benchmark: Montgomery mul = 49 cycles/op on Intel Core i7-6700 (Skylake); 93 cycles/op on Intel Core i3-2328M (Sandy Bridge).

[YH23] Yibin Yang and David Heath. Two shuffles make a RAM: Improved constant overhead zero knowledge RAM. Cryptology ePrint Archive, Paper 2023/1115, 2023.

[ZBK+22] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. Caulk: Lookup arguments in sublinear time. Cryptology ePrint Archive, 2022.

[Zer23] Tim Zerrell. Using continuations to prove any evm transaction. `https://risczero.com/blog/continuations`, May 22 2023. Accessed: 2025-10-01.

[ZGK+18] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. vRAM: Faster verifiable RAM with program-independent preprocessing. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2018.

[ZGK+22] Arantxa Zapico, Ariel Gabizon, Dmitry Khovratovich, Mary Maller, and Carla Ràfols. Baloo: Nearly optimal lookup arguments. Cryptology ePrint Archive, 2022.

[ZSC24] Jiaxing Zhao, Srinath Setty, and Weidong Cui. MicroNova: Folding-based arguments with efficient (on-chain) verification. Cryptology ePrint Archive, Paper 2024/2099, 2024.